THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms. **R.KIRUTHIGA (20524UCSC056)**, a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Practical Sql Queries**		
15	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO:01 Creation of DDL operations using Recipes Table

DATE: 09/07/2026

Aim

To create a Recipes table and perform DDL operations such as CREATE, ALTER, DROP, and TRUNCATE.

Software Used

Oracle / MySQL /sqlite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects in a database.

DDL COMMANDS:

- Create
- Alter
- Drop
- Truncate

CREATE: Create command is used to create a new table and also use constraints.

Syntax

```
CREATE TABLE recipe_table (
column1 datatype constraints,
column2 datatype constraints,
column3 datatype constraints
);
```

Table Structure

```
SQL> CREATE TABLE recipes (
id INTEGER PRIMARY KEY,
dish_name VARCHAR(100),
cuisine VARCHAR(50),
cook_time_minutes INTEGER);
```

Table Created

ALTER: Alter command is used to alter a table or make some changes in a table. It has methods such as ADD, MODIFY, DROP, RENAME.

ADD: Add command is used to insert a new attribute/column with data type.

Query:

```
SQL> ALTER TABLE recipes ADD calories INTEGER;
SQL> ALTER TABLE recipes ADD price INTEGER;
SQL> ALTER TABLE recipes ADD date_added DATE;
```

Table Altered

MODIFY: Modify command is used to change the data type or size of an existing column.

Query:

```
SQL> ALTER TABLE recipes MODIFY dish_name VARCHAR(200);
```

DROP: Drop command is used to delete an attribute/column from a table.

Query:

```
SQL> ALTER TABLE recipes ADD notes VARCHAR(100);
SQL> ALTER TABLE recipes DROP COLUMN notes;
```

RENAME: Rename command is used to change the table name or a column name.

```
SQL> RENAME TABLE recipes TO dishes;
```

Table renamed.

(OR)

```
ALTER TABLE recipes RENAME TO dishes;
```

Column Rename

```
ALTER TABLE recipes RENAME COLUMN dish_name TO recipe_name;
```

Note: For all the remaining observations, the table is retained with the name recipes.

TRUNCATE: Truncate command is used to delete all values in the table while keeping its structure.

Syntax: TRUNCATE TABLE tablename;

DROP: Deletes the table and all its data permanently.

Syntax: DROP TABLE tablename;

Explanation

- Create a table named recipes.
- Add columns calories, price and date_added using ALTER.
- Insert recipe details into the table.
- Display all recipe records.

Final Table Structure

```
CREATE TABLE recipes (
id INTEGER PRIMARY KEY,
dish_name VARCHAR(100),
cuisine VARCHAR(50),
cook_time_minutes INTEGER,
calories INTEGER,
price INTEGER,
date_added DATE
);
```

Sample Data

```
INSERT INTO recipes VALUES (1,'Masala Dosa','South Indian',45,350,60,'2026-01-10');
INSERT INTO recipes VALUES (2,'Veg Fried Rice','Chinese',25,400,90,'2026-01-12');
INSERT INTO recipes VALUES (3,'Margherita Pizza','Italian',35,600,250,'2026-01-15');
INSERT INTO recipes VALUES (4,'Tomato Omelette','Continental',15,200,50,'2026-01-
18');
INSERT INTO recipes VALUES (5,'Chole Bhature','North Indian',50,550,80,'2026-01-20');
INSERT INTO recipes VALUES (6,'Lemon Rice','South Indian',20,300,40,'2026-01-22');
```

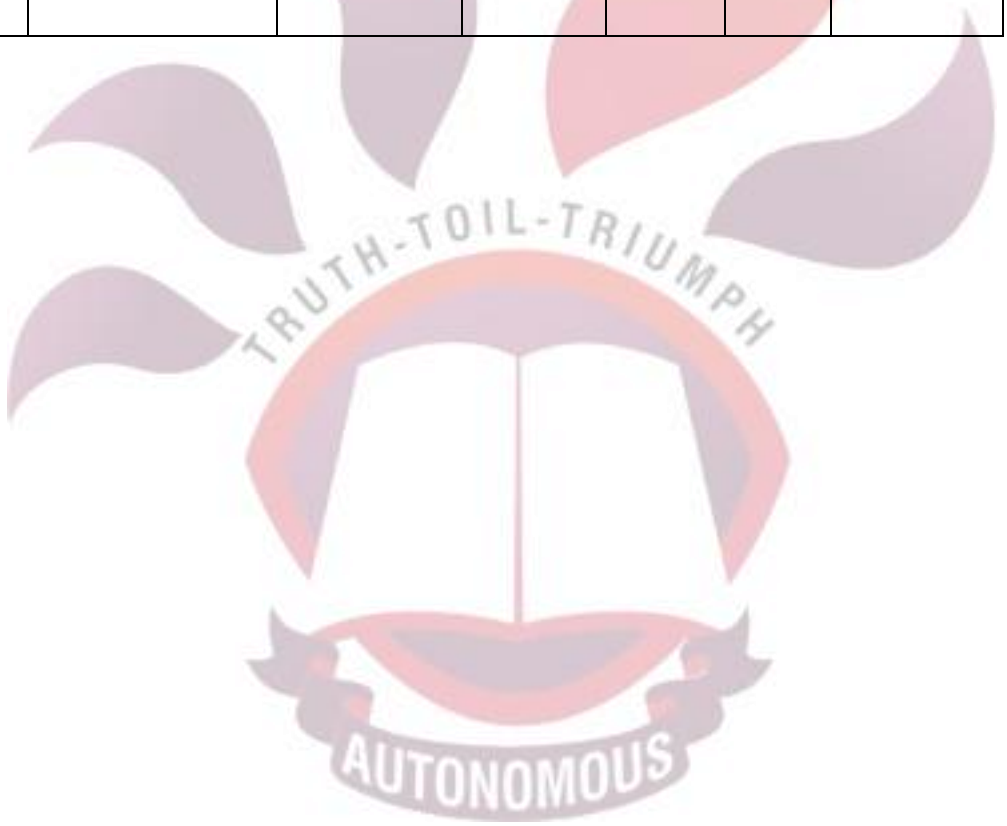
SQL Query

Query: Display all recipe records

```
SQL> SELECT * FROM recipes;
```

Resultant Query

id	dish_name	cuisine	cook_time	calories	price	date_added
1	Masala Dosa	South Indian	45	350	60	2026-01-10
2	Veg Fried Rice	Chinese	25	400	90	2026-01-12
3	Margherita Pizza	Italian	35	600	250	2026-01-15
4	Tomato Omelette	Continental	15	200	50	2026-01-18
5	Chole Bhature	North Indian	50	550	80	2026-01-20
6	Lemon Rice	South Indian	20	300	40	2026-01-22



RESULT:

Thus, the Recipes table was created successfully and the DDL operations CREATE, ALTER, DROP, and TRUNCATE were performed successfully

EX. NO:02 DML operations using Recipes Table

DATE:12/07/2026

Aim

To insert, update, and delete records in the Recipes table using DML commands.

Software Used

Oracle MySQL/sqlite3

Description

DML (Data Manipulation Language) is used to manipulate the data present in the database.

DML COMMANDS:

- Insert
- Update
- Delete

INSERT: Insert command is used to add new records into a table.

Syntax: INSERT INTO table_name VALUES (value1, value2, ...);

UPDATE: Update command is used to modify existing records.

Syntax: UPDATE table_name SET column = value WHERE

condition;

DELETE: Delete command is used to remove records from a table.

Syntax: DELETE FROM table_name WHERE condition;

Explanation

- Insert a new recipe record into the recipes table.
- Update the price of an existing recipe.
- Delete the newly added recipe record.
- Display the table after each operation.

Query 1: Insert a new record

```
SQL> INSERT INTO recipes VALUES (7,'Paneer Butter Masala','North
Indian',40,500,150,'2026-01-25');
SQL> SELECT * FROM recipes;
```

Resultant Query

id	dish_name	cuisine	cook_time	price
1	Masala Dosa	South Indian	45	60
2	Veg Fried Rice	Chinese	25	90
3	Margherita Pizza	Italian	35	250
4	Tomato Omelette	Continental	15	50
5	Chole Bhature	North Indian	50	80
6	Lemon Rice	South Indian	20	40
7	Paneer Butter Masala	North Indian	40	150

Query 2: Update the price of Veg Fried Rice

```
SQL> UPDATE recipes SET price = 100 WHERE id = 2;
SQL> SELECT * FROM recipes WHERE id = 2;
```

Resultant Query

id	dish_name	cuisine	cook_time	price
2	Veg Fried Rice	Chinese	25	100

Query 3: Delete the newly added recipe

```
SQL> DELETE FROM recipes WHERE id = 7;
SQL> SELECT * FROM recipes;
```

Resultant Query

id	dish_name	cuisine	cook_time	price
1	Masala Dosa	South Indian	45	60
2	Veg Fried Rice	Chinese	25	100
3	Margherita Pizza	Italian	35	250
4	Tomato Omelette	Continental	15	50
5	Chole Bhature	North Indian	50	80
6	Lemon Rice	South Indian	20	40

RESULT:

Thus, the DML operations INSERT, UPDATE, and DELETE were performed successfully on the Recipes table.

EX. NO:03 DQL operations using Recipes Table

DATE: 16/07/2026

Aim

To retrieve records from the Recipes table using DQL commands SELECT, WHERE, ORDER BY, GROUP BY, and HAVING.

Software Used

Oracle/MySQL / sqlite3

Description

DQL (Data Query Language) is used to fetch and retrieve data stored in the database.

DQL COMMANDS:

- Select
- Where
- Order By
- Group By
- Having

Explanation

- Display recipes belonging to the South Indian cuisine.
- Sort recipes by price in descending order.
- Group recipes by cuisine and count them.
- Filter cuisines that have more than one recipe.

Query 1: Display recipes of South Indian cuisine

```
SQL> SELECT * FROM recipes WHERE cuisine = 'South Indian';
```

Resultant Query

id	dish_name	cuisine	price
1	Masala Dosa	South Indian	60
6	Lemon Rice	South Indian	40

Query 2: Sort recipes by price (descending)

SQL> SELECT dish_name, price FROM recipes ORDER BY price DESC;

Resultant Query

dish_name	price
Margherita Pizza	250
Chole Bhature	80
Veg Fried Rice	90
Masala Dosa	60
Tomato Omelette	50
Lemon Rice	40

Query 3: Group recipes by cuisine

SQL> SELECT cuisine, COUNT(*) AS total_recipes FROM recipes GROUP BY cuisine;

Resultant Query

cuisine	total_recipes
South Indian	2
Chinese	1
Italian	1
Continental	1
North Indian	1

Query 4: Filter cuisines with more than one recipe

SQL> SELECT cuisine, COUNT(*) AS total_recipes FROM recipes
GROUP BY cuisine HAVING COUNT(*) > 1;

Resultant Query

cuisine	total_recipes
South Indian	2

RESULT:

Thus, records were retrieved successfully from the Recipes table using SELECT, WHERE, ORDER BY, GROUP BY, and HAVING.

EX. NO:04 Transactions using Recipes Table

DATE: 20/07/2026

Aim

To perform transaction control operations COMMIT, ROLLBACK, and SAVEPOINT on the Recipes table.

Software Used

Oracle/MySQL/sqlite3

Description

A transaction is a set of operations performed as a single unit. TCL (Transaction Control Language) commands COMMIT, ROLLBACK, and SAVEPOINT are used to manage transactions.

TCL COMMANDS:

- Commit
- Rollback
- Savepoint

COMMIT: Saves all the changes made in the current transaction permanently.

Syntax: COMMIT;

ROLLBACK: Undoes changes made since the last COMMIT or SAVEPOINT.

Syntax: ROLLBACK [TO savepoint_name];

SAVEPOINT: Marks a point within a transaction to which a later rollback can

occur. Syntax: SAVEPOINT savepoint_name;

Explanation

- Insert a record and commit it permanently.
- Create a savepoint before inserting another record.
- Insert a further record after the savepoint.
- Rollback to the savepoint and verify the change was undone.

Query 1: Insert and commit

SQL> INSERT INTO recipes VALUES (7,'Paneer Butter Masala','North Indian',40,500,150,'2026-01-25');
SQL> COMMIT;

Committed

Query 2: Savepoint before next insert

SQL> SAVEPOINT sp1;
SQL> INSERT INTO recipes VALUES (8,'Veg Manchurian','Chinese',30,350,120,'2026-01-27');

Query 3: Rollback to savepoint

SQL> ROLLBACK TO sp1;
The insert of id 8 (Veg Manchurian) is undone, while id 7 remains since it was already committed.

Query 4: Verify final data

SQL> SELECT * FROM recipes;

Resultant Query

id	dish_name	cuisine	price
1	Masala Dosa	South Indian	60
2	Veg Fried Rice	Chinese	90
3	Margherita Pizza	Italian	250
4	Tomato Omelette	Continental	50
5	Chole Bhature	North Indian	80
6	Lemon Rice	South Indian	40
7	Paneer Butter Masala	North Indian	150



RESULT:

Thus, the transaction control commands COMMIT, ROLLBACK, and SAVEPOINT were executed successfully on the Recipes table.

EX. NO:05 Joins using Recipes and Reviews Table

DATE: 23/07/2026

Aim

To perform Inner, Left, Right, Full, and Self joins using the Recipes and Reviews tables.

Software Used

Oracle / MySQ/sqlite3

Description

A join is used to combine rows from two or more tables based on a related column between them.

JOINS:

- Inner Join
- Left Join
- Right Join
- Full Join
- Self Join

Reviews Table Structure

```
SQL> CREATE TABLE reviews (
review_id INTEGER PRIMARY KEY,
recipe_id INTEGER,
reviewer_name VARCHAR(50),
rating INTEGER
);
```

Sample Data

```
INSERT INTO reviews VALUES (101,1,'Aarav',5);
INSERT INTO reviews VALUES (102,1,'Meera',4);
INSERT INTO reviews VALUES (103,2,'Kiran',3);
INSERT INTO reviews VALUES (104,3,'Divya',5);
INSERT INTO reviews VALUES (105,5,'Aarav',4);
INSERT INTO reviews VALUES (106,6,'Meera',5);
```

Note: The recipe Tomato Omelette (id 4) has no reviews, used to illustrate LEFT/FULL joins.

Query 1: INNER JOIN — recipes with reviews

```
SQL> SELECT r.dish_name, rv.reviewer_name, rv.rating
FROM recipes r INNER JOIN reviews rv ON r.id = rv.recipe_id;
```

Resultant Query

dish_name	reviewer_name	rating
Masala Dosa	Aarav	5
Masala Dosa	Meera	4
Veg Fried Rice	Kiran	3
Margherita Pizza	Divya	5
Chole Bhature	Aarav	4
Lemon Rice	Meera	5

Query 2: LEFT JOIN — all recipes, matched reviews if any

```
SQL> SELECT r.dish_name, rv.reviewer_name, rv.rating
FROM recipes r LEFT JOIN reviews rv ON r.id = rv.recipe_id;
```

Resultant Query

dish_name	reviewer_name	rating
Masala Dosa	Aarav	5
Masala Dosa	Meera	4
Veg Fried Rice	Kiran	3
Margherita Pizza	Divya	5
Tomato Omelette	NULL	NULL
Chole Bhature	Aarav	4
Lemon Rice	Meera	5

Query 3: RIGHT JOIN — all reviews, matched recipes if any

```
SQL> SELECT r.dish_name, rv.reviewer_name, rv.rating
FROM recipes r RIGHT JOIN reviews rv ON r.id = rv.recipe_id;
```

Since every review has a matching recipe in this dataset, the result is the same as the INNER JOIN above.

Query 4: FULL JOIN — all recipes and all reviews

```
SQL> SELECT r.dish_name, rv.reviewer_name, rv.rating
FROM recipes r LEFT JOIN reviews rv ON r.id = rv.recipe_id
UNION
SELECT r.dish_name, rv.reviewer_name, rv.rating
FROM recipes r RIGHT JOIN reviews rv ON r.id = rv.recipe_id;
```

(MySQL/sqlite3 do not support FULL JOIN directly, so it is simulated using UNION of LEFT and RIGHT joins.) Result is the same as Query 2 in this dataset.

Query 5: SELF JOIN — recipes sharing the same cuisine

```
SQL> SELECT a.dish_name AS recipe1, b.dish_name AS recipe2, a.cuisine
FROM recipes a INNER JOIN recipes b
ON a.cuisine = b.cuisine AND a.id < b.id;
```

Resultant Query

recipe1	recipe2	cuisine
Masala Dosa	Lemon Rice	South Indian



RESULT:

Thus, Inner, Left, Right, Full, and Self joins were performed successfully using the Recipes and Reviews tables.

EX. NO: 06 SQL Functions

DATE:27/07/2026

Aim

To apply aggregate, string, and date functions on the Recipes table.

Software Used

Oracle /MySQL/sqlite3

Description

SQL functions perform calculations on data. Aggregate functions summarize data, string functions manipulate text, and date functions manipulate date values.

SQL FUNCTIONS:

- Aggregate Functions
- String Functions
- Date Functions

Query 1: Aggregate Functions

SQL> SELECT COUNT(*) AS total_recipes, AVG(cook_time_minutes) AS avg_time, MAX(price) AS max_price, MIN(price) AS min_price FROM recipes;

Resultant Query

total_recipes	avg_time	max_price	min_price
6	31.7	250	40

Query 2: String Functions

SQL> SELECT UPPER(dish_name) AS dish_upper, LENGTH(dish_name) AS name_length FROM recipes;

Resultant Query

dish_upper	name_length
MASALA DOSA	11
VEG FRIED RICE	14
MARGHERITA PIZZA	17
TOMATO OMELETTE	15
CHOLE BHATURE	13
LEMON RICE	10

Query 3: Date Functions

SQL> SELECT dish_name, date_added, DATEDIFF(CURDATE(), date_added) AS days_since_added
FROM recipes;

(days_since_added is computed against the current system date and will vary with the run date.)

Resultant Query

dish_name	date_added	days_since_added
Masala Dosa	2026-01-10	35
Veg Fried Rice	2026-01-12	33
Margherita Pizza	2026-01-15	30
Tomato Omelette	2026-01-18	27
Chole Bhature	2026-01-20	25
Lemon Rice	2026-01-22	23



RESULT:

Thus, aggregate, string, and date functions were applied successfully on the Recipes table.

EX. NO: 07 Subqueries and Nested Queries

DATE:31/07/2026

Aim

To retrieve records using single-row, multiple-row, and nested subqueries.

Software Used

Oracle / MySQL / sqlite3

Description

A subquery is a query written inside another query. It is used to retrieve data based on the result of another query.

COMMANDS / CONCEPTS:

- Single Row Subquery
- Multiple Row Subquery
- Nested Subquery

SINGLE ROW SUBQUERY

A single-row subquery returns a single value.

Syntax

SELECT column_name

FROM table_name

WHERE column_name = (SELECT column_name FROM table_name WHERE condition);

MULTIPLE ROW SUBQUERY

A multiple-row subquery returns more than one value.

Syntax

SELECT column_name

FROM table_name WHERE

column_name IN

(SELECT column_name FROM table_name WHERE condition);

Explanation

- Find the recipe having the maximum price.
- Display recipes belonging to the cuisine of Masala Dosa.
- Use a subquery inside another query.

Query 1: Find the recipe with maximum price

SQL> SELECT dish_name, price

FROM recipes

WHERE price = (SELECT MAX(price) FROM recipes);

Resultant Query

dish_name	price
Margherita Pizza	250

Query 2: Find recipes from the same cuisine as Masala Dosa

SQL> SELECT dish_name, cuisine FROM
recipes

WHERE cuisine = (SELECT cuisine FROM recipes
WHERE dish_name = 'Masala Dosa');

Resultant Query

dish_name	cuisine
Masala Dosa	South Indian
Lemon Rice	South Indian

Query 3: Multiple-row subquery

```
SQL> SELECT dish_name, price
FROM recipes

WHERE cuisine IN

(SELECT cuisine FROM recipes
WHERE price > 100);
```

Resultant Query

dish_name	price
Margherita Pizza	250
Chole Bhature	80
Paneer Butter Masala	150

RESULT:

Thus, single-row, multiple-row, and nested subqueries were performed successfully on the Recipes table.

EX. NO: 08 Views, Indexes, and Sequences

DATE:03/08/2026

Aim

To create and use views, indexes, and sequences in the database.

Software Used

Oracle / MySQL / sqlite3

Description

Views are virtual tables created from queries. Indexes are used to improve data retrieval. Sequences are used to generate sequential numbers.

COMMANDS / CONCEPTS:

- CREATE VIEW
- CREATE INDEX
- CREATE SEQUENCE

CREATE VIEW

A view is a virtual table based on the result of a query.

Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name

WHERE condition;
```

CREATE INDEX

An index is used to improve the speed of data retrieval.

Syntax

```
CREATE INDEX index_name
ON table_name(column_name);
```

CREATE SEQUENCE

A sequence generates sequential numeric values.

Syntax

```
CREATE SEQUENCE sequence_name
START WITH 1

INCREMENT BY 1;
```

Explanation

- Create a view containing recipe names and prices.
- Create an index on the cuisine column.
- Create a sequence for generating recipe IDs.
- Display the created view.

Query 1: Create a View

```
SQL> CREATE VIEW recipe_view AS
SELECT dish_name, cuisine, price FROM
recipes;
```

Query 2: Display the View

```
SQL> SELECT * FROM recipe_view;
```

Resultant Query

dish_name	cuisine	price
Masala Dosa	South Indian	60
Veg Fried Rice	Chinese	90
Margherita Pizza	Italian	250
Tomato Omelette	Continental	50
Chole Bhature	North Indian	80
Lemon Rice	South Indian	40

Query 3: Create an Index

```
SQL> CREATE INDEX idx_cuisine
ON  recipes(cuisine);
```

Index Created

Query 4: Create a Sequence

```
SQL> CREATE SEQUENCE recipe_seq
START WITH 7

INCREMENT BY 1;
```

Sequence Create

RESULT:

Thus, the view, index, and sequence were created successfully.

EX. NO: 09 Stored Procedures and Functions

DATE:07/08/2026

Aim

To create and execute stored procedures and functions for performing database operations.

Software Used

Oracle

Description

A stored procedure is a group of SQL and PL/SQL statements stored in the database. A function performs an operation and returns a value.

COMMANDS / CONCEPTS:

- CREATE PROCEDURE
- CREATE FUNCTION
- EXECUTE

CREATE PROCEDURE

A procedure is used to perform a specific operation.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name AS
BEGIN
  statements; END;
/
```

CREATE FUNCTION

A function performs an operation and returns a value.

Syntax

```
CREATE OR REPLACE FUNCTION function_name
RETURN datatype

AS
BEGIN

  statements; RETURN
  value;

END;

/
```

Explanation

- Create a procedure to display recipe details.
- Execute the procedure.
- Create a function to return the price of a recipe.
- Execute the function.

Query 1: Create Procedure

```
SQL> CREATE OR REPLACE PROCEDURE show_recipes AS

BEGIN

  FOR r IN (SELECT dish_name, price FROM recipes) LOOP

    DBMS_OUTPUT.PUT_LINE(r.dish_name || ' ' || r.price); END
  LOOP;

END;

/
```

Query 2: Execute Procedure

```
SQL> EXEC show_recipes;
```

Resultant Query / Output

Masala Dosa 60

Veg Fried Rice 90
Margherita Pizza 250

Tomato Omelette 50

Chole Bhature 80

Lemon Rice 40

Query 3: Create Function

```
SQL> CREATE OR REPLACE FUNCTION get_price
(p_id NUMBER)
RETURN NUMBER
AS
  v_price NUMBER;
BEGIN
  SELECT price INTO v_price
  FROM recipes
  WHERE id = p_id;
  RETURN v_price;
END;
/
```

Query 4: Execute Function

```
SQL> SELECT get_price(1) FROM dual;
```

Resultant Query

GET_PRICE(1)

60



RESULT:

Thus, stored procedures and functions were created and executed successfully.

EX. NO: 10 Triggers

DATE:13/08/2026

Aim

To create and execute triggers for INSERT, UPDATE, and DELETE operations.

Software Used

Oracle

Description

A trigger is a stored program that is automatically executed when an INSERT, UPDATE, or DELETE operation occurs on a table.

COMMANDS / CONCEPTS:

- BEFORE Trigger
- AFTER Trigger
- INSERT Trigger
- UPDATE Trigger
- DELETE Trigger

CREATE TRIGGER

Syntax

```
CREATE OR REPLACE TRIGGER trigger_name BEFORE
INSERT OR UPDATE OR DELETE

ON table_name FOR
EACH ROW BEGIN

  statements; END;

/
```

Explanation

- Create a trigger on the Recipes table.
- Execute an operation on the table.
- Automatically display a message when the operation occurs.

Query 1: Create Trigger

```
SQL> CREATE OR REPLACE TRIGGER recipe_insert_trigger BEFORE
INSERT ON recipes

FOR EACH ROW
BEGIN

  DBMS_OUTPUT.PUT_LINE('New recipe is being inserted');
END;

/
```

Trigger Created

Query 2: Insert a record

```
SQL> INSERT INTO recipes

VALUES (7,'Paneer Butter Masala','North Indian',
40,500,150,'2026-01-25');
```

Resultant Query / Output

New recipe is being inserted

Query 3: Display the record

```
SQL> SELECT * FROM recipes WHERE id = 7;
```

Resultant Query

id	dish_name	cuisine	price
7	Paneer Butter Masala	North Indian	150



RESULT:

Thus, the trigger was created and executed successfully during the INSERT operation.

EX. NO: 11 Cursor Implementation

DATE:17/08/2026

Aim

To implement an explicit cursor for retrieving records from the Recipes table.

Software Used

Oracle

Description

A cursor is used to process the rows returned by a query one at a time.

COMMANDS / CONCEPTS:

- Explicit Cursor
- OPEN
- FETCH
- CLOSE

EXPLICIT CURSOR

An explicit cursor is declared and controlled by the programmer.

Syntax

```
DECLARE
  CURSOR cursor_name IS
    SELECT column_name FROM table_name; BEGIN
  OPEN cursor_name;
  FETCH cursor_name INTO variable;
  CLOSE cursor_name;
END;
/
```

Explanation

- Declare a cursor for recipe names.
- Open the cursor.
- Fetch the records one by one.
- Close the cursor.

Query / Program

```
SQL> DECLARE

  CURSOR recipe_cursor IS

    SELECT dish_name, price FROM recipes;

  v_name recipes.dish_name%TYPE;
  v_price recipes.price%TYPE;

BEGIN

  OPEN recipe_cursor;

   LOOP

    FETCH recipe_cursor INTO v_name, v_price; EXIT
    WHEN recipe_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(v_name || ' ' || v_price); END
  LOOP;

  CLOSE recipe_cursor;
END;

/
```

Resultant Query / Output

Masala Dosa 60

Veg Fried Rice 90

Margherita Pizza 250

Tomato Omelette 50

Chole Bhature 80

Lemon Rice 40

RESULT:

Thus, the explicit cursor was declared, opened, fetched, and closed successfully.

EX. NO: 12 PL/SQL Exception Handling

DATE:21/08/2026

Aim

To handle runtime errors using exception handling in PL/SQL.

Software Used

Oracle

Description

Exception handling is used to handle errors that occur during the execution of a PL/SQL program.

COMMANDS / CONCEPTS:

- EXCEPTION
- NO_DATA_FOUND
- TOO_MANY_ROWS
- WHEN OTHERS

EXCEPTION

The EXCEPTION section is used to handle errors in a PL/SQL program.

Syntax

```
BEGIN
  statements;
EXCEPTION
  WHEN exception_name THEN statements;
END;
/
```

Explanation

- Retrieve a recipe using its ID.
- Handle the error if no record is found.
- Display an appropriate message.

Query / Program

```
SQL> DECLARE
  v_name recipes.dish_name%TYPE;
BEGIN
  SELECT dish_name INTO v_name
  FROM recipes
  WHERE id = 100;
  DBMS_OUTPUT.PUT_LINE(v_name);
```

EXCEPTION

```
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No recipe found');
  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('More than one recipe found'); WHEN
  OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred'); END;
/
```

Resultant Query / Output

No recipe found



RESULT:

Thus, the PL/SQL exception handling was performed successfully using predefined exceptions.

EX. NO: 13 Database Normalization (1NF, 2NF, 3NF, BCNF)

DATE:25/08/2026

Aim

To normalize a database table using 1NF, 2NF, 3NF, and BCNF.

Software Used

Oracle / MySQL / sqlite3

Description

Normalization is used to organize data into tables and reduce data redundancy.

COMMANDS / CONCEPTS:

- 1NF
- 2NF
- 3NF
- BCNF

1NF

A table is in 1NF when each column contains atomic values and there are no repeating groups.

2NF

A table is in 2NF when it is in 1NF and every non-key attribute depends on the whole primary key.

3NF

A table is in 3NF when it is in 2NF and there is no transitive dependency.

BCNF

A table is in BCNF when every determinant is a candidate key.

Explanation

- Identify repeating values in the original table.
- Convert the table into 1NF.
- Remove partial dependencies to obtain 2NF.
- Remove transitive dependencies to obtain 3NF.
- Verify the determinant condition for BCNF.

Query 1: Original Table

```
SQL> CREATE TABLE student_course (
  student_id INTEGER,

  student_name VARCHAR(50),
  course_id INTEGER,
  course_name VARCHAR(50),
  instructor VARCHAR(50)

);
```

Query 2: Insert Sample Data

```
SQL> INSERT INTO student_course
VALUES (1,'Arun',101,'Database','Ravi');

SQL> INSERT INTO student_course VALUES
(2,'Meena',102,'Python','Kumar');

SQL> INSERT INTO student_course
VALUES (3,'Kiran',101,'Database','Ravi');
```

Query 3: Student Table

```
SQL> CREATE TABLE students (
  student_id INTEGER PRIMARY KEY,
  student_name VARCHAR(50)
);
```

Query 4: Course Table

```
SQL> CREATE TABLE courses (
  course_id INTEGER PRIMARY KEY,
  course_name VARCHAR(50), instructor
  VARCHAR(50)
);
```

Query 5: Enrollment Table

```
SQL> CREATE TABLE enrollment (
  student_id INTEGER,
  course_id INTEGER,
  PRIMARY KEY (student_id, course_id)
);
```

Resultant Query / Output

StudentsTable

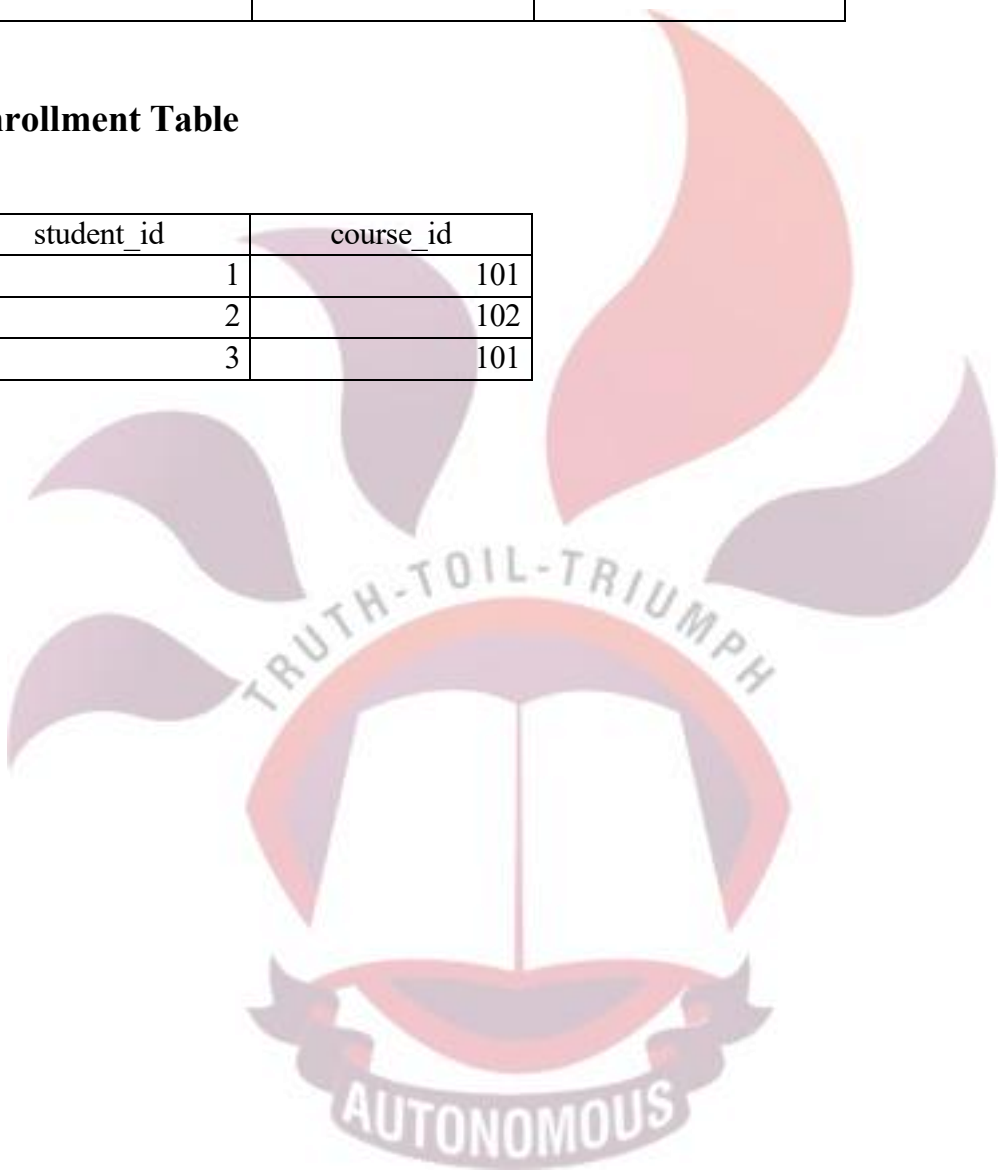
student_id	student_name
1	Arun
2	Meena
3	Kiran

Courses Table

course_id	course_name	instructor
101	Database	Ravi
102	Python	Kumar

Enrollment Table

student_id	course_id
1	101
2	102
3	101



RESULT:

Thus, the database was normalized successfully into 1NF, 2NF, 3NF, and BCNF to reduce data redundancy and improve data organization.

PRACTICAL PROGRAMS:

1. Consider the following tables CABHUB and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:
(i) To display the names of all the white colored vehicles.
(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.
(iii) To display the highest charges at which a vehicle can be hired from CABHUB.
(iv) To display the customer and the corresponding name of the vehicle hired by them.
(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;
(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;
(iii) SELECT COUNT(*), Make FROM CABHUB;
(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

CABHUB

Vcode	VehicleName	Make	Color	Capacity	Charges
100	Innova	Toyota	WHITE	7	15
102	SX4	Suzuki	BLUE	4	14
104	C-Class	Mercedes	RED	4	35
105	A-Star	Suzuki	WHITE	3	14
108	Indigo	Tata	SILVER	3	12

CUSTOMER

Code	CName	VCode
1	HemantSahu	101
2	Raj Lal	108
3	Feroza Shah	105
4	Ketan Dhal	104

SYNTAX:

CREATE TABLE CABHUB(Vcode INT PRIMARY KEY,VehicleName VARCHAR(20),Make VARCHAR(20),Color VARCHAR(10),Capacity INT,Charges INT);

INSERT INTO CABHUB VALUES(100,'Innova','Toyota','WHITE',7,15);

INSERT INTO CABHUB VALUES(102,'SX4','Suzuki','BLUE',4,14);

INSERT INTO CABHUB VALUES(104,'C-Class','Mercedes','RED',4,35);

INSERT INTO CABHUB VALUES(105,'A-Star','Suzuki','WHITE',3,14);

INSERT INTO CABHUB VALUES(108,'Indigo','Tata','SILVER',3,12);

CREATE TABLE CUSTOMER(Code INT PRIMARY KEY,CName VARCHAR(20),VCode INT);

INSERT INTO CUSTOMER VALUES(1,'Hemant Sahu',101);

INSERT INTO CUSTOMER VALUES(2,'Raj Lal',108);

INSERT INTO CUSTOMER VALUES(3,'Feroza Shah',105);

INSERT INTO CUSTOMER VALUES(4,'Ketan Dhal',104);

QUESTION:

(a) Write SQL commands for the following statements:

(i) To display the names of all the white colored vehicles.

INPUT:

SELECT VehicleName FROM CABHUB WHERE Color='WHITE';

OUTPUT:



QUESTION:

(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

INPUT:

SELECT VehicleName,Make,Capacity FROM CABHUB ORDER BY Capacity ASC;

OUTPUT:

Results Explain Describe Saved SQL History

VEHICLENAME	MAKE	CAPACITY
A-Star	Suzuki	3
Indigo	Tata	3
C-Class	Mercedes	4
SX4	Suzuki	4
Innova	Toyota	7

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

INPUT:

SELECT MAX(Charges) FROM CABHUB;

OUTPUT:

Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv) To display the customer and the corresponding name of the vehicle hired by them.

INPUT:

SELECT CUSTOMER.CName,CABHUB.VehicleName FROM CUSTOMER,CABHUB
WHERE CUSTOMER.VCode=CABHUB.VCode;

OUTPUT:

Results Explain Describe Saved SQL History

CNAME	VEHICLENAME
Ketan Dhal	C-Class
Feroza Shah	A-Star
Raj Lal	Indigo

3 rows returned in 0.02 seconds CSV Export

TRIUMPH AUTONOMOUS

QUESTION:

(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

INPUT:

SELECT COUNT(DISTINCT Make) FROM CABHUB;

OUTPUT:



QUESTION:

(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;

INPUT:

SELECT MAX(Charges),MIN(Charges) FROM CABHUB;

OUTPUT:



QUESTION:

(iii) SELECT COUNT(*), Make FROM CABHUB;

INPUT:

SELECT COUNT(*),Make FROM CABHUB GROUP BY Make;

OUTPUT:



Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.02 seconds CSV Export

QUESTION:

(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

INPUT:

SELECT VehicleName FROM CABHUB WHERE Capacity=4;

OUTPUT:



Results Explain Describe Saved SQL History

VEHICLENAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

2. Given the following CO2_EMISSION relation, write SQL commands for questions (a) to (f).

(a) Display all the details of vehicles that use Petrol as the fuel type.

(b) Display the Company and Model of vehicles whose EngineSize is greater than 1.5 litres.

(c) Display the Company, Model, and CO2Emission in ascending order of CO2Emission.

(d) Display the details of vehicles whose CO2Emission is greater than 150 g/km.

(e) Count the number of vehicles that use Diesel as the fuel type.

(f) Insert the following record into the CO2_EMISSION table. 108, 'MG', 'Hector', 'Petrol', 1.5, 160

(g) Write the output of the following SQL commands.

(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;

(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;

(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol';

(iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

VehicleID	Company	Model	FuelType	EngineSize	CO2Emission
101	Toyota	Corolla	Petrol	1.8	165
102	Hyundai	Creta	Diesel	1.5	145
103	Honda	City	Petrol	1.5	152
104	Tata	Nexon	Diesel	1.5	138
105	Maruti	Swift	Petrol	1.2	125
106	Kia	Seltos	Diesel	1.5	150
107	Mahindra	XUV700	Diesel	2.2	185

SYNTAX:

CREATE TABLE CO2_EMISSION(VehicleID INT PRIMARY KEY,Company VARCHAR(20),Model VARCHAR(20),FuelType VARCHAR(10),EngineSize DECIMAL(3,1),CO2Emission INT);

INSERT INTO CO2_EMISSION VALUES(101,'Toyota','Corolla','Petrol',1.8,165);

INSERT INTO CO2_EMISSION VALUES(102,'Hyundai','Creta','Diesel',1.5,145);

INSERT INTO CO2_EMISSION VALUES(103,'Honda','City','Petrol',1.5,152);

INSERT INTO CO2_EMISSION VALUES(104,'Tata','Nexon','Diesel',1.5,138);

INSERT INTO CO2_EMISSION VALUES(105,'Maruti','Swift','Petrol',1.2,125);

INSERT INTO CO2_EMISSION VALUES(106,'Kia','Seltos','Diesel',1.5,150);

INSERT INTO CO2_EMISSION VALUES(107,'Mahindra','XUV700','Diesel',2.2,185);

QUESTION:

(a) Display all the details of vehicles that use Petrol as the fuel type.

INPUT:

SELECT * FROM CO2_EMISSION WHERE FuelType='Petrol';

OUTPUT:



Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
105	Maruti	Swift	Petrol	1.2	125
108	MG	Hector	Petrol	1.5	160

4 rows returned in 0.02 seconds CSV Export

QUESTION:

(b) Display the Company and Model of vehicles whose EngineSize is greater than 1.5 litres.

INPUT:

SELECT Company,Model FROM CO2_EMISSION WHERE EngineSize>1.5;

OUTPUT:



Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla
Mahindra	XUV700

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) Display the Company, Model, and CO2Emission in ascending order of CO2Emission.

INPUT:

SELECT Company,Model FROM CO2_EMISSION WHERE EngineSize>1.5;

OUTPUT:



Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla
Mahindra	XUV700

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) Display the details of vehicles whose CO2Emission is greater than 150 g/km.

INPUT:

SELECT Company,Model,CO2Emission FROM CO2_EMISSION ORDER BY CO2Emission ASC;

OUTPUT:

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
Maruti	Swift	125
Tata	Nexon	138
Hyundai	Creta	145
Kia	Seltos	150
Honda	City	152
MG	Hector	160
Toyota	Corolla	165
Mahindra	XUV700	185

8 rows returned in 0.01 seconds CSV Export

QUESTION:

(e) Count the number of vehicles that use Diesel as the fuel type.

INPUT:

SELECT * FROM CO2_EMISSION WHERE CO2Emission>150;

OUTPUT:

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
107	Mahindra	XUV700	Diesel	2.2	185
108	MG	Hector	Petrol	1.5	160

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(f) Insert the following record into the CO2_EMISSION table. 108, 'MG', 'Hector', 'Petrol', 1.5, 160

INPUT:

INSERT INTO CO2_EMISSION VALUES(108,'MG','Hector','Petrol',1.5,160);

OUTPUT:



QUESTION:

(g) Write the output of the following SQL commands.

(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;

INPUT:

SELECT MAX(CO2Emission) FROM CO2_EMISSION;

OUTPUT:



QUESTION:

(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;

INPUT:

SELECT MIN(EngineSize) FROM CO2_EMISSION;

OUTPUT:



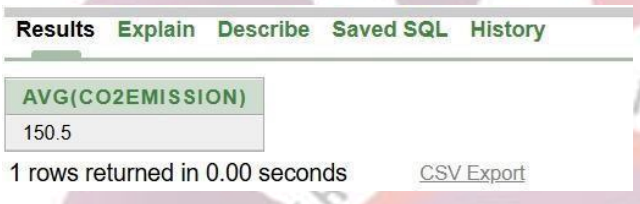
QUESTION:

(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol';

INPUT:

SELECT AVG(CO2Emission) FROM CO2_EMISSION WHERE FuelType='Petrol';

OUTPUT:



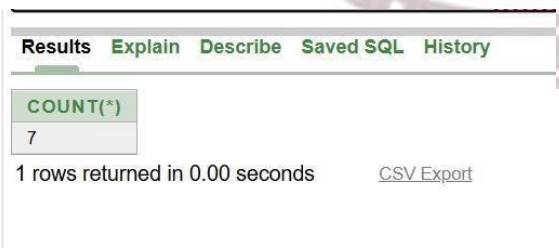
QUESTION:

(iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

INPUT:

SELECT COUNT(*) FROM CO2_EMISSION WHERE EngineSize>=1.5;

OUTPUT:



3. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables APPLICANTS and COURSES.

(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

(b) display the names of applicants, who are paying fee more than 30000.

(c) To display name of all applicants in ascending order of their joinyear.

(d) To display the year and the total number of applicants joined in each YEAR from thetable APPLICANTS.

(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in thecourse from the APPLICANTS table.

(f) To display the applicant's name with their respective course's name from thetables APPLICANTS and COURSES.

(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= '02';

(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= 'M';

(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= 'A01' OR C_ID= 'A05';

(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT (*)=2;

APPLICANTS

NO	NAME	FEE	GENDER	C_ID	JOINYEAR
1012	Amandeep	30000	M	A01	2012
1102	Avisha	25000	F	A02	2009
1103	Ekant	30000	M	A02	2011
1049	Arun	30000	M	A03	2009
1025	Amber	40000	M	A02	2011
1106	Ela	40000	F	A05	2010
1017	Nikita	35000	F	A03	2012
1108	Arleena	30000	F	A03	2012
2109	Shakti	35000	M	A04	2011
1101	Kirat	25000	M	A01	2012

COURSES

C_ID	COURSE
A01	Fashion Design
A02	Networking
A03	Hotel Management
A04	Event Management
A05	Office Management

SYNTAX:

CREATE TABLE APPLICANTS(NO INT PRIMARY KEY,NAME VARCHAR(30),FEE INT,GENDER CHAR(1),C_ID VARCHAR(5),JOINYEAR INT);

INSERT INTO APPLICANTS VALUES(1012,'Amandeep',30000,'M','A01',2012);

INSERT INTO APPLICANTS VALUES(1102,'Avisha',25000,'F','A02',2009);

INSERT INTO APPLICANTS VALUES(1103,'Ekant',30000,'M','A02',2011);

INSERT INTO APPLICANTS VALUES(1049,'Arun',30000,'M','A03',2009);

INSERT INTO APPLICANTS VALUES(1025,'Amber',40000,'M','A02',2011);

INSERT INTO APPLICANTS VALUES(1106,'Ela',40000,'F','A05',2010);

INSERT INTO APPLICANTS VALUES(1017,'Nikita',35000,'F','A03',2012);

INSERT INTO APPLICANTS VALUES(1108,'Arleena',30000,'F','A03',2012);

INSERT INTO APPLICANTS VALUES(2109,'Shakti',35000,'M','A04',2011);

INSERT INTO APPLICANTS VALUES(1101,'Kirat',25000,'M','A01',2012);

CREATE TABLE COURSES(C_ID VARCHAR(5) PRIMARY KEY,COURSE VARCHAR(30));

INSERT INTO COURSES VALUES('A01','Fashion Design');

INSERT INTO COURSES VALUES('A02','Networking');

INSERT INTO COURSES VALUES('A03','Hotel Management');

INSERT INTO COURSES VALUES('A04','Event Management');

INSERT INTO COURSES VALUES('A05','Office Management');

QUESTION:

(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

INPUT:

SELECT NAME,FEE,GENDER,JOINYEAR FROM APPLICANTS WHERE JOINYEAR<2010;

OUTPUT:

Results | Explain | Describe | Saved SQL | History

NAME	FEE	GENDER	JOINYEAR
Avisha	25000	F	2009
Arun	30000	M	2009

2 rows returned in 0.02 seconds CSV Export

QUESTION:

(b) display the names of applicants, who are paying fee more than 30000.

INPUT:

SELECT NAME FROM APPLICANTS WHERE FEE>30000;

OUTPUT:

Results | Explain | Describe | Saved SQL | History

NAME
Amber
Ela
Nikita
Shakti

4 rows returned in 0.02 seconds CSV Export

QUESTION:

(c) To display name of all applicants in ascending order of their joinyear.

INPUT:

SELECT NAME FROM APPLICANTS ORDER BY JOINYEAR ASC;

OUTPUT:



Results Explain Describe Saved SQL History

NAME
Arun
Avisha
Ela
Ekant
Amber
Shakti
Nikita
Arleena
Amandeep
Kirat

10 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display the year and the total number of applicants joined in each YEAR from the table APPLICANTS.

INPUT:

SELECT JOINYEAR,COUNT(*) FROM APPLICANTS GROUP BY JOINYEAR;

OUTPUT:



Results Explain Describe Saved SQL History

JOINYEAR	COUNT(*)
2009	2
2010	1
2011	3
2012	4

4 rows returned in 0.01 seconds CSV Export

QUESTION:

(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.

INPUT:

SELECT C_ID,COUNT(*) FROM APPLICANTS GROUP BY C_ID;

OUTPUT:

Results | Explain | Describe | Saved SQL | History

C_ID	COUNT(*)
A03	3
A04	1
A02	3
A05	1
A01	2

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(f) To display the applicant's name with their respective course's name from thetables APPLICANTS and COURSES.

INPUT:

SELECT APPLICANTS.NAME,COURSES.COURSE FROM APPLICANTS,COURSES WHERE APPLICANTS.C_ID=COURSES.C_ID;

OUTPUT:

Results | Explain | Describe | Saved SQL | History

NAME	COURSE
Amandeep	Fashion Design
Avisha	Networking
Ekant	Networking
Arun	Hotel Management
Amber	Networking
Ela	Office Management
Nikita	Hotel Management
Arleena	Hotel Management
Shakti	Event Management
Kirat	Fashion Design

10 rows returned in 0.01 seconds CSV Export

QUESTION:

(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= '02';

INPUT:

SELECT NAME,JOINYEAR FROM APPLICANTS WHERE GENDER='F' AND C_ID='02';

OUTPUT:



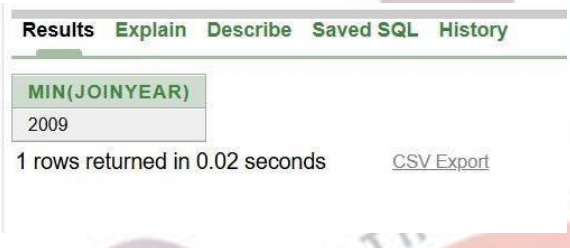
QUESTION:

(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= ‘M’;

INPUT:

SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE GENDER='M';

OUTPUT:



QUESTION:

(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= ‘A01’ OR C_ID= ‘A05’;

INPUT:

SELECT AVG(FEE) FROM APPLICANTS WHERE C_ID='A01' OR C_ID='A05';

OUTPUT:



QUESTION:

(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT (*)=2;

INPUT:

SELECT SUM(FEE),C_ID FROM APPLICANTS GROUP BY C_ID HAVING COUNT(*)=2;

OUTPUT:



4. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables PRODUCTS AND SUPPLIERS.

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

(c) To display the number of products which are supplied by each supplier i.e. the expected output should be S01 2 S02 2 S03 1

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI. (f) To display the name of the companies and the name of the products in descending order of company names.

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

PRODUCTS

PID	SNAME	QTY	PRICE	COMPANY	SUPCODE
101	DIGITAL CAMERA14X	120	12000	RENIX	S01
102	DIGITAL PAD 11i	100	22000	DIGI POP	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01
106	LED SCREEN	70	28000	DISEXPERTS	S02
105	CAR GPS SYSSTEM	60	12000	MOVEON	S03

SUPPLIERS

SUPCODE	SNAME	CITY
S01	GET ALL INC	KOLKATA
S03	EASY MARKET CORP	DELHI
S02	DIGI BUSY GROUP	CHENNAI

SYNTAX:

CREATE TABLE PRODUCTS(PID INT PRIMARY KEY,SNAME VARCHAR(30),QTY INT,PRICE INT,COMPANY VARCHAR(30),SUPCODE VARCHAR(10));
INSERT INTO PRODUCTS VALUES(101,'DIGITAL CAMERA 14X',120,12000,'RENIX','S01');
INSERT INTO PRODUCTS VALUES(102,'DIGITAL PAD 11i',100,22000,'DIGI POP','S02');
INSERT INTO PRODUCTS VALUES(104,'PEN DRIVE 16 GB',500,1100,'STOREKING','S01');
INSERT INTO PRODUCTS VALUES(106,'LED SCREEN',70,28000,'DISEXPERTS','S02');
INSERT INTO PRODUCTS VALUES(105,'CAR GPS SYSTEM',60,12000,'MOVEON','S03');
CREATE TABLE SUPPLIERS(SUPCODE VARCHAR(10) PRIMARY KEY,SNAME VARCHAR(30),CITY VARCHAR(20));
INSERT INTO SUPPLIERS VALUES('S01','GET ALL INC','KOLKATA');
INSERT INTO SUPPLIERS VALUES('S03','EASY MARKET CORP','DELHI');
INSERT INTO SUPPLIERS VALUES('S02','DIGI BUSY GROUP','CHENNAI');

QUESTION:

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).

INPUT:

SELECT * FROM PRODUCTS ORDER BY SNAME;

OUTPUT:

Results Explain Describe Saved SQL History

PID	SNAME	QTY	PRICE	COMPANY	SUPCODE
105	CAR GPS SYSTEM	60	12000	MOVEON	S03
101	DIGITAL CAMERA 14X	120	12000	RENIX	S01
106	LED SCREEN	70	28000	DISEXPERTS	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01

4 rows returned in 0.02 seconds CSV Export

QUESTION:

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

INPUT:

SELECT SNAME,PRICE FROM PRODUCTS WHERE PRICE BETWEEN 10000 AND 15000;

OUTPUT:

Results Explain Describe Saved SQL History

SNAME	PRICE
DIGITAL CAMERA 14X	12000
CAR GPS SYSTEM	12000

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) To display the number of products which are supplied by each supplier i.e. the expected output should be S01 2 S02 2 S03 1

INPUT:

SELECT SUPCODE,COUNT(*) FROM PRODUCTS GROUP BY SUPCODE;

OUTPUT:

Results Explain Describe Saved SQL History

SUPCODE	COUNT(*)
S01	2
S02	1
S03	1

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

INPUT:

SELECT PRICE,SNAME,QTY FROM PRODUCTS WHERE QTY>100;

OUTPUT:

Results Explain Describe Saved SQL History

PRICE	SNAME	QTY
12000	DIGITAL CAMERA 14X	120
1100	PEN DRIVE 16 GB	500

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.

INPUT:

SELECT SNAME FROM SUPPLIERS WHERE CITY='DELHI' OR CITY='CHENNAI';

OUTPUT:

Results Explain Describe Saved SQL History

SNAME
EASY MARKET CORP
DIGI BUSY GROUP

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(f) To display the name of the companies and the name of the products in descending order of company names.

INPUT:

SELECT P.COMPANY,P.SNAME FROM PRODUCTS P ORDER BY P.COMPANY DESC;

OUTPUT:

Results Explain Describe Saved SQL History

COMPANY	SNAME
STOREKING	PEN DRIVE 16 GB
RENIX	DIGITAL CAMERA 14X
MOVEON	CAR GPS SYSTEM
DISEXPERTS	LED SCREEN

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

INPUT:

SELECT DISTINCT SUPCODE FROM PRODUCTS;

OUTPUT:

Results Explain Describe Saved SQL History

SUPCODE
S01
S02
S03

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

INPUT:

SELECT MAX(PRICE),MIN(PRICE) FROM PRODUCTS; INPUT:

OUTPUT:

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
28000	1100

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

INPUT:

SELECT PRICE*QTY AS AMOUNT FROM PRODUCTS WHERE PID=104;

OUTPUT:



Results Explain Describe Saved SQL History

AMOUNT
550000

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

INPUT:

SELECT P.SNAME,S.SNAME FROM PRODUCTS P,SUPPLIERS S WHERE P.SUPCODE=S.SUPCODE AND QTY>100;

OUTPUT:



Results Explain Describe Saved SQL History

SNAME	SNAME
PEN DRIVE 16 GB	GET ALL INC
DIGITAL CAMERA 14X	GET ALL INC

2 rows returned in 0.00 seconds CSV Export

5. Answer the questions (a) and (b) on the basis of the following tables SHOPPE and ACCESSORIES

(a) Write the SQL queries:

(i) To display Name and Price of all the accessories in ascending order of their Price.

(ii) To display Id and SName of all Shoppe in Nehru Place.

(iii) To display Minimum and Maximum Price of each Name of accessories.

(iv) To display Name, Price of all accessories and their respective SName where they are available.

(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

(ii) SELECT Area, COUNT (*) FROM GROUP BY Area;*

(iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;

*(iv) SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN ('S02, 'S03');

SHOPPE

ID	SName	Area
S0001	ABC Computeronics	CP
S0002	All Infotech Media	GK II
S0003	Tech Shoppe	CP
S0004	Greeks Techno Soft	Nehru Place
S0005	Hitech Tech Store	Nehru Place

ACCESSORIES

No	Name	Price	ID
A01	Mother Board	12000	S01
A02	Hard Disk	5000	S01
A03	Keyboard	500	S02
A04	Mouse	300	S01
A05	Mother Board	13000	S02
A06	Keyboard	400	S03
A07	LCD	6000	S04
T08	LCD	5500	S05
T09	Mouse	350	S05
T10	Hard Disk	4500	S03

SYNTAX:

```
CREATE TABLE SHOPPE(ID VARCHAR(10) PRIMARY KEY,SName VARCHAR(30),Area VARCHAR(20));
INSERT INTO SHOPPE VALUES('S0001','ABC Computeronics','CP');
INSERT INTO SHOPPE VALUES('S0002','All Infotech Media','GK II');
INSERT INTO SHOPPE VALUES('S0003','Tech Shoppe','CP');
INSERT INTO SHOPPE VALUES('S0004','Greeks Techno Soft','Nehru Place');
INSERT INTO SHOPPE VALUES('S0005','Hitech Tech Store','Nehru Place');
CREATE TABLE ACCESSORIES(No VARCHAR(10) PRIMARY KEY,Name VARCHAR(30),Price INT,ID VARCHAR(10));
INSERT INTO ACCESSORIES VALUES('A01','Mother Board',12000,'S01');
INSERT INTO ACCESSORIES VALUES('A02','Hard Disk',5000,'S01');
INSERT INTO ACCESSORIES VALUES('A03','Keyboard',500,'S02');
INSERT INTO ACCESSORIES VALUES('A04','Mouse',300,'S01');
INSERT INTO ACCESSORIES VALUES('A05','Mother Board',13000,'S02');
INSERT INTO ACCESSORIES VALUES('A06','Keyboard',400,'S03');
INSERT INTO ACCESSORIES VALUES('A07','LCD',6000,'S04');
INSERT INTO ACCESSORIES VALUES('T08','LCD',5500,'S05');
```

INSERT INTO ACCESSORIES VALUES('T09','Mouse',350,'S05');
INSERT INTO ACCESSORIES VALUES('T10','Hard Disk',4500,'S03');

QUESTION:

(a) Write the SQL queries:
(i) To display Name and Price of all the accessories in ascending order of their Price.

INPUT:

SELECT Name,Price FROM ACCESSORIES ORDER BY Price ASC;

OUTPUT:

Results Explain Describe Saved SQL History

NAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.02 seconds CSV Export

QUESTION:

(ii) To display Id and SName of all Shoppe in Nehru Place.

INPUT:

SELECT ID,SName FROM SHOPPE WHERE Area='Nehru Place';

OUTPUT:

Results Explain Describe Saved SQL History

ID	SNAME
S0004	Greeks Techno Soft
S0005	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display Minimum and Maximum Price of each Name of accessories.

INPUT:

SELECT Name,MIN(Price),MAX(Price) FROM ACCESSORIES GROUP BY Name;

OUTPUT:



Results Explain Describe Saved SQL History

NAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.00 seconds CSV Export

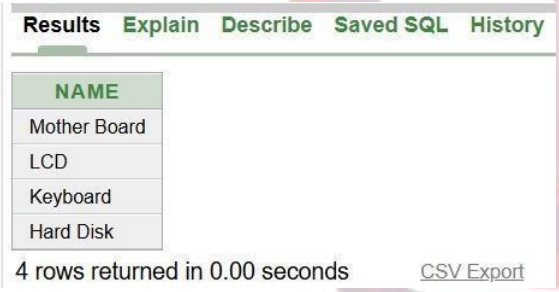
QUESTION:

(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

INPUT:

SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

OUTPUT:



Results Explain Describe Saved SQL History

NAME
Mother Board
LCD
Keyboard
Hard Disk

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) SELECT Area, COUNT (*) FROM GROUP BY Area;*

INPUT:

SELECT Area,COUNT(*) FROM SHOPPE GROUP BY Area;

OUTPUT:



Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;

INPUT:

SELECT COUNT(DISTINCT Area) FROM SHOPPE**;**

OUTPUT:



Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUESTION:

*(iv) SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN (‘S02, ‘S03’);

INPUT:

SELECT Name,Price*0.05 AS DISCOUNT FROM ACCESSORIES WHERE ID IN('S02','S03');

OUTPUT:



Results | Explain | Describe | Saved SQL | History

NAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.00 seconds CSV Export

6. Write SQL commands for (a) to (f) and Write the outputs for (g) on the basis of table HOSPITAL .

(a) To select all the information of patients of cardiology department.

(b) To list the names of female patients who are in ENT department.

(c) To list name of all patients with their date of admission in ascending order.

(d) To display Patient's Name, Charges, Age for only female patients.

(e) To count the number of patients with Age<30.

(f) To inset in a new row in the HOSPITAL table with the following data: 11, "Aftab", 24, "Surgery", {25/02/98}, 300, "M".

(g) Give the output of following SQL statements:

(i) Select COUNT (DISTINCT charges) from HOSPITAL;

(ii) Select MIN (Age) from HOSPITAL where Gender = "F".

(iii) Select SUM (Charges) from HOSPITAL where Department = "ENT".

(iv) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}

No.	Name	Age	Department	Date of Admission	Charges	Gender
1	Arpit	62	Surgery	21/01/98	300	M
2	Zarina	22	ENT	12/12/97	250	F
3	Kareem	32	Orthopedic	19/02/98	200	M
4	Arun	12	Surgery	11/01/98	300	M
5	Zubin	30	ENT	24/02/98	250	M
6	Ketaki	16	ENT	12/01/98	250	M
7	Ankita	29	Cardiology	20/02/98	800	F
8	Zareen	45	Gynecology	22/02/98	300	F
9	Kush	19	Cardiology	13/01/98	800	M
10	Shilpa	23	Nuclear Medicine	21/02/98	400	F

SYNTAX:

CREATE TABLE HOSPITAL(No INT PRIMARY KEY,Name VARCHAR(30),Age INT,Department VARCHAR(30),Datofadm DATE,Charges INT,Gender CHAR(1));
INSERT INTO HOSPITAL VALUES(1,'Arpit',62,'Surgery',DATE '1998-01-21',300,'M');
INSERT INTO HOSPITAL VALUES(2,'Zarina',22,'ENT',DATE '1997-12-12',250,'F');
INSERT INTO HOSPITAL VALUES(3,'Kareem',32,'Orthopedic',DATE '1998-02-19',200,'M');
INSERT INTO HOSPITAL VALUES(4,'Arun',12,'Surgery',DATE '1998-01-11',300,'M');
INSERT INTO HOSPITAL VALUES(5,'Zubin',30,'ENT',DATE '1998-02-24',250,'M');
INSERT INTO HOSPITAL VALUES(6,'Ketaki',16,'ENT',DATE '1998-01-12',250,'M');
INSERT INTO HOSPITAL VALUES(7,'Ankita',29,'Cardiology',DATE '1998-02-20',800,'F');
INSERT INTO HOSPITAL VALUES(8,'Zareen',45,'Gynecology',DATE '1998-02-22',300,'F');
INSERT INTO HOSPITAL VALUES(9,'Kush',19,'Cardiology',DATE '1998-01-13',800,'M');
INSERT INTO HOSPITAL VALUES(10,'Shilpa',23,'Nuclear Medicine',DATE '1998-02-21',400,'F');

QUESTION:

(a) To select all the information of patients of cardiology department.

INPUT:

SELECT * FROM HOSPITAL WHERE Department='Cardiology';

OUTPUT:

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFADMISSION	CHARGES	GENDER
7	Ankita	29	Cardiology	20-FEB-98	800	F
9	Kush	19	Cardiology	13-JAN-98	800	M

2 rows returned in 0.01 seconds CSV Export

QUESTION:

(b) To list the names of female patients who are in ENT department.

INPUT:

SELECT Name FROM HOSPITAL WHERE Gender='F' AND Department='ENT';

OUTPUT:

Results Explain Describe Saved SQL History

NAME
Zarina

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) To list name of all patients with their date of admission in ascending order.

INPUT:

SELECT Name,DateofAdmission FROM HOSPITAL ORDER BY DateofAdmission ASC;

OUTPUT:

Results Explain Describe Saved SQL History

NAME	DATEOFADMISSION
Zarina	12-DEC-97
Arun	11-JAN-98
Ketaki	12-JAN-98
Kush	13-JAN-98
Arpit	21-JAN-98
Kareem	19-FEB-98
Ankita	20-FEB-98
Shilpa	21-FEB-98
Zareen	22-FEB-98
Zubin	24-FEB-98

10 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display Patient's Name, Charges, Age for only female patients.

INPUT:

SELECT Name,Charges,Age FROM HOSPITAL WHERE Gender='F';

OUTPUT:

Results Explain Describe Saved SQL History

NAME	CHARGES	AGE
Zarina	250	22
Ankita	800	29
Zareen	300	45
Shilpa	400	23

4 rows returned in 0.00 seconds CSV Export

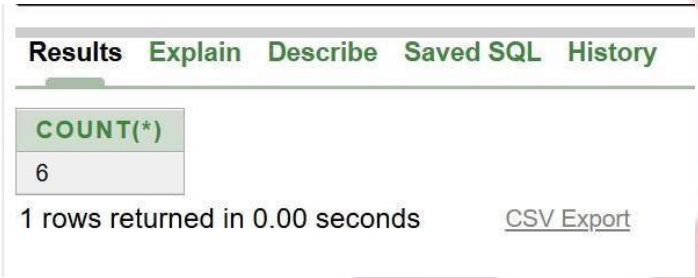
QUESTION:

(e) To count the number of patients with Age<30.

INPUT:

SELECT COUNT(*) FROM HOSPITAL WHERE Age<30;

OUTPUT:



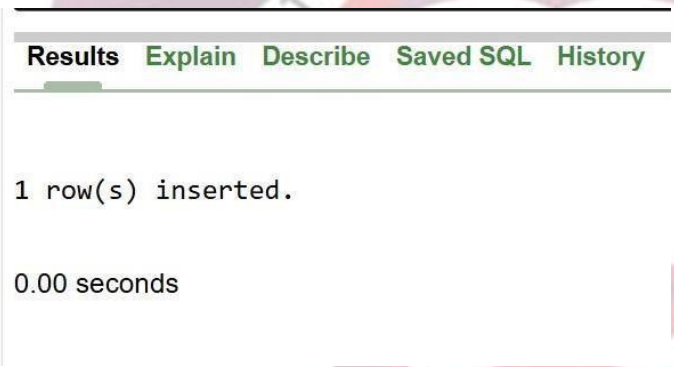
QUESTION:

(f) To inset in a new row in the HOSPITAL table with the following data: 11, “Aftab”, 24, “Surgery”, {25/02/98}, 300, “M”.

INPUT:

INSERT INTO HOSPITAL VALUES(11,'Aftab',24,'Surgery',DATE '1998-02-25',300,'M');

OUTPUT:



QUESTION:

(g) Give the output of following SQL statements:

(i) Select COUNT (DISTINCT charges) from HOSPITAL;

INPUT:

SELECT COUNT(DISTINCT Charges) FROM HOSPITAL;

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(DISTINCTCHARGES)
5

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) Select MIN (Age) from HOSPITAL where Gender = "F".

INPUT:

SELECT MIN(Age) FROM HOSPITAL WHERE Gender='F';

OUTPUT:

Results Explain Describe Saved SQL History

MIN(AGE)
22

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) Select SUM (Charges) from HOSPITAL where Department = "ENT".

INPUT:

SELECT SUM(Charges) FROM HOSPITAL WHERE Department='ENT';

OUTPUT:

Results Explain Describe Saved SQL History

SUM(CHARGES)
750

1 rows returned in 0.00 seconds CSV Export

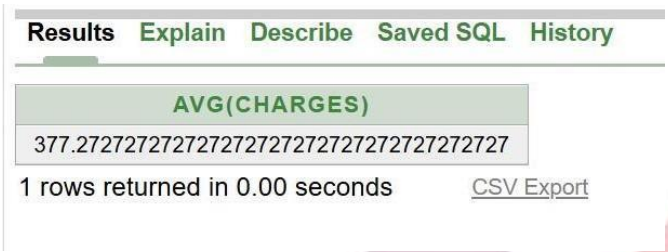
QUESTION:

(iv) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}

INPUT:

SELECT AVG(Charges) FROM HOSPITAL WHERE DateofAdmission<TO_DATE('12-AUG-1998','DD-MON-YYYY');

OUTPUT:



7. Answer the questions (a) and (b) on the basis of the following tables STORE and ITEM

(a) Write the SQL queries (i) to (iv):

(i) To display IName and Price of all the items in ascending order of their Price.

(ii) To display SNo and SName of all store location in CP.

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

(iv) To display IName, Price of all items and their respective SName where they areavailable.

(b) Write the output of the following SQL commands (i) to (iv):

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;

(iii) SELECT COUNT (DISTINCT Area) FROM STORE:

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

STORE

SNo	SName	Area
S01	ABC Computronics	GK II
S02	All InfoTech Media	CP
S03	Tech Shoppe	Nehru Place
S04	Geeks Techno Soft	Nehru Place
S05	Hitech Tech Store	CP

ITEM

INo	IName	Price	SNo
T01	Mother Board	12000	S01
T02	Hard Disk	5000	S01
T03	Keyboard	500	S02
T04	Mouse	300	S01
T05	Mother Board	13000	S02
T06	Keyboard	400	S03
T07	LCD	6000	S04
T08	LCD	5500	S05
T09	Mouse	350	S05
T10	Hard Disk	4500	S03

SYNTAX:

```
CREATE TABLE STORE(SNo VARCHAR(10) PRIMARY KEY,SName VARCHAR(30),Area VARCHAR(20));
INSERT INTO STORE VALUES('S01','ABC Computeronics','GK II');
INSERT INTO STORE VALUES('S02','All InfoTech Media','CP');
INSERT INTO STORE VALUES('S03','Tech Shoppe','Nehru Place');
INSERT INTO STORE VALUES('S04','Geeks Techno Soft','Nehru Place');
INSERT INTO STORE VALUES('S05','Hitech Tech Store','CP');
CREATE TABLE ITEM(INo VARCHAR(10) PRIMARY KEY,IName VARCHAR(30),Price INT,SNo VARCHAR(10));
INSERT INTO ITEM VALUES('T01','Mother Board',12000,'S01');
INSERT INTO ITEM VALUES('T02','Hard Disk',5000,'S01');
INSERT INTO ITEM VALUES('T03','Keyboard',500,'S02');
INSERT INTO ITEM VALUES('T04','Mouse',300,'S01');
INSERT INTO ITEM VALUES('T05','Mother Board',13000,'S02');
INSERT INTO ITEM VALUES('T06','Keyboard',400,'S03');
```

```
INSERT INTO ITEM VALUES('T07','LCD',6000,'S04');
INSERT INTO ITEM VALUES('T08','LCD',5500,'S05');
INSERT INTO ITEM VALUES('T09','Mouse',350,'S05');
INSERT INTO ITEM VALUES('T10','Hard Disk',4500,'S03');
```

QUESTION:

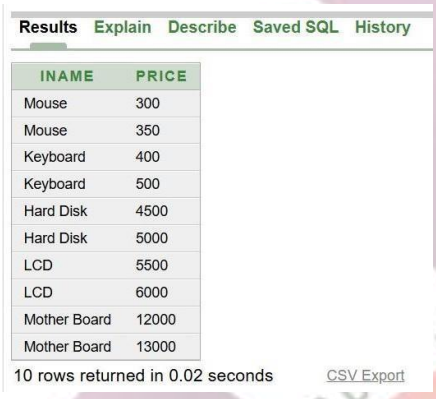
(a) Write the SQL queries (i) to (iv):

(i) To display IName and Price of all the items in ascending order of their Price.

INPUT:

```
SELECT IName,Price FROM ITEM ORDER BY Price ASC;
```

OUTPUT:



Results | Explain | Describe | Saved SQL | History

INAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.02 seconds CSV Export

QUESTION:

(ii) To display SNo and SName of all store location in CP.

INPUT:

```
SELECT SNo,SName FROM STORE WHERE Area='CP';
```

OUTPUT:



Results | Explain | Describe | Saved SQL | History

SNO	SNAME
S02	All InfoTech Media
S05	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

INPUT:

SELECT IName,MIN(Price),MAX(Price) FROM ITEM GROUP BY IName;

OUTPUT:

Results Explain Describe Saved SQL History

INAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.01 seconds CSV Export

QUESTION:

(iv) To display IName, Price of all items and their respective SName where they are available.

INPUT:

SELECT I.IName,I.Price,S.SName FROM ITEM I,STORE S WHERE I.SNo=S.SNo;

OUTPUT:

Results Explain Describe Saved SQL History

INAME	PRICE	SNAME
Mother Board	12000	ABC Computeronics
Hard Disk	5000	ABC Computeronics
Keyboard	500	All InfoTech Media
Mouse	300	ABC Computeronics
Mother Board	13000	All InfoTech Media
Keyboard	400	Tech Shoppe
LCD	6000	Geeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.02 seconds CSV Export

QUESTION:

(b) Write the output of the following SQL commands (i) to (iv):

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

INPUT:

SELECT DISTINCT IName FROM ITEM WHERE Price>=5000;

OUTPUT:



Results Explain Describe Saved SQL History

INAME
Mother Board
LCD
Hard Disk

3 rows returned in 0.01 seconds CSV Export

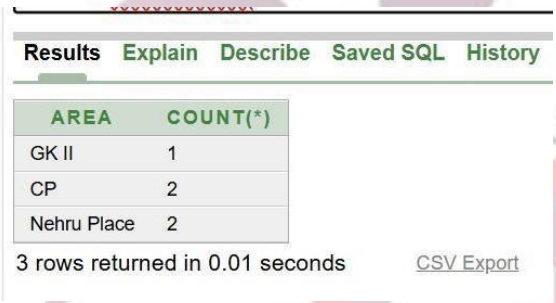
QUESTION:

(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;

INPUT:

SELECT Area,COUNT(*) FROM STORE GROUP BY Area;

OUTPUT:



Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.01 seconds CSV Export

QUESTION:

(iii) SELECT COUNT (DISTINCT Area) FROM STORE:

INPUT:

SELECT COUNT(DISTINCT Area) FROM STORE;

OUTPUT:



Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

INPUT:

```
SELECT IName,Price*0.05 AS DISCOUNT FROM ITEM WHERE SNo IN('S02','S03');
```

OUTPUT:

Results | Explain | Describe | Saved SQL | History

INAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.02 seconds CSV Export

8. Write SQL commands for (a) to (f) and write output for (g) on the basis of Teacher relation given below:

(a) To show all information about the teacher of history department
(b) To list the names of female teacher who are in Hindi department
(c) To list names of all teachers with their date of joining in ascending order.
(d) To display teacher's Name, Salary, Age for male teacher only
(e) To count the number of teachers with Age>23.
(f) To inset a new row in the TEACHER table with the following data: 9, "Raja", 26, "Computer", {13/05/95}, 2300, "M"
(g) Give the output of following SQL statements:
(i) Select COUNT (distinct department) from TEACHER;
(II) Select MAX (Age) from TEACHER where Gender= "F"
(iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96};
(iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};

No.	Name	Age	Department	Date of Join	Salary	Gender
1	Jugal	34	Computer	10/01/97	12000	M
2	Sharmila	31	History	24/03/98	20000	F
3	Sandeep	32	Maths	12/12/96	30000	M
4	Sangeeta	35	History	01/07/99	40000	F
5	Rakesh	42	Maths	05/09/97	25000	M
6	Shyam	50	History	27/06/98	30000	M
7	Shiv Om	44	Computer	25/02/97	21000	M
8	Shalakha	33	Maths	31/07/97	20000	F

SYNTAX:

CREATE TABLE TEACHER(No INT PRIMARY KEY,Name VARCHAR(20),Age INT,Department VARCHAR(20),DateofJoin DATE,Salary INT,Gender CHAR(1));

INSERT INTO TEACHER VALUES(1,'Jugal',34,'Computer',TO_DATE('1997-01-10','YYYY-MM-DD'),12000,'M');

INSERT INTO TEACHER VALUES(2,'Sharmila',31,'History',TO_DATE('1998-03-24','YYYY-MM-DD'),20000,'F');

INSERT INTO TEACHER VALUES(3,'Sandeep',32,'Maths',TO_DATE('1996-12-12','YYYY-MM-DD'),30000,'M');

INSERT INTO TEACHER VALUES(4,'Sangeeta',35,'History',TO_DATE('1999-07-01','YYYY-MM-DD'),40000,'F');

INSERT INTO TEACHER VALUES(5,'Rakesh',42,'Maths',TO_DATE('1997-09-05','YYYY-MM-DD'),25000,'M');

INSERT INTO TEACHER VALUES(6,'Shyam',50,'History',TO_DATE('1998-06-27','YYYY-MM-DD'),30000,'M');

INSERT INTO TEACHER VALUES(7,'Shiv Om',44,'Computer',TO_DATE('1997-02-25','YYYY-MM-DD'),21000,'M');

INSERT INTO TEACHER VALUES(8,'Shalakha',33,'Maths',TO_DATE('1997-07-31','YYYY-MM-DD'),20000,'F');

INSERT INTO TEACHER VALUES(9,'Raja',26,'Computer',TO_DATE('1995-05-13','YYYY-MM-DD'),2300,'M');

QUESTION:

(a) To show all information about the teacher of history department

INPUT:

SELECT * FROM TEACHER WHERE Department='History';

OUTPUT:

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFJOIN	SALARY	GENDER
2	Sharmila	31	History	24-MAR-98	20000	F
4	Sangeeta	35	History	01-JUL-99	40000	F
6	Shyam	50	History	27-JUN-98	30000	M

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(b) To list the names of female teacher who are in Hindi department

INPUT:

SELECT Name FROM TEACHER WHERE Gender='F' AND Department='Hindi'

OUTPUT:



QUESTION:

(c) To list names of all teachers with their date of joining in ascending order.

INPUT:

SELECT Name,DateofJoin FROM TEACHER ORDER BY DateofJoin ASC;

OUTPUT:

Results Explain Describe Saved SQL History

NAME	DATEOFJOIN
Raja	13-MAY-95
Sandeep	12-DEC-96
Jugal	10-JAN-97
Shiv Om	25-FEB-97
Shalakha	31-JUL-97
Rakesh	05-SEP-97
Sharmila	24-MAR-98
Shyam	27-JUN-98
Sangeeta	01-JUL-99

9 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display teacher's Name, Salary, Age for male teacher only

INPUT:

SELECT Name,Salary,Age FROM TEACHER WHERE Gender='M';

OUTPUT:

Results Explain Describe Saved SQL History

NAME	SALARY	AGE
Jugal	12000	34
Sandeep	30000	32
Rakesh	25000	42
Shyam	30000	50
Shiv Om	21000	44
Raja	2300	26

6 rows returned in 0.02 seconds CSV Export

QUESTION:

(e) To count the number of teachers with Age>23.

INPUT:

SELECT COUNT(*) FROM TEACHER WHERE Age>23;

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(*)
9

1 rows returned in 0.00 seconds CSV Export

TRUTH-TOIL-TRIUMPH AUTONOMOUS

QUESTION:

(f) To inset a new row in the TEACHER table with the following data: 9, “Raja”, 26, “Computer”, {13/05/95}, 2300, “M”

INPUT:

SELECT COUNT(DISTINCT Department) FROM TEACHER;

OUTPUT:



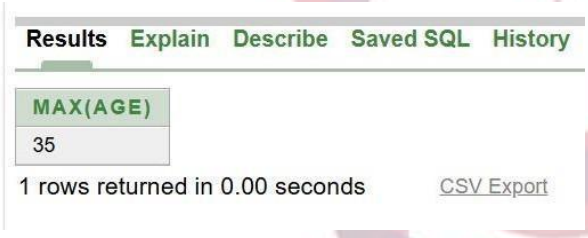
QUESTION:

(g) Give the output of following SQL statements:
(i) Select COUNT (distinct department) from TEACHER;

INPUT:

SELECT MAX(Age) FROM TEACHER WHERE Gender='F';

OUTPUT:



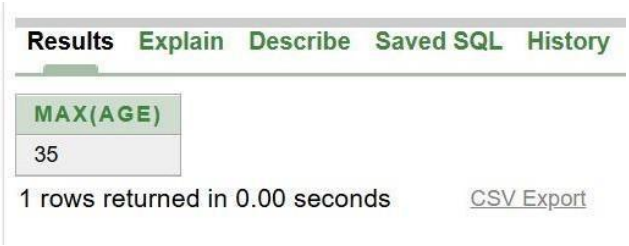
QUESTION:

(II) Select MAX (Age) from TEACHER where Gender= “F”

INPUT:

SELECT MAX(Age) FROM TEACHER WHERE Gender='F';

OUTPUT:



QUESTION:

(iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96};

INPUT:

SELECT AVG(Salary) FROM TEACHER WHERE DateofJoin<TO_DATE('1996-07-12','YYYY-MM-DD');

OUTPUT:



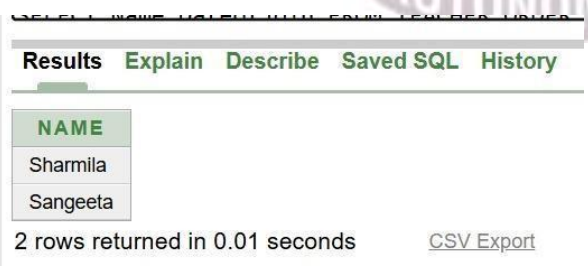
QUESTION:

(iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};

INPUT:

SELECT SUM(Salary) FROM TEACHER WHERE DateofJoin<TO_DATE('1996-07-12','YYYY-MM-DD');

OUTPUT:



9. Given the following tables for a database LIBRARY: Write SQL queries for (a) to (f):

(a) To show Book name, Author name and Price of books of EPB publishers.

(b) To list the names of the books of Fiction Type.

(c) To display the names and price of the books in descending order of their price.

(d) To increase the price of all books of first publisher by 50.

(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).

(f) To insert a new row in the table Issued following the data: "F0002",4.

(g) Give the output of the following queries based on the above tables:

(i) SELECT COUNT(DISTINCT Publishers) FROM Books;

(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;

(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;

(iv) SELECT COUNT (*) FROM Books;

Books

Book Id	Book Name	Author Name	Publishers	Price	Type	Qty.
F0001	The Tears	William Hopkins	First Publ.	750	Fiction	10
F0002	Thunderbolts	Anna Roberts	First Publ.	700	Fiction	5
T0001	My First C++	Briian & Brooke	EPB	250	Text	10
T0002	C++ Brainworks	A. W. Rossaine	TDH	325	Text	5
C0001	Fast Cook	LataKapoor	EPB	350	Cookery	8

Issued

Book Id	Quantity Issued
F0001	3
T0001	1
C0001	5

SYNTAX:

```
CREATE TABLE Books(Book_Id VARCHAR(10) PRIMARY KEY,Book_Name
VARCHAR(30),Author_Name VARCHAR(30),Publishers VARCHAR(20),Price INT,Type
VARCHAR(20),Qty INT);
INSERT INTO Books VALUES('F0001','The Tears','William Hopkins','First
Publ.',750,'Fiction',10);
INSERT INTO Books VALUES('F0002','Thunderbolts','Anna Roberts','First
Publ.',700,'Fiction',5);
```

```
INSERT INTO Books VALUES('T0001','My First C++','Brian & Brooke','EPB',250,'Text',10);
INSERT INTO Books VALUES('T0002','C++ Brainworks','A. W. Rossaine','TDH',325,'Text',5);
INSERT INTO Books VALUES('C0001','Fast Cook','Lata Kapoor','EPB',350,'Cookery',8);
CREATE TABLE Issued(Book_Id VARCHAR(10),Quantity_Issued INT);
INSERT INTO Issued VALUES('F0001',3);
INSERT INTO Issued VALUES('T0001',1);
INSERT INTO Issued VALUES('C0001',5);
SELECT Book_Name,Author_Name,Price FROM Books WHERE Publishers='EPB';
SELECT Book_Name FROM Books WHERE Type='Fiction';
SELECT Book_Name,Price FROM Books ORDER BY Price DESC;
UPDATE Books SET Price=Price+50 WHERE Publishers='First Publ.';
SELECT B.Book_Id,B.Book_Name,I.Quantity_Issued FROM Books B,Issued I WHERE
B.Book_Id=I.Book_Id;
INSERT INTO Issued VALUES('F0002',4);
SELECT COUNT(DISTINCT Publishers) FROM Books;
SELECT SUM(Price) FROM Books WHERE Qty>5;
SELECT Book_Name,Author_Name FROM Books WHERE Price<500;
SELECT COUNT(*) FROM Books;
```

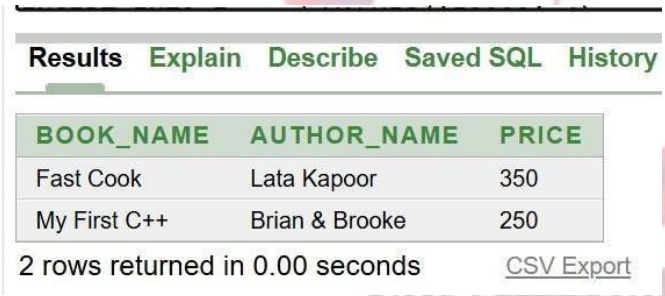
QUESTION:

(a) To show Book name, Author name and Price of books of EPB publishers.

INPUT:

```
SELECT Book_Name,Author_Name,Price FROM Books WHERE Publishers='EPB';
```

OUTPUT:



Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME	PRICE
Fast Cook	Lata Kapoor	350
My First C++	Brian & Brooke	250

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(b) To list the names of the books of Fiction Type.

INPUT:

```
SELECT Book_Name FROM Books WHERE Type='Fiction';
```

OUTPUT:



Results Explain Describe Saved SQL History

BOOK_NAME
The Tears
Thunderbolts

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) To display the names and price of the books in descending order of their price.

INPUT:

SELECT Book_Name,Price FROM Books ORDER BY Price DESC;

OUTPUT:



Results Explain Describe Saved SQL History

BOOK_NAME	PRICE
The Tears	750
Thunderbolts	700
Fast Cook	350
C++ Brainworks	325
My First C++	250

5 rows returned in 0.00 seconds CSV Export

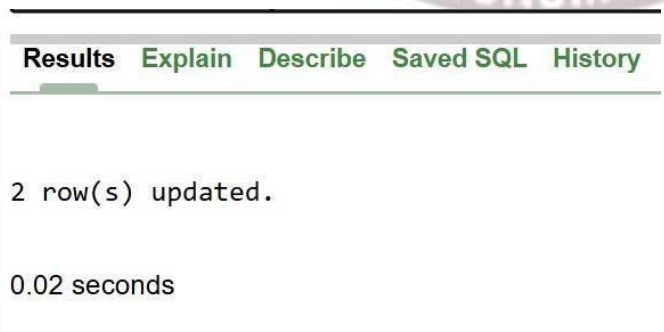
QUESTION:

(d) To increase the price of all books of first publisher by 50.

INPUT:

UPDATE Books SET Price=Price+50 WHERE Publishers='First Publ.';

OUTPUT:



Results Explain Describe Saved SQL History

2 row(s) updated.

0.02 seconds

QUESTION:

(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).

INPUT:

SELECT B.Book_Id,B.Book_Name,I.Quantity_Issued FROM Books B,Issued I WHERE B.Book_Id=I.Book_Id;

OUTPUT:

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	QUANTITY_ISSUED
C0001	Fast Cook	5
F0001	The Tears	3
F0001	The Tears	3
T0001	My First C++	1

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(f) To insert a new row in the table Issued following the data: “F0002”,4.

INPUT:

INSERT INTO Issued VALUES('F0002',4);

OUTPUT:

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

QUESTION:

(g) Give the output of the following queries based on the above tables:
(i) SELECT COUNT(DISTINCT Publishers) FROM Books;

INPUT:

SELECT COUNT(DISTINCT Publishers) FROM Books;

OUTPUT:



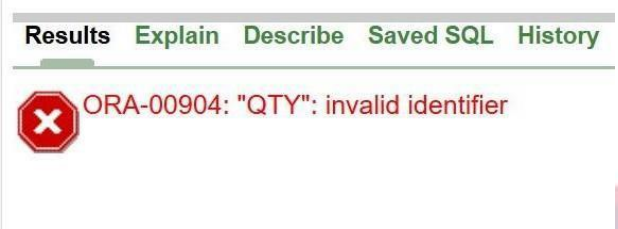
QUESTION:

(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;

INPUT:

SELECT SUM(Price) FROM Books WHERE Qty>5;

OUTPUT:



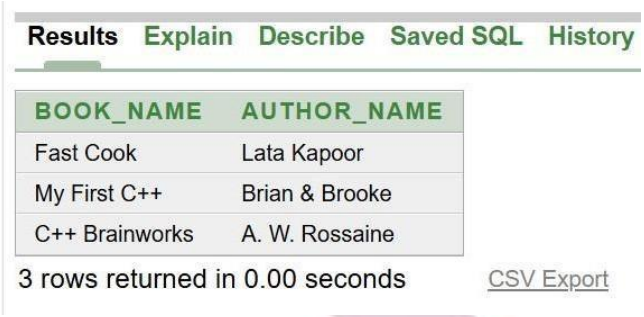
QUESTION:

(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;

INPUT:

SELECT Book_Name,Author_Name FROM Books WHERE Price<500;

OUTPUT:



Results | Explain | Describe | Saved SQL | History

BOOK_NAME	AUTHOR_NAME
Fast Cook	Lata Kapoor
My First C++	Brian & Brooke
C++ Brainworks	A. W. Rossaine

3 rows returned in 0.00 seconds CSV Export

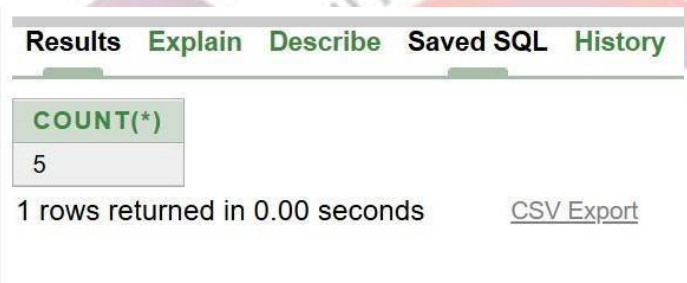
QUESTION:

(iv) SELECT COUNT (*) FROM Books;

INPUT:

SELECT COUNT(*) FROM Books;

OUTPUT:



Results | Explain | Describe | Saved SQL | History

COUNT(*)
5

1 rows returned in 0.00 seconds CSV Export

10. Consider the following tables CARDEN and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the name of all the SILVER colored cars.

(ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

(iv) To display the customer name and the corresponding name of the cards hired by them. (b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

(iii) SELECT COUNT (*), Make FROM CARDEN;

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

CARDEN

Ccode	CarName	Make	Color	Capacity	Charges
501	A-star	Suzuki	RED	3	14
503	Indigo	Tata	SILVER	3	12
502	Innova	Toyota	WHITE	7	15
509	SX4	Suzuki	SILVER	4	14
510	C-Class	Mercedes	RED	4	35

CUSTOMER

CCode	Cname	Ccode (Car Code)
1001	HamantSahu	501
1002	Raj Lal	509
1003	Feroja Shah	503
1004	Ketan Dhal	502

SYNTAX:

CREATE TABLE CARDEN (Ccode INT PRIMARY KEY, CarName VARCHAR(20), Make VARCHAR(20), Color VARCHAR(10), Capacity INT, Charges INT);
INSERT INTO CARDEN VALUES (501, 'A-star', 'Suzuki', 'RED', 3, 14);
INSERT INTO CARDEN VALUES (503, 'Indigo', 'Tata', 'SILVER', 3, 12);
INSERT INTO CARDEN VALUES (502, 'Innova', 'Toyota', 'WHITE', 7, 15);
INSERT INTO CARDEN VALUES (509, 'SX4', 'Suzuki', 'SILVER', 4, 14);
INSERT INTO CARDEN VALUES (510, 'C-Class', 'Mercedes', 'RED', 4, 35);
CREATE TABLE CUSTOMER (CCCode INT PRIMARY KEY, CName VARCHAR(30), Ccode INT);
INSERT INTO CUSTOMER VALUES (1001, 'HemantSahu', 501);
INSERT INTO CUSTOMER VALUES (1002, 'Raj Lal', 509);
INSERT INTO CUSTOMER VALUES (1003, 'Feroja Shah', 503);
INSERT INTO CUSTOMER VALUES (1004, 'Ketan Dhal', 502);

QUESTION:

(a) Write SQL commands for the following statements:
(i) To display the name of all the SILVER colored cars.

INPUT:

SELECT CarName FROM CARDEN WHERE Color = 'SILVER';

OUTPUT:

Results | Explain | Describe | Saved SQL | History

CARNAME
Indigo
SX4

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

INPUT:

SELECT CarName, Make, Capacity FROM CARDEN ORDER BY Capacity DESC;

OUTPUT:

Results | Explain | Describe | Saved SQL | History

CARNAME	MAKE	CAPACITY
Innova	Toyota	7
C-Class	Mercedes	4
SX4	Suzuki	4
Indigo	Tata	3
A-star	Suzuki	3

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

INPUT:

SELECT MAX(Charges) FROM CARDEN;

OUTPUT:



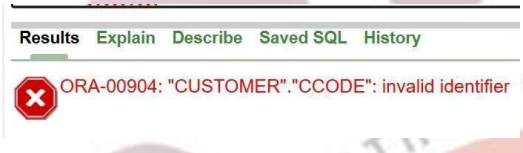
QUESTION:

(iv) To display the customer name and the corresponding name of the cards hired by them.

INPUT:

SELECT CUSTOMER.CName,CARDEN.CarName FROM CUSTOMER,CARDEN WHERE CUSTOMER.Ccode=CARDEN.Ccode;

OUTPUT:



QUESTION:

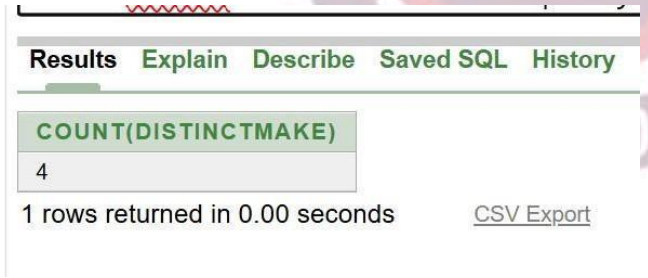
(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

INPUT:

SELECT COUNT(DISTINCT Make) FROM CARDEN;

OUTPUT:



QUESTION:

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

INPUT:

SELECT MAX(Charges),MIN(Charges) FROM CARDEN;

OUTPUT:

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) SELECT COUNT (*), Make FROM CARDEN;

INPUT:

SELECT COUNT(*),Make FROM CARDEN GROUP BY Make;

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.02 seconds CSV Export

QUESTION:

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

INPUT:

SELECT CarName FROM CARDEN WHERE Capacity=4;

OUTPUT:

Results Explain Describe Saved SQL History

CARNAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

11. Consider the following tables EMPLOYEE and SALGRADE and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between '09-FEB-2006' and '08-AUG-2009'.

(iv) TO add a new row in the EMPLOYEE table with the following data: 109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

EMPLOYEE

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-2003	13-JAN-1980
102	Ravi Chander	HEAD-IT	S02	12-FEB-2010	22-JUL-1987
103	John Ken	Receptionist	S03	24-JUN-2009	24-FEB-1983
105	NazarAmeen	GM	S02	11-AUG-2006	03-MAR-1984
108	PriyamSen	CEO	S01	29-DEC-2004	19-JAN-1982

SALGRADE

SGRADE	SALARY	HRA
S01	56000	18000
S02	32000	12000
S03	24000	8000

AUTONOMOUS

SYNTAX:

CREATE TABLE EMPLOYEE(ECODE INT PRIMARY KEY,NAME VARCHAR(30),DESIG VARCHAR(30),SGRADE VARCHAR(5),DOJ DATE,DOB DATE);

```
INSERT INTO EMPLOYEE VALUES(101,'Abdul Ahmad','EXECUTIVE','S03',DATE '2003-03-23',DATE '1980-01-13');
INSERT INTO EMPLOYEE VALUES(102,'Ravi Chander','HEAD-IT','S02',DATE '2010-02-12',DATE '1987-07-22');
INSERT INTO EMPLOYEE VALUES(103,'John Ken','RECEPTIONIST','S03',DATE '2009-06-24',DATE '1983-02-24');
INSERT INTO EMPLOYEE VALUES(105,'Nazar Ameen','GM','S02',DATE '2006-08-11',DATE '1984-03-03');
INSERT INTO EMPLOYEE VALUES(108,'Priyam Sen','CEO','S01',DATE '2004-12-29',DATE '1982-01-19');
CREATE TABLE SALGRADE(SGRADE VARCHAR(5) PRIMARY KEY,SALARY INT,HRA INT);
INSERT INTO SALGRADE VALUES('S01',56000,18000);
INSERT INTO SALGRADE VALUES('S02',32000,12000);
INSERT INTO SALGRADE VALUES('S03',24000,8000);
SELECT * FROM EMPLOYEE ORDER BY DOJ DESC;
SELECT NAME,DESIG FROM EMPLOYEE WHERE SGRADE IN('S02','S03');
SELECT * FROM EMPLOYEE WHERE DOJ BETWEEN DATE '2006-02-09' AND DATE '2009-08-08';
SELECT COUNT(SGRADE),SGRADE FROM EMPLOYEE GROUP BY SGRADE;
SELECT MIN(DOB),MAX(DOJ) FROM EMPLOYEE;
SELECT NAME,SALARY FROM EMPLOYEE E,SALGRADE S WHERE E.SGRADE=S.SGRADE AND E.ECODE<103;
SELECT SGRADE,SALARY+HRA FROM SALGRADE WHERE SGRADE='S02';
INSERT INTO EMPLOYEE VALUES(109,'Harish Roy','HEAD-IT','S02',DATE '2007-09-09',DATE '1983-04-21');
```

QUESTION:

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

INPUT:

```
SELECT * FROM EMPLOYEE ORDER BY DOJ DESC;
```

OUTPUT:

Results | Explain | Describe | Saved SQL | History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
102	Ravi Chander	HEAD-IT	S02	12-FEB-10	22-JUL-87
103	John Ken	RECEPTIONIST	S03	24-JUN-09	24-FEB-83
105	Nazar Ameen	GM	S02	11-AUG-06	03-MAR-84
108	Priyam Sen	CEO	S01	29-DEC-04	19-JAN-82
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-03	13-JAN-80

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

INPUT:

```
SELECT NAME,DESIG FROM EMPLOYEE WHERE SGRADE IN('S02','S03');
```

OUTPUT:

Results | Explain | Describe | Saved SQL | History

NAME	DESIG
Ravi Chander	HEAD-IT
Abdul Ahmad	EXECUTIVE
John Ken	RECEPTIONIST
Nazar Ameen	GM

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between ’09-FEB-2006’ and ’08-AUG-2009’.

INPUT:

```
SELECT * FROM EMPLOYEE WHERE DOJ BETWEEN DATE '2006-02-09' AND DATE '2009-08-08';
```

OUTPUT:

Results | Explain | Describe | Saved SQL | History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
103	John Ken	RECEPTIONIST	S03	24-JUN-09	24-FEB-83
105	Nazar Ameen	GM	S02	11-AUG-06	03-MAR-84

2 rows returned in 0.02 seconds CSV Export

QUESTION:

(iv) TO add a new row in the EMPLOYEE table with the following data: 109, ‘Harish Roy’, ‘HEAD-IT’, ‘S02’, ’09-SEP-2007’, ’21-APR-1983’.

INPUT:

INSERT INTO EMPLOYEE VALUES(109,'Harish Roy','HEAD-IT','S02',DATE '2007-09-09',DATE '1983-04-21');

OUTPUT:

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.02 seconds
```

QUESTION:

(b) Give the output of the following SQL queries:
(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

INPUT:

SELECT COUNT(SGRADE),SGRADE FROM EMPLOYEE GROUP BY SGRADE;

OUTPUT:

Results Explain Describe Saved SQL History

COUNT(SGRADE)	SGRADE
2	S02
2	S03
1	S01

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

INPUT:

SELECT MIN(DOB),MAX(DOJ) FROM EMPLOYEE;

OUTPUT:

Results Explain Describe Saved SQL History

MIN(DOB)	MAX(DOJ)
13-JAN-80	12-FEB-10

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

INPUT:

SELECT NAME,SALARY FROM EMPLOYEE E,SALGRADE S WHERE E.SGRADE=S.SGRADE AND E.ECODE<103;

OUTPUT:



Results Explain Describe Saved SQL History

NAME	SALARY
Ravi Chander	32000
Abdul Ahmad	24000

2 rows returned in 0.02 seconds CSV Export

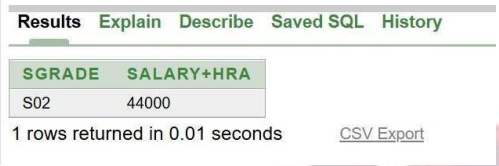
QUESTION:

(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

INPUT:

SELECT SGRADE,SALARY+HRA FROM SALGRADE WHERE SGRADE='S02';

OUTPUT:



Results Explain Describe Saved SQL History

SGRADE	SALARY+HRA
S02	44000

1 rows returned in 0.01 seconds CSV Export

12. Consider the following tables WORKER and PAYLAVEL and answer (a) and (b) parts of this question

(a) Write SQL commands for the following statements:

(i) To display the detail of all WORKER in descending order of DOB.
(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.
(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.
(iv) To add a new row with the following: 19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.
(b) Give the output of the following SQL queries:
(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;
(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;
(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;
(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL ='P003';

WORKER

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
11	RadheShyam	Supervisor	P001	13-SEP-2004	23-AUG-1981
12	ChanderNath	Operator	P003	22-FEB-2010	12-JUL-1987
13	Fizza	Operator	P003	14-JUN-2009	14-OCT-1983
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-2006	13-MAR-1984
18	Sanya	Clerk	P002	19-DEC-2005	09-JUN-1983

PAYLAVEL

PLEVEL	PAY	ALLOWANCE
P001	26000	12000
P002	22000	10000
P003	12000	6000

SYNTAX:

```
CREATE TABLE WORKER(ECODE INT,NAME VARCHAR(20),DESIGN VARCHAR(20),PLEVEL VARCHAR(5),DOJ DATE,DOB DATE);
INSERT INTO WORKER VALUES(11,'RadheShyam','Supervisor','P001','13-SEP-2004','23-AUG-1981');
INSERT INTO WORKER VALUES(12,'ChanderNath','Operator','P003','22-FEB-2010','12-JUL-1987');
INSERT INTO WORKER VALUES(13,'Fizza','Operator','P003','14-JUN-2009','14-OCT-1983');
INSERT INTO WORKER VALUES(15,'Ahmeen Ahmad','Mechanic','P002','21-AUG-2006','13-MAR-1984');
INSERT INTO WORKER VALUES(18,'Sanya','Clerk','P002','19-DEC-2005','09-JUN-1983');
```

```
CREATE TABLE PAYLEVEL(PLEVEL VARCHAR(5),PAY INT,ALLOWANCE INT);
INSERT INTO PAYLEVEL VALUES('P001',26000,12000);
INSERT INTO PAYLEVEL VALUES('P002',22000,10000);
INSERT INTO PAYLEVEL VALUES('P003',12000,6000);
```

QUESTION:

(a) Write SQL commands for the following statements:
(i) To display the detail of all WORKER in descending order of DOB.

INPUT:

```
SELECT * FROM WORKER ORDER BY DOB DESC;
```

OUTPUT:

Results | Explain | Describe | Saved SQL | History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
12	ChanderNath	Operator	P003	22-FEB-10	12-JUL-87
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84
13	Fizza	Operator	P003	14-JUN-09	14-OCT-83
18	Sanya	Clerk	P002	19-DEC-05	09-JUN-83
11	RadheShyam	Supervisor	P001	13-SEP-04	23-AUG-81

5 rows returned in 0.01 seconds CSV Export

QUESTION:

(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.

INPUT:

```
SELECT NAME,DESIGN FROM WORKER WHERE PLEVEL='P001' OR PLEVEL='P002';
```

OUTPUT:

Results | Explain | Describe | Saved SQL | History

NAME	DESIGN
RadheShyam	Supervisor
Ahmeen Ahmad	Mechanic
Sanya	Clerk

3 rows returned in 0.02 seconds CSV Export

QUESTION:

(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

INPUT:

SELECT * FROM WORKER WHERE DOB BETWEEN '19-JAN-1984' AND '18-JAN-1987';

OUTPUT:



Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84

1 rows returned in 0.00 seconds CSV Export

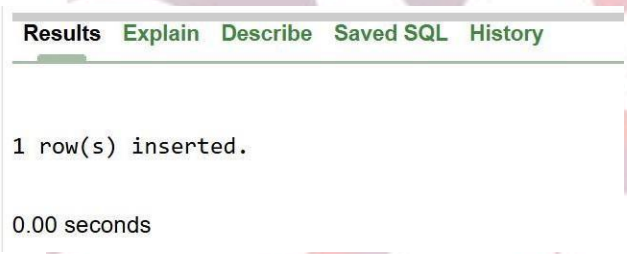
QUESTION:

(iv) To add a new row with the following: 19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.

INPUT:

INSERT INTO WORKER VALUES(19,'Daya Kishore','Operator','P003','19-JUN-2008','11-JUL-1984');

OUTPUT:



Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

QUESTION:

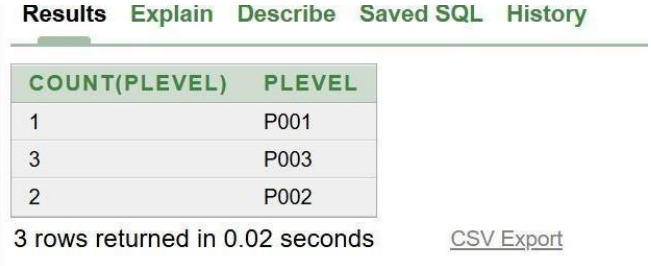
(b) Give the output of the following SQL queries:

(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

INPUT:

SELECT COUNT(PLEVEL),PLEVEL FROM WORKER GROUP BY PLEVEL;

OUTPUT:



Results Explain Describe Saved SQL History

COUNT(PLEVEL)	PLEVEL
1	P001
3	P003
2	P002

3 rows returned in 0.02 seconds CSV Export

QUESTION:

(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

INPUT:

SELECT MAX(DOB),MIN(DOJ) FROM WORKER;

OUTPUT:



Results Explain Describe Saved SQL History

MAX(DOB)	MIN(DOJ)
12-JUL-87	13-SEP-04

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;

INPUT:

SELECT NAME,PAY FROM WORKER W,PAYLEVEL P WHERE W.PLEVEL=P.PLEVEL AND W.ECODE<13;

OUTPUT:



Results Explain Describe Saved SQL History

NAME	PAY
RadheShyam	26000
ChanderNath	12000

2 rows returned in 0.01 seconds CSV Export

QUESTION:

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL ='P003';

INPUT:

SELECT PLEVEL,PAY+ALLOWANCE FROM PAYLEVEL WHERE PLEVEL='P003';

OUTPUT:



13. Consider the following tables STORE and SUPPLIERS and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy.

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

(iii) To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

(iv) To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

(ii) SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

(iii) SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeAND ItemNo=2006;

(iv) SELECT MAX(LastBuy) FROM STORE;

STORE

ItemNo	Item	Scode	Qty	Rate	LastBuy
2005	Sharpener Classic	23	60	8	31-JUN-09
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2002	Gel Pen Premium	21	150	12	24-FEB-10
2006	Gel Pen Classic	21	250	20	11-MAY-09
2001	Eraser Small	22	220	6	19-JAN-09
2004	Eraser Big	22	110	8	02-DEC-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

SUPPLIERS

Scode	Sname
21	Premium Stationers
23	Soft Plastics
22	Tetra Supply

SYNTAX:

```
DROP TABLE STORE CASCADE CONSTRAINTS;
DROP TABLE SUPPLIERS CASCADE CONSTRAINTS;
CREATE TABLE STORE(ItemNo INT PRIMARY KEY,Item VARCHAR(30),Scode INT,Qty INT,Rate INT,LastBuy DATE);
INSERT INTO STORE VALUES(2001,'Eraser Small',22,220,6,TO_DATE('19-JAN-2009','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2002,'Gel Pen Premium',21,150,12,TO_DATE('24-FEB-2010','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2003,'Ball Pen 0.25',22,50,25,TO_DATE('01-FEB-2010','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2004,'Eraser Big',22,110,8,TO_DATE('02-DEC-2009','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2005,'Sharpener Classic',23,60,8,TO_DATE('30-JUN-2009','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2006,'Gel Pen Classic',21,250,20,TO_DATE('11-MAR-2009','DD-MON-YYYY'));
INSERT INTO STORE VALUES(2009,'Ball Pen 0.5',21,180,18,TO_DATE('03-NOV-2009','DD-MON-YYYY'));
CREATE TABLE SUPPLIERS(Scode INT PRIMARY KEY,Sname VARCHAR(30));
INSERT INTO SUPPLIERS VALUES(21,'Premium Stationers');
INSERT INTO SUPPLIERS VALUES(22,'Tetra Supply');
```

INSERT INTO SUPPLIERS VALUES(23,'Soft Plastics');

QUESTION:

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy.

INPUT:

SELECT * FROM STORE ORDER BY LastBuy ASC;

OUTPUT:

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Eraser Small	22	220	6	19-JAN-09
2006	Gel Pen Classic	21	250	20	11-MAR-09
2005	Sharpener Classic	23	60	8	30-JUN-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09
2004	Eraser Big	22	110	8	02-DEC-09
2003	Ball Pen 0.25	22	50	25	01-FEB-10
2002	Gel Pen Premium	21	150	12	24-FEB-10

7 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

INPUT:

SELECT ItemNo,Item FROM STORE WHERE Rate>15;

OUTPUT:

Results Explain Describe Saved SQL History

ITEMNO	ITEM
2003	Ball Pen 0.25
2006	Gel Pen Classic
2009	Ball Pen 0.5

3 rows returned in 0.01 seconds CSV Export

QUESTION:

(iii) To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

INPUT:

SELECT * FROM STORE WHERE Scode=22 OR Qty>110;

OUTPUT:

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Eraser Small	22	220	6	19-JAN-09
2002	Gel Pen Premium	21	150	12	24-FEB-10
2003	Ball Pen 0.25	22	50	25	01-FEB-10
2004	Eraser Big	22	110	8	02-DEC-09
2006	Gel Pen Classic	21	250	20	11-MAR-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

6 rows returned in 0.01 seconds CSV Export

QUESTION:

(iv) To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

INPUT:

SELECT Scode,MIN(Rate) FROM STORE GROUP BY Scode;

OUTPUT:

Results Explain Describe Saved SQL History

SCODE	MIN(RATE)
22	6
21	12
23	8

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

INPUT:

SELECT COUNT(DISTINCT Scode) FROM STORE;

OUTPUT:



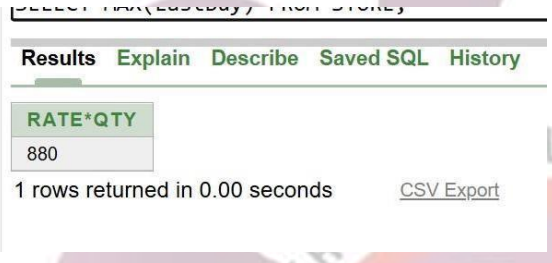
QUESTION:

(ii) SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

INPUT:

SELECT Rate*Qty FROM STORE WHERE ItemNo=2004;

OUTPUT:



QUESTION:

(iii) SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeAND ItemNo=2006;

INPUT:

SELECT Item,Sname FROM STORE S,SUPPLIERS P WHERE S.Scode=P.Scode AND S.ItemNo=2006;

OUTPUT:



QUESTION:

(iv) SELECT MAX(LastBuy) FROM STORE;

INPUT:

SELECT MAX(LastBuy) FROM STORE;

OUTPUT:



14. Consider the following table GARMENT and FABRIC, Write SQL commands for the statements (i) to (iv) and give outputs for the SQL queries (v) to (viii).

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

(ii) To display the details of all the GARMENT, which have READYDATE in between 08-DEC-07 and 16-JUN-08 (inclusive if both the dates).

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = 'F01';

(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

(vii) SELECT MAX (FCODE) FROM FABRIC;

(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

GARMENT

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10023	Pencil Skirt	1150	F03	19-DEC-08
10001	Formal Shirt	1250	F01	12-JAN-08
10012	Informal Shirt	1550	F02	06-JUN-08
10024	Baby Top	750	F03	07-APR-07
10090	Tulip Skirt	850	F02	31-MAR-07
10019	Evening Gown	850	F03	06-JUN-08
10009	Informal Pant	1500	F02	20-OCT-08
10007	Formal Pant	1350	F01	09-MAR-08
10020	Frock	850	F04	09-SEP-07
10089	Slacks	750	F03	20-OCT-08

FABRIC

FCODE	TYPE
F04	Polyester
F02	Cotton
F03	Silk
F01	Terelene

SYNTAX:

CREATE TABLE GARMENT(GCODE INT,DESCRIPTION VARCHAR(30),PRICE INT,FCODE VARCHAR(5),READYDATE DATE);

INSERT INTO GARMENT VALUES(10023,'Pencil Skirt',1150,'F03','19-DEC-08');

INSERT INTO GARMENT VALUES(10001,'Formal Shirt',1250,'F01','12-JAN-08');

INSERT INTO GARMENT VALUES(10012,'Informal Shirt',1550,'F02','06-JUN-08');

INSERT INTO GARMENT VALUES(10024,'Baby Top',750,'F03','07-APR-07');

INSERT INTO GARMENT VALUES(10090,'Tulip Skirt',850,'F02','31-MAR-07');

INSERT INTO GARMENT VALUES(10019,'Evening Gown',850,'F03','06-JUN-08');

INSERT INTO GARMENT VALUES(10009,'Informal Pant',1500,'F02','20-OCT-08');

INSERT INTO GARMENT VALUES(10007,'Formal Pant',1350,'F01','09-MAR-08');

INSERT INTO GARMENT VALUES(10020,'Frock',850,'F04','09-SEP-07');

INSERT INTO GARMENT VALUES(10089,'Slacks',750,'F03','20-OCT-08');

CREATE TABLE FABRIC(FCODE VARCHAR(5),TYPE VARCHAR(20));

INSERT INTO FABRIC VALUES('F04','Polyester');

INSERT INTO FABRIC VALUES('F02','Cotton');

INSERT INTO FABRIC VALUES('F03','Silk');

INSERT INTO FABRIC VALUES('F01','Terelene');

QUESTION:

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

INPUT:

SELECT GCODE,DESCRIPTION FROM GARMENT ORDER BY GCODE DESC;

OUTPUT:

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION
10090	Tulip Skirt
10089	Slacks
10024	Baby Top
10023	Pencil Skirt
10020	Frock
10019	Evening Gown
10012	Informal Shirt
10009	Informal Pant
10007	Formal Pant
10001	Formal Shirt

10 rows returned in 0.00 seconds CSV Export

QUESTION:

(ii) To display the details of all the GARMENT, which have READYDATE in between 08-DEC-07 and 16-JUN-08 (inclusive if both the dates).

INPUT:

SELECT * FROM GARMENT WHERE READYDATE BETWEEN '08-DEC-07' AND '16-JUN-08';

OUTPUT:

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10012	Informal Shirt	1550	F02	06-JUN-08
10001	Formal Shirt	1250	F01	12-JAN-08
10019	Evening Gown	850	F03	06-JUN-08
10007	Formal Pant	1350	F01	09-MAR-08

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

INPUT:

SELECT AVG(PRICE) FROM GARMENT WHERE FCODE='F03';

OUTPUT:



Results Explain Describe Saved SQL History

AVG(PRICE)
875

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

INPUT:

SELECT FCODE,MAX(PRICE),MIN(PRICE) FROM GARMENT GROUP BY FCODE;

OUTPUT:



Results Explain Describe Saved SQL History

FCODE	MAX(PRICE)	MIN(PRICE)
F02	1550	850
F04	850	850
F03	1150	750
F01	1350	1250

4 rows returned in 0.00 seconds CSV Export

QUESTION:

(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = ‘F01’;

INPUT:

SELECT SUM(PRICE) FROM GARMENT WHERE FCODE='F01';

OUTPUT:



Results Explain Describe Saved SQL History

SUM(PRICE)
2600

1 rows returned in 0.00 seconds CSV Export

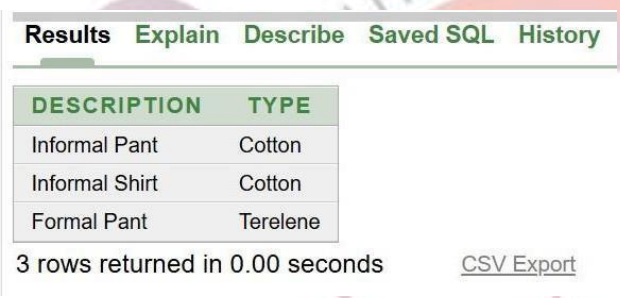
QUESTION:

(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

INPUT:

SELECT DESCRIPTION,TYPE FROM GARMENT G,FABRIC F WHERE G.FCODE=F.FCODE AND G.PRICE>=1260;

OUTPUT:



Results Explain Describe Saved SQL History

DESCRIPTION	TYPE
Informal Pant	Cotton
Informal Shirt	Cotton
Formal Pant	Terelene

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(vii) SELECT MAX (FCODE) FROM FABRIC;

INPUT:

SELECT MAX(FCODE) FROM FABRIC;

OUTPUT:



Results Explain Describe Saved SQL History

MAX(FCODE)
F04

1 rows returned in 0.00 seconds CSV Export

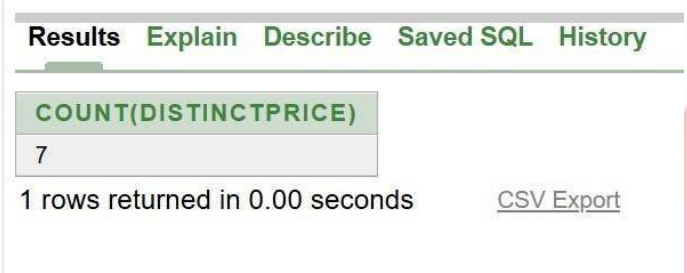
QUESTION:

(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

INPUT:

SELECT COUNT(DISTINCT PRICE) FROM GARMENT;

OUTPUT:



15. Given the following tables for a database FURNITURE : Write SQL command for (a) to (f) and write the outputs for (g) on the bases of tables FURNITURE AND ARRIVALS.

(a) To show all information about the baby cots from the FURNITURE table.

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

(e) To count the number of items, whose TYPE is "Sofa" from FURNITURE table.

(f) To insert a new row in the ARRIVALS table with the following data: 14, "Velvet touch", Double bed", {25/03/03}, 25000, 30.

(g) Give the output of following SQL statement:

Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.

(i) Select COUNT(distinct TYPE) from FURNITURE;

(ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = "Baby cot";

(iv) Select SUM(PRICE) from FURNITURE where DATEOFSTOCK < {12/02/02};

FURNITURE

No.	Item Name	Type	Date of Stock	Price	Discount
1	White lotus	Double Bed	23/02/02	30000	25
2	Pink feather	Baby cot	20/01/02	7000	20
3	Dolphin	Baby cot	19/02/02	9500	20
4	Decent	Office Table	01/01/02	25000	30
5	Comfort zone	Double Bed	12/01/02	25000	25
6	Donald	Baby cot	24/02/02	6500	15
7	Royal Finish	Office Table	20/02/02	18000	30
8	Royal tiger	Sofa	22/02/02	31000	30
9	Econo sitting	Sofa	13/12/01	9500	25
10	Eating paradise	Dining Table	19/02/02	11500	25

ARRIVALS

No.	Item Name	Type	Date of Stock	Price	Discount
11	Wood Comfort	Double Bed	23/03/03	25000	25
12	Old Fox	Sofa	20/02/03	17000	20
13	Micky	Baby cot	21/02/02	7500	15

SYNTAX:

```
CREATE TABLE FURNITURE(NO INT PRIMARY KEY,ITEMNAME
VARCHAR(30),TYPE VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT
INT);
INSERT INTO FURNITURE VALUES(1,'White lotus','Double Bed',DATE '2002-02-
23',30000,25);
INSERT INTO FURNITURE VALUES(2,'Pink feather','Baby cot',DATE '2002-01-20',7000,20);
INSERT INTO FURNITURE VALUES(3,'Dolphin','Baby cot',DATE '2002-02-19',9500,20);
INSERT INTO FURNITURE VALUES(4,'Decent','Office Table',DATE '2002-01-01',25000,30);
INSERT INTO FURNITURE VALUES(5,'Comfort zone','Double Bed',DATE '2002-01-
12',25000,25);
INSERT INTO FURNITURE VALUES(6,'Donald','Baby cot',DATE '2002-02-24',6500,15);
INSERT INTO FURNITURE VALUES(7,'Royal Finish','Office Table',DATE '2002-02-
20',18000,30);
INSERT INTO FURNITURE VALUES(8,'Royal tiger','Sofa',DATE '2002-02-22',31000,30);
INSERT INTO FURNITURE VALUES(9,'Econo sitting','Sofa',DATE '2001-12-13',9500,25);
INSERT INTO FURNITURE VALUES(10,'Eating paradise','Dining Table',DATE '2002-02-
19',11500,25);
CREATE TABLE ARRIVALS(NO INT PRIMARY KEY,ITEMNAME VARCHAR(30),TYPE
VARCHAR(20),DATEOFSTOCK DATE,PRICE INT,DISCOUNT INT);
```

INSERT INTO ARRIVALS VALUES(11,'Wood Comfort','Double Bed',DATE '2003-03-23',25000,25);
INSERT INTO ARRIVALS VALUES(12,'Old Fox','Sofa',DATE '2003-02-20',17000,20);
INSERT INTO ARRIVALS VALUES(13,'Micky','Baby cot',DATE '2002-02-21',7500,15);

QUESTION:

(a) To show all information about the baby cots from the FURNITURE table.

INPUT:

SELECT * FROM FURNITURE WHERE TYPE='Baby cot';

OUTPUT:

Results Explain Describe Saved SQL History

NO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
2	Pink feather	Baby cot	20-JAN-02	7000	20
3	Dolphin	Baby cot	19-FEB-02	9500	20
6	Donald	Baby cot	24-FEB-02	6500	15

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

INPUT:

SELECT ITEMNAME FROM FURNITURE WHERE PRICE>15000;

OUTPUT:

Results Explain Describe Saved SQL History

ITEMNAME
White lotus
Decent
Comfort zone
Royal Finish
Royal tiger

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

INPUT:

SELECT ITEMNAME,TYPE FROM FURNITURE WHERE DATEOFSTOCK<DATE '2002-01-01' ORDER BY ITEMNAME DESC;

OUTPUT:



Results Explain Describe Saved SQL History

ITEMNAME	TYPE
Econo sitting	Sofa

1 rows returned in 0.01 seconds CSV Export

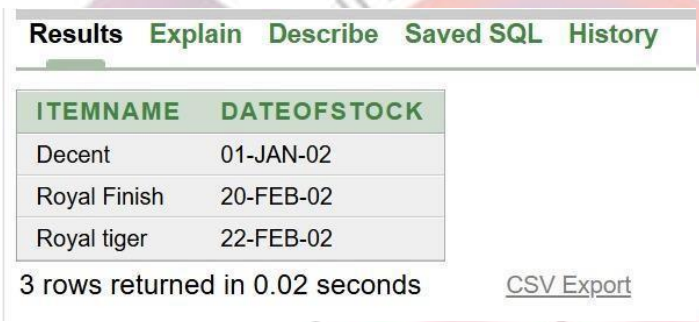
QUESTION:

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

INPUT:

SELECT ITEMNAME,DATEOFSTOCK FROM FURNITURE WHERE DISCOUNT>25;

OUTPUT:



Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
Decent	01-JAN-02
Royal Finish	20-FEB-02
Royal tiger	22-FEB-02

3 rows returned in 0.02 seconds CSV Export

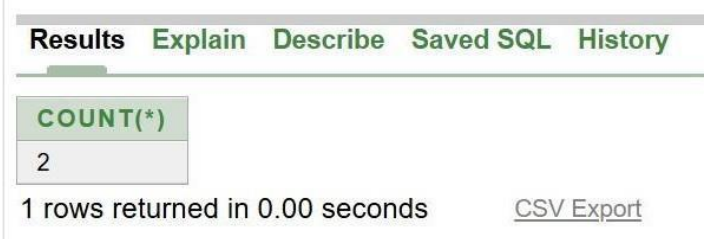
QUESTION:

(e) To count the number of items, whose TYPE is “Sofa” from FURNITURE table.

INPUT:

SELECT COUNT(*) FROM FURNITURE WHERE TYPE='Sofa';

OUTPUT:



Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

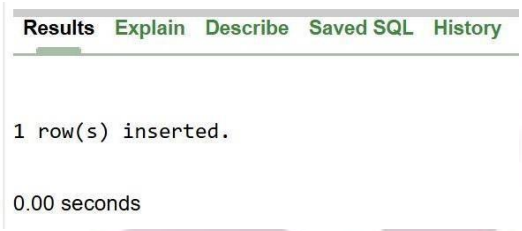
QUESTION:

(f) To insert a new row in the ARRIVALS table with the following data: 14, “Velvet touch”, Double bed”, {25/03/03}, 25000, 30.

INPUT:

```
INSERT INTO ARRIVALS VALUES(14,'Velvet touch','Double bed',DATE '2003-03-25',25000,30);
```

OUTPUT:



QUESTION:

(g) Give the output of following SQL statement:

Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.

(i) Select COUNT(distinct TYPE) from FURNITURE;

INPUT:

```
SELECT COUNT(DISTINCT TYPE) FROM FURNITURE;
```

OUTPUT:



QUESTION:

(ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

INPUT:

SELECT MAX(FURNITURE.DISCOUNT) FROM FURNITURE,ARRIVALS;

OUTPUT:


Results Explain Describe Saved SQL History

MAX(FURNITURE.DISCOUNT)
30

1 rows returned in 0.00 seconds CSV Export

QUESTION:

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = “Baby cot”;

INPUT:

SELECT AVG(DISCOUNT) FROM FURNITURE WHERE TYPE='Baby cot';

OUTPUT:


Results Explain Describe Saved SQL History

AVG(DISCOUNT)
18.333333333333333333333333333333333333

1 rows returned in 0.00 seconds CSV Export

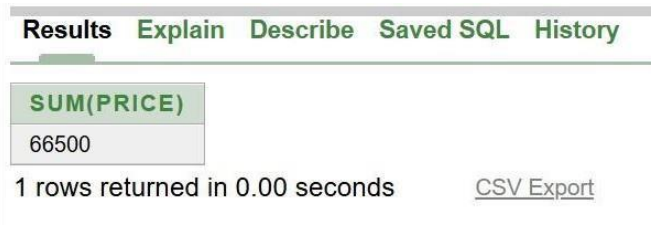
QUESTION:

(iv) Select SUM(PRICE) from FURNITURE where DATEOFSTOCK < {12/02/02};

INPUT:

SELECT SUM(PRICE) FROM FURNITURE WHERE DATEOFSTOCK<DATE '2002-02-12';

OUTPUT:


Results Explain Describe Saved SQL History

SUM(PRICE)
66500

1 rows returned in 0.00 seconds CSV Export

16. Given the following COVID19 relation, write SQL commands for questions (a) to (f). a) Display all details of patients from Tamil Nadu.

b) Display the names of patients whose Status is Recovered. c) Display PatientName and Age in ascending order of Age.

d) Display the names of patients who are Vaccinated = 'Yes'.

e) Count the number of patients whose status is Active.

f) Insert the following record into the COVID19 table: 108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

(g) Write the output of the following SQL commands.

a. SELECT COUNT*() FROM COVID19;*

b. SELECT MAX(Age) FROM COVID19;

c. SELECT COUNT() FROM COVID19 WHERE Vaccinated='Yes';

d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

PatientID	PatientName	State	Age	Status	Vaccinated
101	Arun	Tamil Nadu	35	Recovered	Yes
102	Priya	Kerala	42	Active	Yes
103	Karthik	Karnataka	28	Recovered	No
104	Divya	Tamil Nadu	31	Active	Yes
105	Rahul	Andhra Pradesh	55	Deceased	No
106	Meena	Kerala	24	Recovered	Yes
107	Suresh	Tamil Nadu	48	Active	No

SYNTAX:

```
CREATE TABLE COVID19 (PatientID INT PRIMARY KEY, PatientName VARCHAR(20),
State VARCHAR(30), Age INT, Status VARCHAR(15), Vaccinated VARCHAR(5));
INSERT INTO COVID19 VALUES (101, 'Arun', 'Tamil Nadu', 35, 'Recovered', 'Yes');
INSERT INTO COVID19 VALUES (102, 'Priya', 'Kerala', 42, 'Active', 'Yes');
INSERT INTO COVID19 VALUES (103, 'Karthik', 'Karnataka', 28, 'Recovered', 'No');
INSERT INTO COVID19 VALUES (104, 'Divya', 'Tamil Nadu', 31, 'Active', 'Yes');
INSERT INTO COVID19 VALUES (105, 'Rahul', 'Andhra Pradesh', 55, 'Deceased', 'No');
INSERT INTO COVID19 VALUES (106, 'Meena', 'Kerala', 24, 'Recovered', 'Yes');
INSERT INTO COVID19 VALUES (107, 'Suresh', 'Tamil Nadu', 48, 'Active', 'No');
```

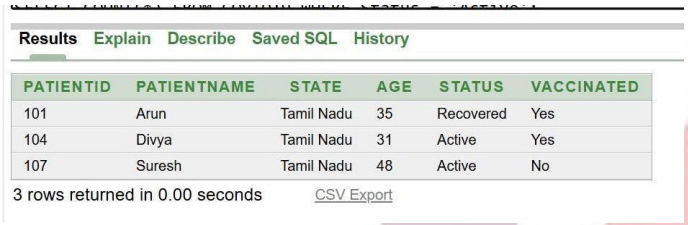
QUESTION:

a) Display all details of patients from Tamil Nadu.

INPUT:

```
SELECT * FROM COVID19 WHERE State = 'Tamil Nadu';
```

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	35	Recovered	Yes
104	Divya	Tamil Nadu	31	Active	Yes
107	Suresh	Tamil Nadu	48	Active	No

3 rows returned in 0.00 seconds CSV Export

QUESTION:

b) Display the names of patients whose Status is Recovered.

INPUT:

```
SELECT PatientName FROM COVID19 WHERE Status = 'Recovered';
```

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Karthik
Meena

3 rows returned in 0.00 seconds CSV Export

QUESTION:

c) Display PatientName and Age in ascending order of Age.

INPUT:

```
SELECT PatientName, Age FROM COVID19 ORDER BY Age ASC;
```

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Meena	24
Karthik	28
Divya	31
Arun	35
Priya	42
Suresh	48
Rahul	55

7 rows returned in 0.00 seconds CSV Export

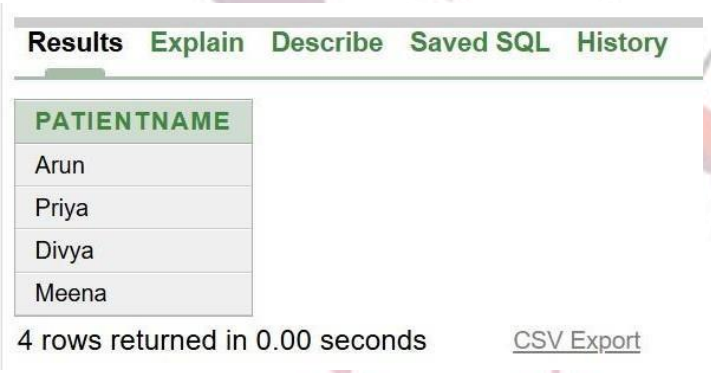
QUESTION:

d) Display the names of patients who are Vaccinated = 'Yes'.

INPUT:

SELECT PatientName FROM COVID19 WHERE Vaccinated = 'Yes';

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Priya
Divya
Meena

4 rows returned in 0.00 seconds CSV Export

QUESTION:

e) Count the number of patients whose status is Active.

INPUT:

SELECT COUNT(*) FROM COVID19 WHERE Status = 'Active';

OUTPUT:



Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.00 seconds CSV Export

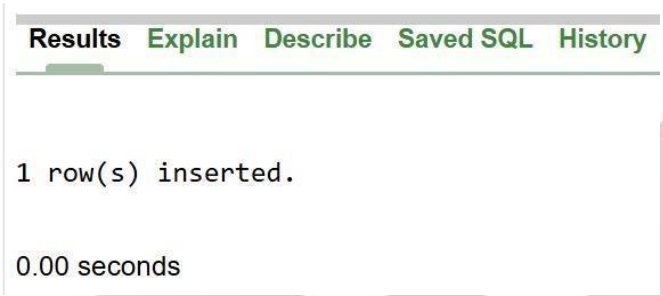
QUESTION:

f) Insert the following record into the COVID19 table: 108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

INPUT:

```
INSERT INTO COVID19 VALUES (108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');
```

OUTPUT:



QUESTION:

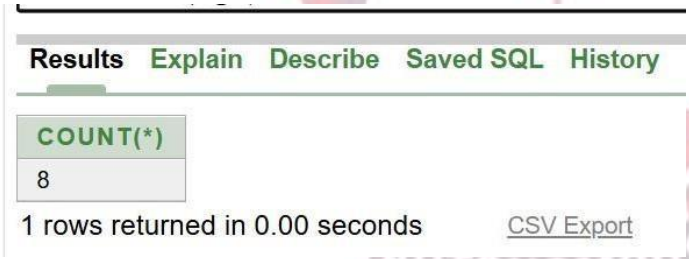
(g) Write the output of the following SQL commands.

a. SELECT COUNT*() FROM COVID19;*

INPUT:

```
SELECT COUNT(*) FROM COVID19;
```

OUTPUT:



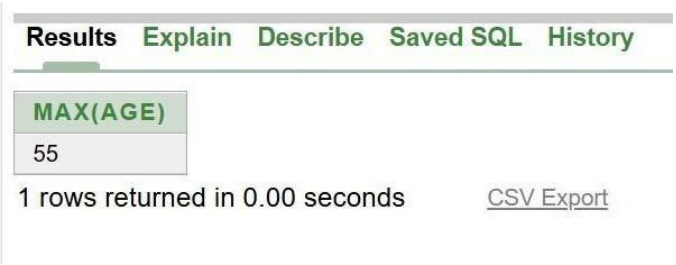
QUESTION:

b. SELECT MAX(Age) FROM COVID19*;*

INPUT:

SELECT MAX(Age) FROM COVID19;

OUTPUT:



QUESTION:

c. SELECT COUNT() FROM COVID19 WHERE Vaccinated='Yes';

INPUT:

SELECT COUNT(*) FROM COVID19 WHERE Vaccinated = 'Yes';

OUTPUT:



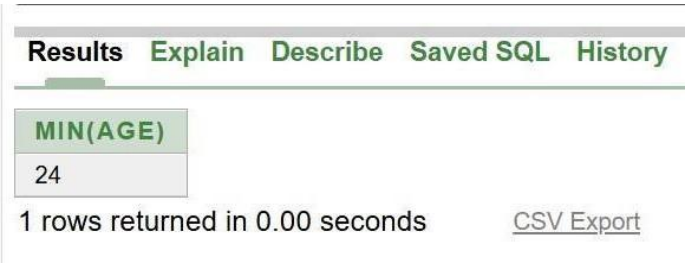
QUESTION:

d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';

INPUT:

SELECT MIN(Age) FROM COVID19 WHERE Status = 'Recovered';

OUTPUT:



17. Given the following HOUSEPRICE relation, write SQL commands for questions (a) to (f).

a) Display all houses located in Chennai.

b) Display the Location and Price of houses having 3 Bedrooms. c) Display Location, AreaSqft, and Price in descending order of Price.

d) Display the details of houses costing more than ₹70,00,000.

e) Count the number of Apartments.

f) Insert the following record: 208, 'Vellore', 3, 1700, 7300000, 'Villa'

(g) Write the output of the following SQL commands.

(i) SELECT MAX(Price)FROM HOUSEPRICE;

(i) SELECT MIN(AreaSqft)FROM HOUSEPRICE;

(ii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';

(iii) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

HouseID	Location	Bedrooms	AreaSqft	Price	HouseType
201	Chennai	2	1100	5200000	Apartment
202	Coimbatore	3	1450	6800000	Villa
203	Madurai	2	980	4100000	Apartment
204	Chennai	4	2200	9800000	Villa
205	Trichy	3	1650	7200000	Independent
206	Salem	2	1200	5000000	Apartment
207	Erode	3	1800	7600000	Independent

SYNTAX:

CREATE TABLE HOUSEPRICE (HouseID INT PRIMARY KEY, Location VARCHAR(20), Bedrooms INT, AreaSqft INT, Price INT, HouseType VARCHAR(20));

INSERT INTO HOUSEPRICE VALUES (201, 'Chennai', 2, 1100, 5200000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES (202, 'Coimbatore', 3, 1450, 6800000, 'Villa');

INSERT INTO HOUSEPRICE VALUES (203, 'Madurai', 2, 980, 4100000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES (204, 'Chennai', 4, 2200, 9800000, 'Villa');

INSERT INTO HOUSEPRICE VALUES (205, 'Trichy', 3, 1650, 7200000, 'Independent');

INSERT INTO HOUSEPRICE VALUES (206, 'Salem', 2, 1200, 5000000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES (207, 'Erode', 3, 1800, 7600000, 'Independent');

QUESTION:

a) Display all houses located in Chennai.

INPUT:

SELECT * FROM HOUSEPRICE WHERE Location = 'Chennai';

OUTPUT:

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1100	5200000	Apartment
204	Chennai	4	2200	9800000	Villa

2 rows returned in 0.01 seconds CSV Export

QUESTION:

b) Display the Location and Price of houses having 3 Bedrooms.

INPUT:

SELECT Location, Price FROM HOUSEPRICE WHERE Bedrooms = 3;

OUTPUT:

Results Explain Describe Saved SQL History

LOCATION	PRICE
Coimbatore	6800000
Trichy	7200000
Erode	7600000

3 rows returned in 0.00 seconds CSV Export

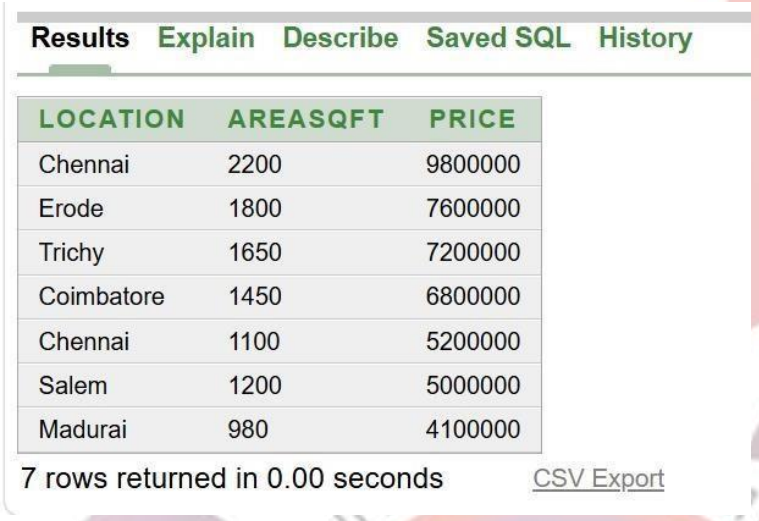
QUESTION:

c) Display Location, AreaSqft, and Price in descending order of Price.

INPUT:

SELECT Location, AreaSqft, Price FROM HOUSEPRICE ORDER BY Price DESC;

OUTPUT:



Results Explain Describe Saved SQL History

LOCATION	AREASQFT	PRICE
Chennai	2200	9800000
Erode	1800	7600000
Trichy	1650	7200000
Coimbatore	1450	6800000
Chennai	1100	5200000
Salem	1200	5000000
Madurai	980	4100000

7 rows returned in 0.00 seconds CSV Export

QUESTION:

d) Display the details of houses costing more than ₹70,00,000.

INPUT:

SELECT * FROM HOUSEPRICE WHERE Price > 7000000;

OUTPUT:



Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
204	Chennai	4	2200	9800000	Villa
205	Trichy	3	1650	7200000	Independent
207	Erode	3	1800	7600000	Independent

3 rows returned in 0.00 seconds CSV Export

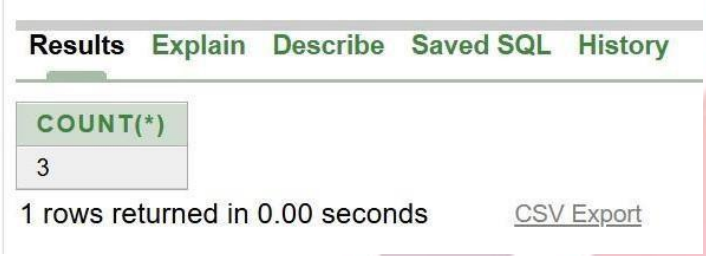
QUESTION:

e) Count the number of Apartments.

INPUT:

SELECT COUNT(*) FROM HOUSEPRICE WHERE HouseType = 'Apartment';

OUTPUT:



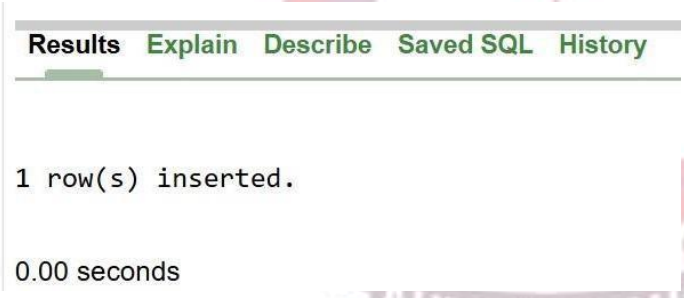
QUESTION:

f) Insert the following record: 208, 'Vellore', 3, 1700, 7300000, 'Villa'

INPUT:

INSERT INTO HOUSEPRICE VALUES (208, 'Vellore', 3, 1700, 7300000, 'Villa');

OUTPUT:



QUESTION:

(g) Write the output of the following SQL commands.

(i) SELECT MAX(Price)FROM HOUSEPRICE;

INPUT:

SELECT MAX(Price) FROM HOUSEPRICE;

OUTPUT:



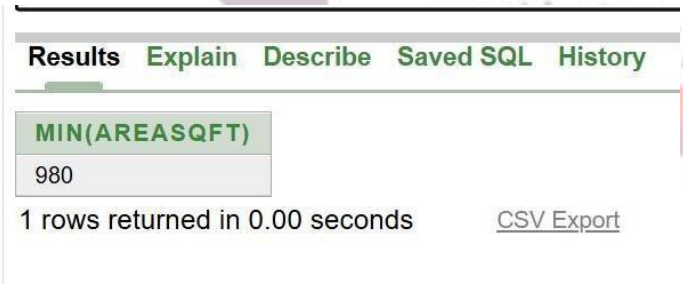
QUESTION:

(i) SELECT MIN(AreaSqft)FROM HOUSEPRICE;

INPUT:

SELECT MIN(AreaSqft) FROM HOUSEPRICE;

OUTPUT:



QUESTION:

(ii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';

INPUT:

SELECT AVG(Price) FROM HOUSEPRICE WHERE HouseType = 'Apartment';

OUTPUT:



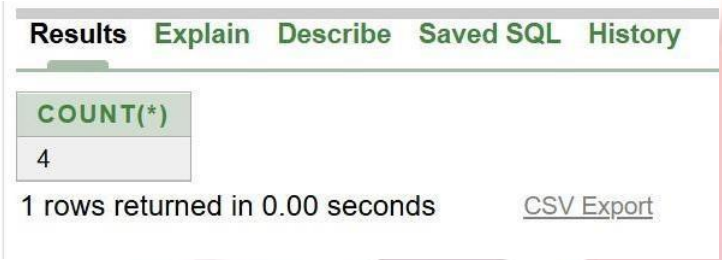
QUESTION:

(iii) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;

INPUT:

SELECT COUNT(*) FROM HOUSEPRICE WHERE Bedrooms = 3;

OUTPUT:



18. Given the following CANCERDATA relation, write SQL commands for questions (a) to (f).

a) Display all patients diagnosed with Breast Cancer.
b) Display the names of patients whose Stage is III.
c) Display PatientName and Age in ascending order of Age.
d) Display the names of patients having SurvivalYears greater than 5.
e) Count the number of patients diagnosed with Lung Cancer.
f) Insert the following record: 308, 'Anand', 'Leukemia', 50, 'II', 7
(g) Write the output of the following SQL commands.
(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;
(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;
(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';
(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

PatientID	PatientName	CancerType	Age	Stage	SurvivalYears
301	Kavya	Breast	45	II	8
302	Arun	Lung	60	III	4
303	Divya	Breast	38	I	10
304	Hari	Colon	52	II	6
305	Meena	Lung	57	IV	2
306	Ravi	Prostate	64	III	5
307	Priya	Breast	41	II	9

SYNTAX:

CREATE TABLE CANCERDATA (PatientID INT PRIMARY KEY, PatientName VARCHAR(20), CancerType VARCHAR(20), Age INT, Stage VARCHAR(5), SurvivalYears INT);

INSERT INTO CANCERDATA VALUES (301, 'Kavya', 'Breast', 45, 'II', 8);
INSERT INTO CANCERDATA VALUES (302, 'Arun', 'Lung', 60, 'III', 4);
INSERT INTO CANCERDATA VALUES (303, 'Divya', 'Breast', 38, 'I', 10);
INSERT INTO CANCERDATA VALUES (304, 'Hari', 'Colon', 52, 'II', 6);
INSERT INTO CANCERDATA VALUES (305, 'Meena', 'Lung', 57, 'IV', 2);
INSERT INTO CANCERDATA VALUES (306, 'Ravi', 'Prostate', 64, 'III', 5);
INSERT INTO CANCERDATA VALUES (307, 'Priya', 'Breast', 41, 'II', 9);

QUESTION:

a) Display all patients diagnosed with Breast Cancer.

INPUT:

SELECT * FROM CANCERDATA WHERE CancerType='Breast';

OUTPUT:



QUESTION:

b) Display the names of patients whose Stage is III.

INPUT:

SELECT PatientName FROM CANCERDATA WHERE Stage='III';

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Ravi

2 rows returned in 0.00 seconds CSV Export

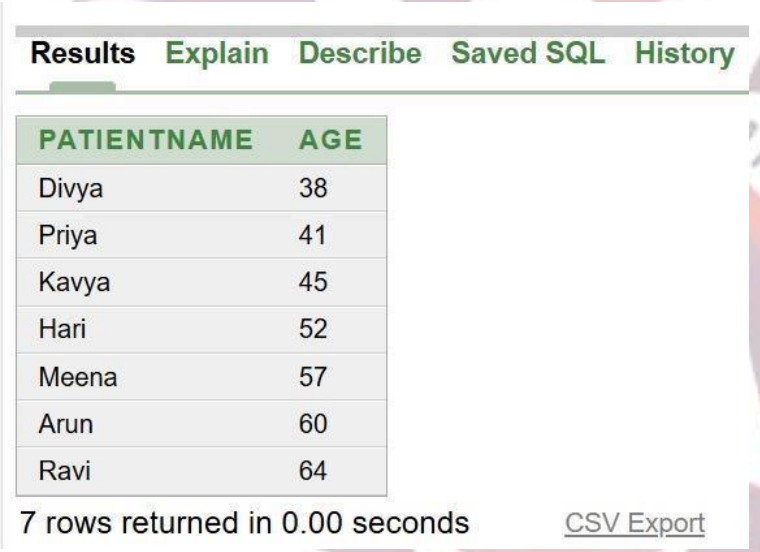
QUESTION:

c) Display PatientName and Age in ascending order of Age.

INPUT:

SELECT PatientName,Age FROM CANCERDATA ORDER BY Age ASC;

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Divya	38
Priya	41
Kavya	45
Hari	52
Meena	57
Arun	60
Ravi	64

7 rows returned in 0.00 seconds CSV Export

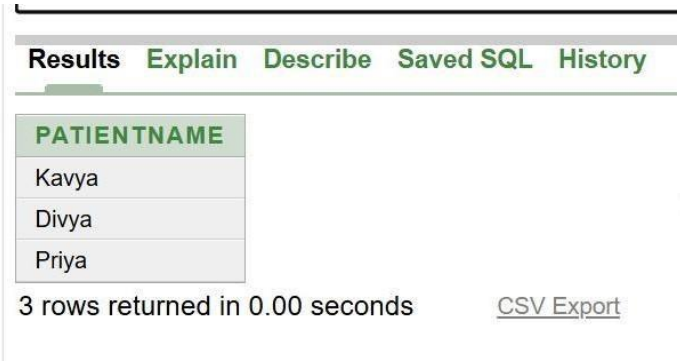
QUESTION:

d) Display the names of patients having SurvivalYears greater than 5.

INPUT:

SELECT PatientName FROM CANCERDATA WHERE SurvivalYears>6;

OUTPUT:



Results Explain Describe Saved SQL History

PATIENTNAME
Kavya
Divya
Priya

3 rows returned in 0.00 seconds CSV Export

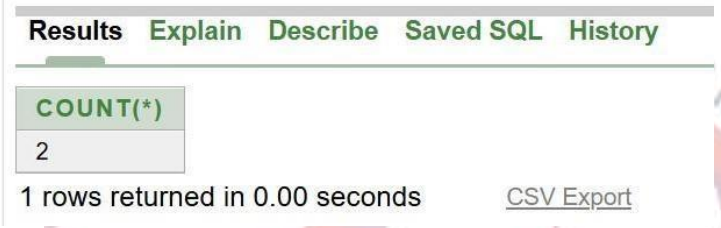
QUESTION:

e) Count the number of patients diagnosed with Lung Cancer.

INPUT:

SELECT COUNT(*) FROM CANCERDATA WHERE CancerType='Lung';

OUTPUT:



Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

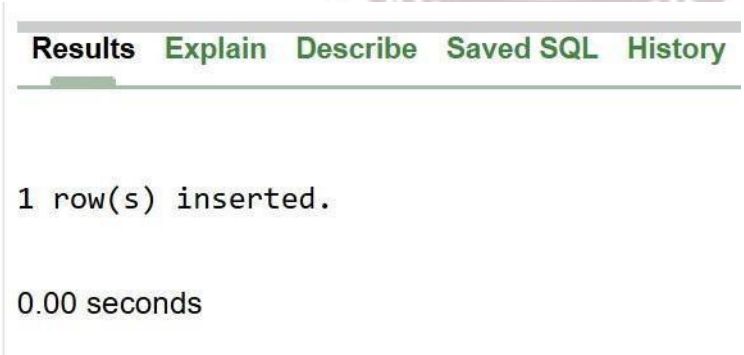
QUESTION:

f) Insert the following record: 308, 'Anand', 'Leukemia', 50, 'II', 7

INPUT:

INSERT INTO CANCERDATA VALUES (308, 'Anand', 'Leukemia', 50, 'II', 7);

OUTPUT:



Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

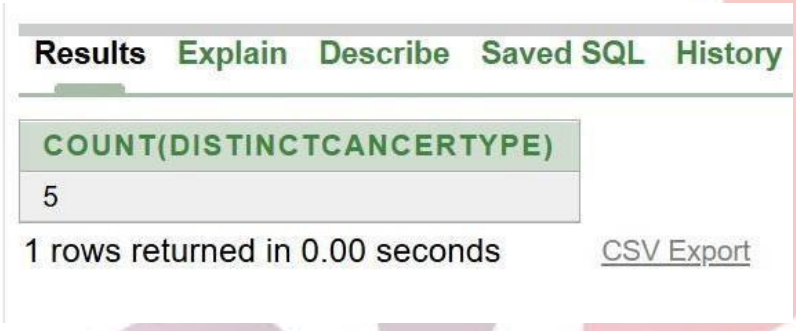
QUESTION:

(g) Write the output of the following SQL commands.
(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

INPUT:

SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

OUTPUT:



QUESTION:

(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;

INPUT:

SELECT MAX(SurvivalYears) FROM CANCERDATA;

OUTPUT:



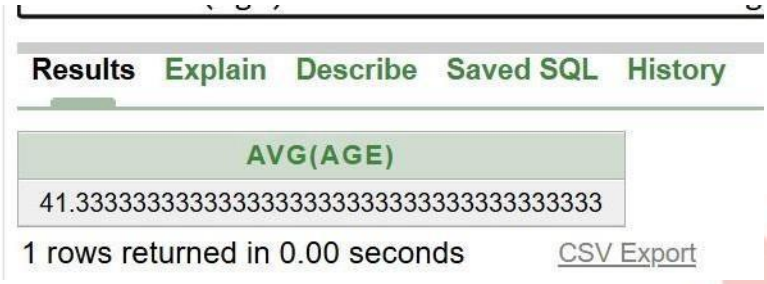
QUESTION:

(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

INPUT:

SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

OUTPUT:



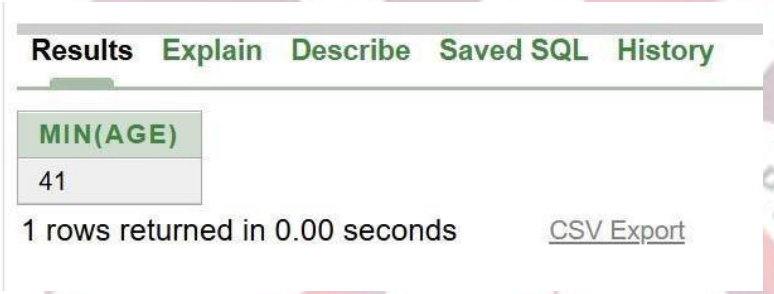
QUESTION:

(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

INPUT:

SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

OUTPUT:



19. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables ITEMS and TRADERS

(a) To display the details of all the items in ascending order of item names (i.e. INAME).

(b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

(c) To display the number of items, which are traded by each trader. The expected output of this query should be T01 2 T02 2 T03 1

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

(f) To display the name of the companies and the bane of the items in descending order of company names.

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

(g3) SELE CT DISTINCT TCODE FROM ITEMS;
(g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

ITEMS

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1001	DIGITAL PAD12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T02
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T01
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T02
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T03

TRADERS

TCODE	TNAME	CITY
T01	ELECTRONIC SALES	MUMBAI
T03	BUSY STORE CORP	DELHI
T02	DISP HOUSE INC	CHENNAI

SYNTAX:

CREATE TABLE ITEMS (CODE INT PRIMARY KEY, INAME VARCHAR(30), QTY INT, PRICE INT, COMPANY VARCHAR(20), TCODE VARCHAR(5));
INSERT INTO ITEMS VALUES (1001, 'DIGITAL PAD12i', 120, 11000, 'XENITA', 'T01');
INSERT INTO ITEMS VALUES (1006, 'LED SCREEN 40', 70, 38000, 'SANTORA', 'T03');
INSERT INTO ITEMS VALUES (1004, 'CAR GPS SYSTEM', 50, 21500, 'GEOKNOW', 'T02');
INSERT INTO ITEMS VALUES (1003, 'DIGITAL CAMERA 12X', 160, 8000, 'DIGICLICK', 'T01');
INSERT INTO ITEMS VALUES (1005, 'PEN DRIVE 32 GB', 600, 1200, 'STOREHOME', 'T02');
CREATE TABLE TRADERS (TCODE VARCHAR(5) PRIMARY KEY, TNAME VARCHAR(30), CITY VARCHAR(20));
INSERT INTO TRADERS VALUES ('T01', 'ELECTRONIC SALES', 'MUMBAI');
INSERT INTO TRADERS VALUES ('T03', 'BUSY STORE CORP', 'DELHI');
INSERT INTO TRADERS VALUES ('T02', 'DISP HOUSE INC', 'CHENNAI');

QUESTION:

(a) To display the details of all the items in ascending order of item names (i.e. INAME).

INPUT:

SELECT * FROM ITEMS ORDER BY INAME ASC;

OUTPUT:

Results Explain Describe Saved SQL History

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T02
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T01
1001	DIGITAL PAD12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T03
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T02

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

INPUT:

SELECT INAME, PRICE FROM ITEMS WHERE PRICE BETWEEN 10000 AND 22000;

OUTPUT:

Results Explain Describe Saved SQL History

INAME	PRICE
DIGITAL PAD12i	11000
CAR GPS SYSTEM	21500

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(c) To display the number of items, which are traded by each trader. The expected output of this query should be T01 2 T02 2 T03 1

INPUT:

SELECT TCODE, COUNT(*) FROM ITEMS GROUP BY TCODE;

OUTPUT:

Results Explain Describe Saved SQL History

TCODE	COUNT(*)
T01	2
T03	1
T02	2

3 rows returned in 0.00 seconds CSV Export

QUESTION:

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

INPUT:

SELECT PRICE, INAME, QTY FROM ITEMS WHERE QTY>150;

OUTPUT:



Results Explain Describe Saved SQL History

PRICE	INAME	QTY
8000	DIGITAL CAMERA 12X	160
1200	PEN DRIVE 32 GB	600

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

INPUT:

SELECT TNAME FROM TRADERS WHERE CITY='DELHI' OR CITY='MUMBAI';

OUTPUT:



Results Explain Describe Saved SQL History

TNAME
ELECTRONIC SALES
BUSY STORE CORP

2 rows returned in 0.00 seconds CSV Export

QUESTION:

(f) To display the name of the companies and the bane of the items in descending order of company names.

INPUT:

SELECT COMPANY, INAME FROM ITEMS ORDER BY COMPANY DESC;

OUTPUT:

Results Explain Describe Saved SQL History

COMPANY	INAME
XENITA	DIGITAL PAD12i
STOREHOME	PEN DRIVE 32 GB
SANTORA	LED SCREEN 40
GEOKNOW	CAR GPS SYSTEM
DIGICLICK	DIGITAL CAMERA 12X

5 rows returned in 0.00 seconds CSV Export

QUESTION:

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

INPUT:

SELECT MAX(PRICE), MIN(PRICE) FROM ITEMS;

OUTPUT:

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
38000	1200

1 rows returned in 0.00 seconds CSV Export

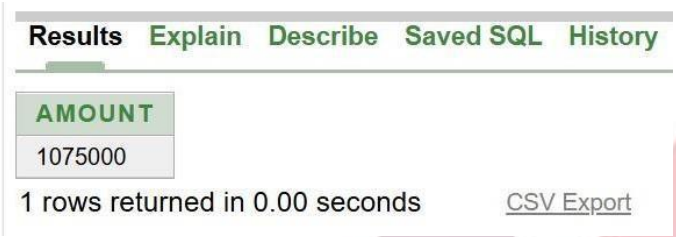
QUESTION:

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

INPUT:

SELECT PRICE*QTY AS AMOUNT FROM ITEMS WHERE CODE=1004;

OUTPUT:



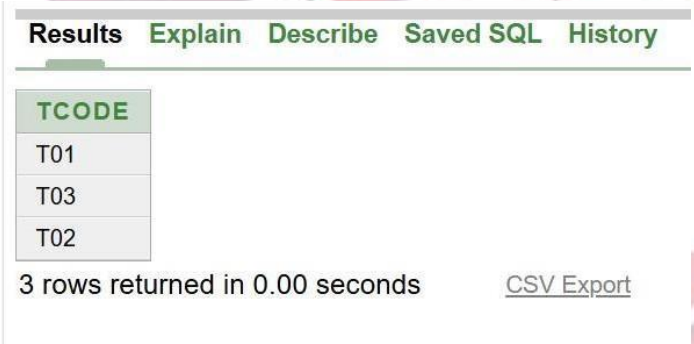
QUESTION:

(g3) SELE CT DISTINCT TCODE FROM ITEMS;

INPUT:

SELECT DISTINCT TCODE FROM ITEMS;

OUTPUT:



QUESTION:

(g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

INPUT:

SELECT INAME, TNAME FROM ITEMS I, TRADERS T WHERE I.TCODE=T.TCODE AND QTY<100;

OUTPUT:



Results Explain Describe Saved SQL History

INAME	TNAME
LED SCREEN 40	BUSY STORE CORP
CAR GPS SYSTEM	DISP HOUSE INC

2 rows returned in 0.01 seconds CSV Export

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

EX. NO: 13 MINI PROJECT

DATE: **05/09/2026** **BANKING ACCOUN AND TRANSACTION MANAGEMENT**

Aim

To develop a Banking Account and Transaction Management System using SQL and PL/SQL/SQLite to store, manage, process, and analyze banking transaction records such as deposits, withdrawals, transaction amounts, and account balances.

Software Used

- **• GitHub Codespaces**
- **• SQLite**
- **• Oracle Database**
- **• SQL**
- **• PL/SQL**

Description

The Banking Account and Transaction Management System is developed to manage bank account transactions and customer banking activities. The system stores account details, transaction dates, transaction types, and transaction amounts.

SQL queries are used to retrieve and analyze transaction information, calculate total deposits and withdrawals, identify customers with the highest transaction amounts, display transactions exceeding ₹50,000, and calculate monthly transaction totals.

PL/SQL procedures and functions are used to process deposits and withdrawals, validate transaction amounts, retrieve the current account balance, and generate monthly account statements with appropriate validation and error handling.

Database Tables

1. BANK_TRANSACTIONS

Column Name	Data Type	Constraint
RANSACTION_ID	NTEGER	RIMARY KEY
CCOUNT_ID	NTEGER	OT NULL
USTOMER_NAME	ARCHAR(50)	OT NULL
RANSACTION_DATE	ATE/TEXT	OT NULL
RANSACTION_TYPE	ARCHAR(20)	OT NULL

MOUNT ECIMAL(10,2) OT NULL

Table Create

```
DROP TABLE IF EXISTS BANK_TRANSACTIONS;


CREATE TABLE BANK_TRANSACTIONS (
   transaction_id INTEGER PRIMARY KEY,
   account_id INTEGER NOT NULL,
   customer_name TEXT NOT NULL,
   transaction_date TEXT NOT NULL,
   transaction_type TEXT NOT NULL CHECK(transaction_type IN ('DEPOSIT',
'WITHDRAWAL')),
   amount REAL NOT NULL CHECK(amount > 0)
);
```

25 Records Insert

```
INSERT INTO BANK_TRANSACTIONS
(transaction_id, account_id, customer_name, transaction_date, transaction_type, amount)
VALUES
(1, 1001, 'Arun',   '2026-01-05', 'DEPOSIT',    50000),
(2, 1001, 'Arun',   '2026-01-10', 'WITHDRAWAL', 10000),
(3, 1001, 'Arun',   '2026-01-20', 'DEPOSIT',    30000),
(4, 1001, 'Arun',   '2026-02-05', 'DEPOSIT',    45000),
(5, 1001, 'Arun',   '2026-02-15', 'WITHDRAWAL', 15000),

(6, 1002, 'Priya',  '2026-01-03', 'DEPOSIT',    70000),
(7, 1002, 'Priya',  '2026-01-12', 'WITHDRAWAL', 20000),
(8, 1002, 'Priya',  '2026-01-25', 'DEPOSIT',    25000),
```

(9, 1002, 'Priya', '2026-02-08', 'DEPOSIT',   60000),

(10, 1002, 'Priya', '2026-02-20', 'WITHDRAWAL', 10000),

(11, 1003, 'Karthik', '2026-01-02', 'DEPOSIT',   40000),

(12, 1003, 'Karthik', '2026-01-15', 'WITHDRAWAL', 5000),

(13, 1003, 'Karthik', '2026-01-28', 'DEPOSIT',   55000),

(14, 1003, 'Karthik', '2026-02-10', 'WITHDRAWAL', 20000),

(15, 1003, 'Karthik', '2026-02-25', 'DEPOSIT',   30000),

(16, 1004, 'Divya',   '2026-01-07', 'DEPOSIT',   90000),

(17, 1004, 'Divya',   '2026-01-18', 'WITHDRAWAL', 25000),

(18, 1004, 'Divya',   '2026-02-03', 'DEPOSIT',   35000),

(19, 1004, 'Divya',   '2026-02-17', 'WITHDRAWAL', 10000),

(20, 1004, 'Divya',   '2026-02-28', 'DEPOSIT',   50000),

(21, 1005, 'Rahul',   '2026-01-09', 'DEPOSIT',   30000),

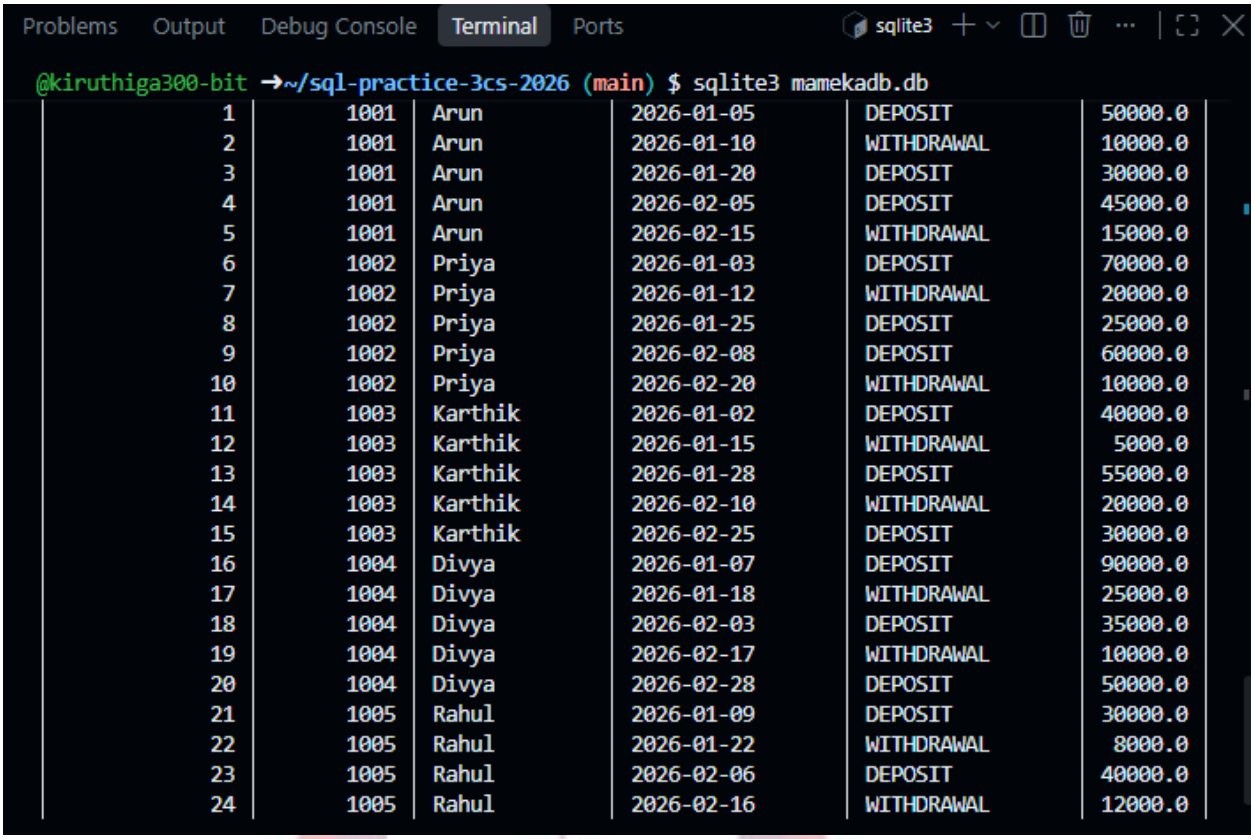
(22, 1005, 'Rahul',   '2026-01-22', 'WITHDRAWAL', 8000),

(23, 1005, 'Rahul',   '2026-02-06', 'DEPOSIT',   40000),

(24, 1005, 'Rahul',   '2026-02-16', 'WITHDRAWAL', 12000),

(25, 1005, 'Rahul',   '2026-02-26', 'DEPOSIT',   25000);

Output:



SQL Query 1

Display all transactions for a selected bank account

```
SELECT *
FROM BANK_TRANSACTIONS
WHERE account_id = 1001
ORDER BY transaction_date;
```

Output:

```
sqlite> SELECT *
FROM BANK_TRANSACTIONS
WHERE account_id = 1001
ORDER BY transaction_date;
```

transaction_id	account_id	customer_name	transaction_date	transaction_type	amount
1	1001	Arun	2026-01-05	DEPOSIT	50000.0
2	1001	Arun	2026-01-10	WITHDRAWAL	10000.0
3	1001	Arun	2026-01-20	DEPOSIT	30000.0
4	1001	Arun	2026-02-05	DEPOSIT	45000.0
5	1001	Arun	2026-02-15	WITHDRAWAL	15000.0

```
sqlite>
```

SQL Query 2

Calculate total deposit and withdrawal amounts

```
SELECT
   SUM(CASE
      WHEN transaction_type = 'DEPOSIT'
      THEN amount ELSE 0
   END) AS total_deposits,

   SUM(CASE
      WHEN transaction_type = 'WITHDRAWAL'
      THEN amount ELSE 0
   END) AS total_withdrawals
FROM BANK_TRANSACTIONS;
```

Output:

```
sqlite> SELECT
    SUM(CASE
        WHEN transaction_type = 'DEPOSIT'
        THEN amount ELSE 0
    END) AS total_deposits,

    SUM(CASE
        WHEN transaction_type = 'WITHDRAWAL'
        THEN amount ELSE 0
    END) AS total_withdrawals
FROM BANK_TRANSACTIONS;
```

total_deposits	total_withdrawals
675000.0	135000.0

```
sqlite>
```

SQL Query 3

Identify customers with highest total transaction amount

```
SELECT
    customer_name,
    account_id,
    SUM(amount) AS total_transaction_amount
FROM BANK_TRANSACTIONS
GROUP BY account_id, customer_name
ORDER BY total_transaction_amount DESC
LIMIT 1;
```

Output:

```
sqlite> SELECT
    customer_name,
    account_id,
    SUM(amount) AS total_transaction_amount
FROM BANK_TRANSACTIONS
GROUP BY account_id, customer_name
ORDER BY total_transaction_amount DESC
LIMIT 1;
```

customer_name	account_id	total_transaction...
Divya	1004	210000.0

```
sqlite>
```

SQL Query 4

Display transactions exceeding ₹50,000

SELECT *

FROM BANK_TRANSACTIONS

WHERE amount > 50000

ORDER BY amount DESC;

Output:

```
sqlite> SELECT *
FROM BANK_TRANSACTIONS
WHERE amount > 50000
ORDER BY amount DESC;
```

transaction_id	account_id	customer_name	transaction_date	transaction_type	amount
16	1004	Divya	2026-01-07	DEPOSIT	90000.0
6	1002	Priya	2026-01-03	DEPOSIT	70000.0
9	1002	Priya	2026-02-08	DEPOSIT	60000.0
13	1003	Karthik	2026-01-28	DEPOSIT	55000.0

```
sqlite>
```

SQL Query 5

Calculate monthly transaction totals

```
SELECT
    strftime('%Y-%m', transaction_date) AS month,
    SUM(amount) AS monthly_total
FROM BANK_TRANSACTIONS
GROUP BY strftime('%Y-%m', transaction_date)
ORDER BY month;
```

Output:

```
sqlite> SELECT
    strftime('%Y-%m', transaction_date) AS month,
    SUM(amount) AS monthly_total
FROM BANK_TRANSACTIONS
GROUP BY strftime('%Y-%m', transaction_date)
ORDER BY month;
```

month	monthly_total
2026-01	458000.0
2026-02	352000.0

```
sqlite>
```

PL/SQL Task 1 → SQLite Equivalent

Create a procedure to process a deposit or withdrawal after validating the transaction amount.

```
CREATE TRIGGER validate_transaction_amount

BEFORE INSERT ON BANK_TRANSACTIONS

FOR EACH ROW

WHEN NEW.amount <= 0

BEGIN

   SELECT RAISE(ABORT, 'Transaction amount must be greater than zero');

END;
```

Output:

```
sqlite> INSERT INTO BANK_TRANSACTIONS
(account_id, customer_name, transaction_date, transaction_type, amount)
VALUES
(1001, 'Arun', '2026-03-02', 'WITHDRAWAL', 999999);
Error near line 12: Insufficient funds
sqlite>
```

SELEC

T

PL/SQL Task 2 → Current Balance

Create a function to retrieve the current balance of an account.

```
SELECT

   account_id,

   customer_name,

   SUM(

      CASE

         WHEN transaction_type = 'DEPOSIT'
```

THEN amount

ELSE -amount

END

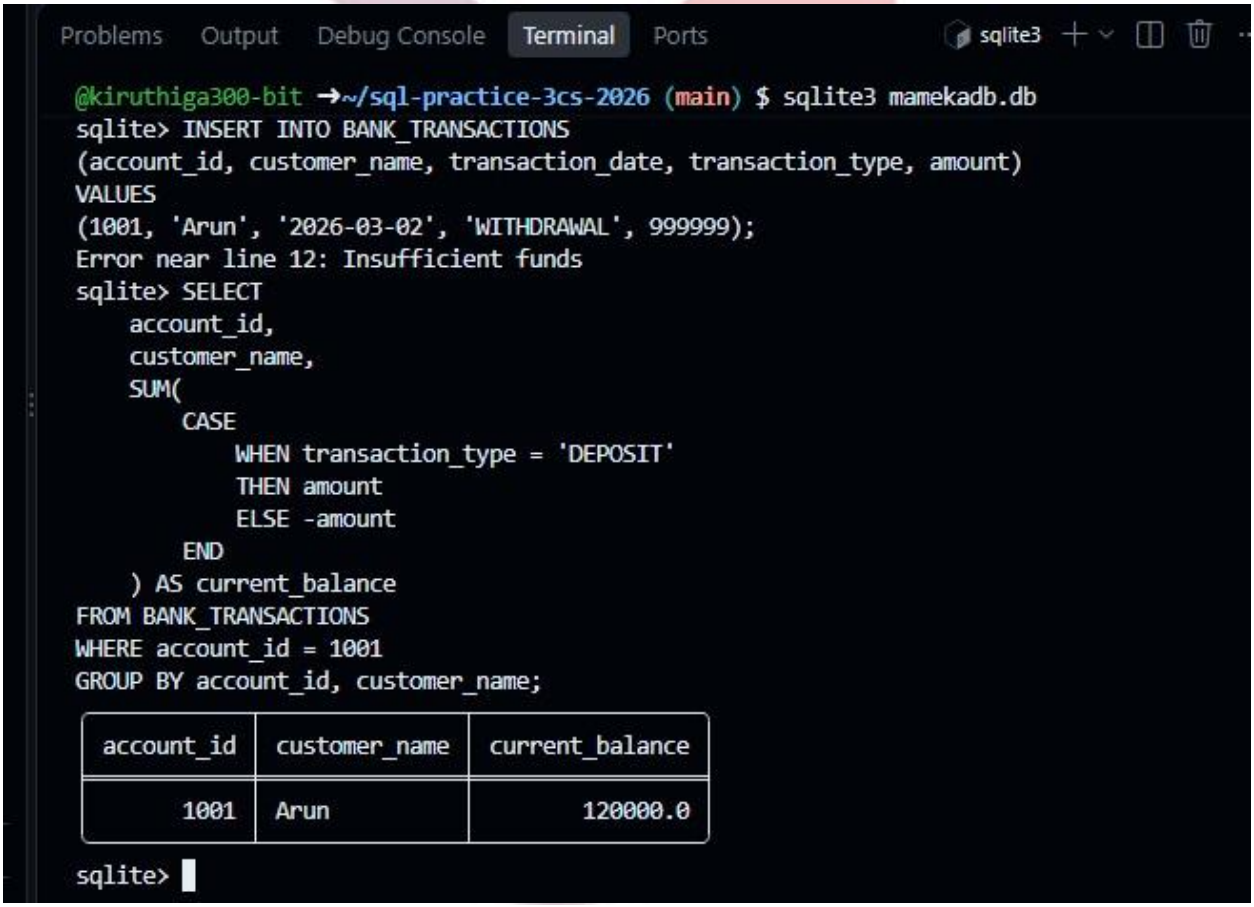
) AS current_balance

FROM BANK_TRANSACTIONS

WHERE account_id = 1001

GROUP BY account_id, customer_name;

output



PL/SQL Task 3 → Monthly Account Statement

Create a procedure to generate a monthly account statement, handling invalid account IDs and insufficient funds.

SELECT

transaction_id,

```
    account_id,
    customer_name,
    transaction_date,
    transaction_type,
    amount
FROM BANK_TRANSACTIONS
WHERE account_id = 1001
AND strftime('%Y-%m', transaction_date) = '2026-02'
ORDER BY transaction_date;
```

Output:

```
sqlite> SELECT
    transaction_id,
    account_id,
    customer_name,
    transaction_date,
    transaction_type,
    amount
FROM BANK_TRANSACTIONS
WHERE account_id = 1001
AND strftime('%Y-%m', transaction_date) = '2026-02'
ORDER BY transaction_date;
```

transaction_id	account_id	customer_name	transaction_date	transaction_type	amount
4	1001	Arun	2026-02-05	DEPOSIT	45000.0
5	1001	Arun	2026-02-15	WITHDRAWAL	15000.0

```
sqlite>
```

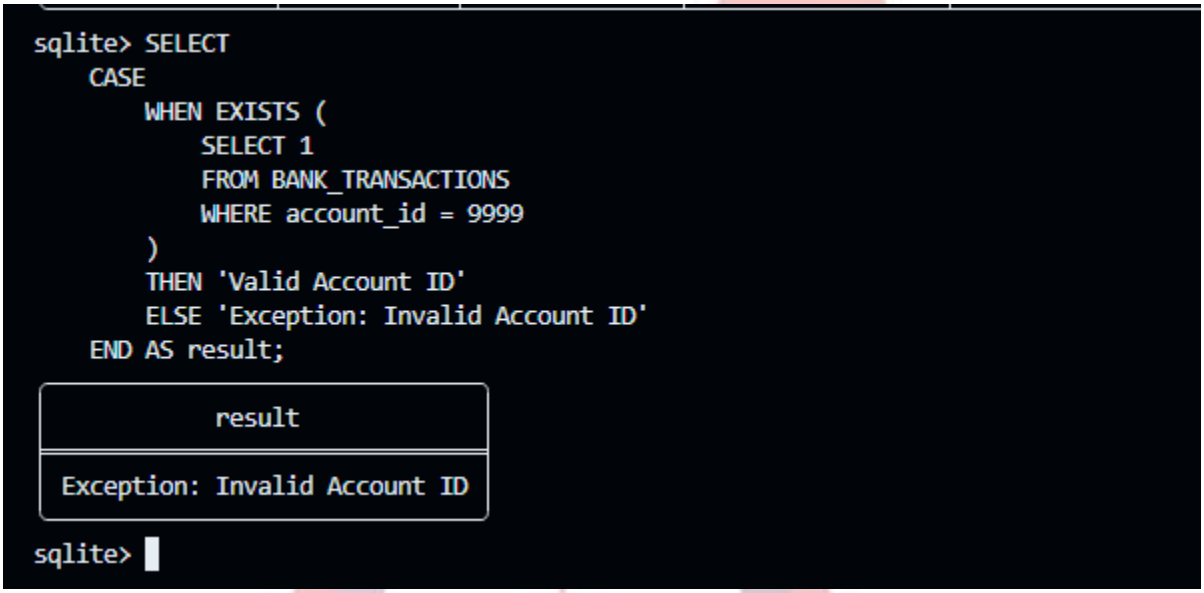
Exception Handling

Case 1 – Invalid Account ID

```
SELECT
  CASE
    WHEN EXISTS (
      SELECT 1
```

```
    FROM BANK_TRANSACTIONS
    WHERE account_id = 9999
  )
  THEN 'Valid Account ID'
  ELSE 'Exception: Invalid Account ID'
END AS result;
```

Output:

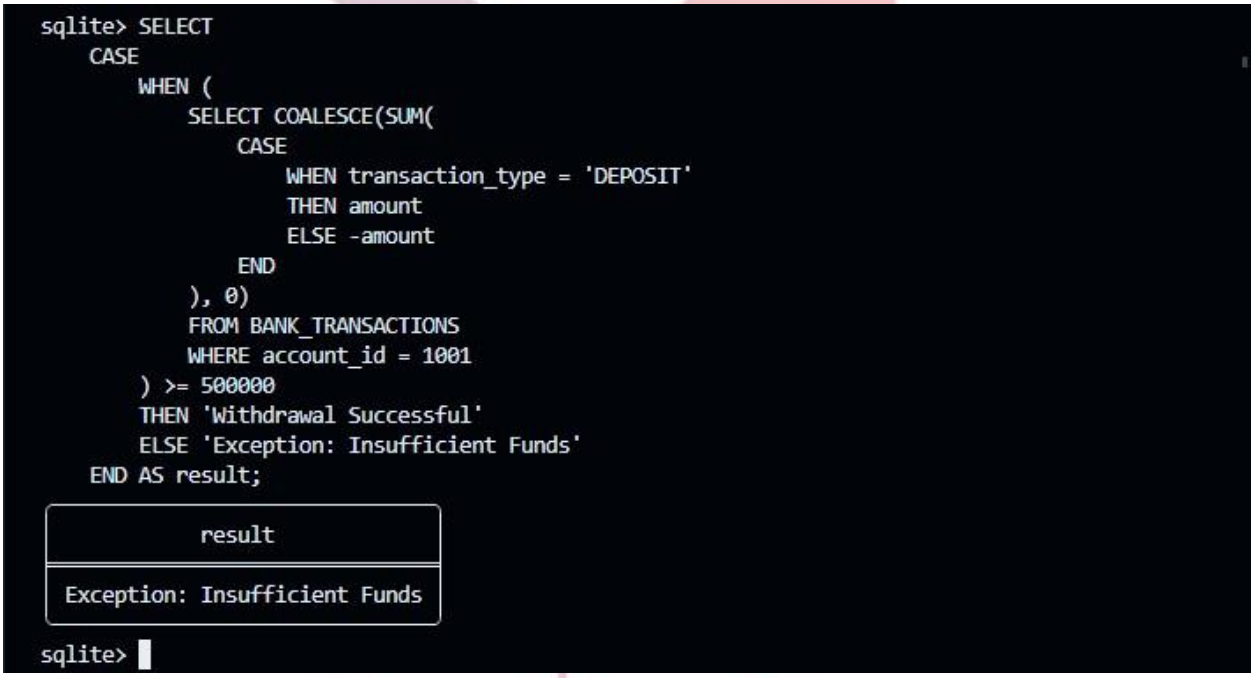


Case 2 – Insufficient Funds

```
SELECT
  CASE
    WHEN (
      SELECT COALESCE(SUM(
        CASE
          WHEN transaction_type = 'DEPOSIT'
          THEN amount
          ELSE -amount
        END
```

```
    ), 0)
    FROM BANK_TRANSACTIONS
    WHERE account_id = 1001
) >= 500000
THEN 'Withdrawal Successful'
ELSE 'Exception: Insufficient Funds'
END AS result;
```

Output:



Case 3 – Invalid Transaction Amount

```
SELECT
  CASE
    WHEN -500 > 0
    THEN 'Valid Transaction Amount'
    ELSE 'Exception: Invalid Transaction Amount'
  END AS result;
```

Output:

```
sqlite> SELECT
    CASE
        WHEN -500 > 0
        THEN 'Valid Transaction Amount'
        ELSE 'Exception: Invalid Transaction Amount'
    END AS result;
```

result
Exception: Invalid Transaction Amount

```
sqlite>
```

Dataset Insights

- **The dataset contains 25 banking transaction records.**
- **The transactions belong to 5 different bank accounts/customers.**
- **The transaction types include Deposit and Withdrawal.**
- **Transaction dates and amounts are recorded for each banking transaction.**
- **Deposit and withdrawal amounts can be compared to analyze account activity.**
- **Transactions exceeding ₹50,000 can be identified using SQL.**
- **Account-wise aggregation helps analyze the total transaction amount and account activity.**
- **Monthly transaction aggregation helps analyze deposit and withdrawal trends.**

Result

Thus, the Banking Account and Transaction Management system was successfully implemented using SQL and SQLite. The required table was created, 25 transaction records were inserted, and the required SQL queries were executed successfully. Transaction details, total deposits and withdrawals, highest transaction customers, transactions exceeding ₹50,000, and monthly transaction totals were analyzed successfully. SQLite-compatible validation and balance calculation operations were also implemented successfully.

Title: Khan Academy

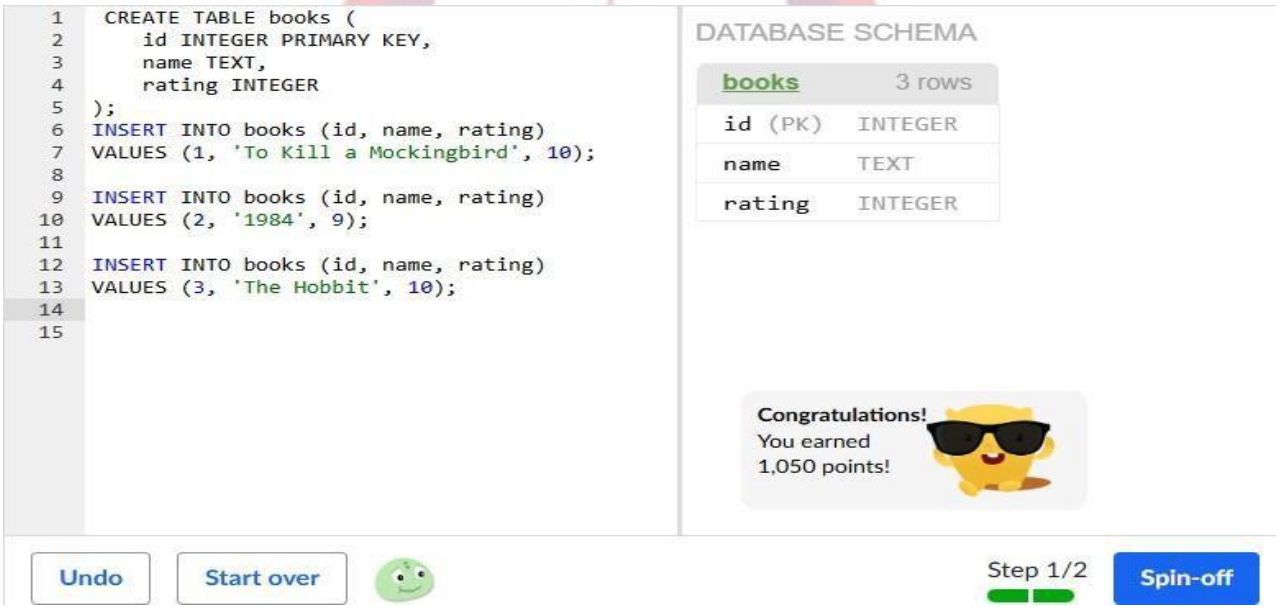
Challenge: Book list database

Step 1

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating.

Solution

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY,
  name TEXT,
  rating INTEGER
);
```

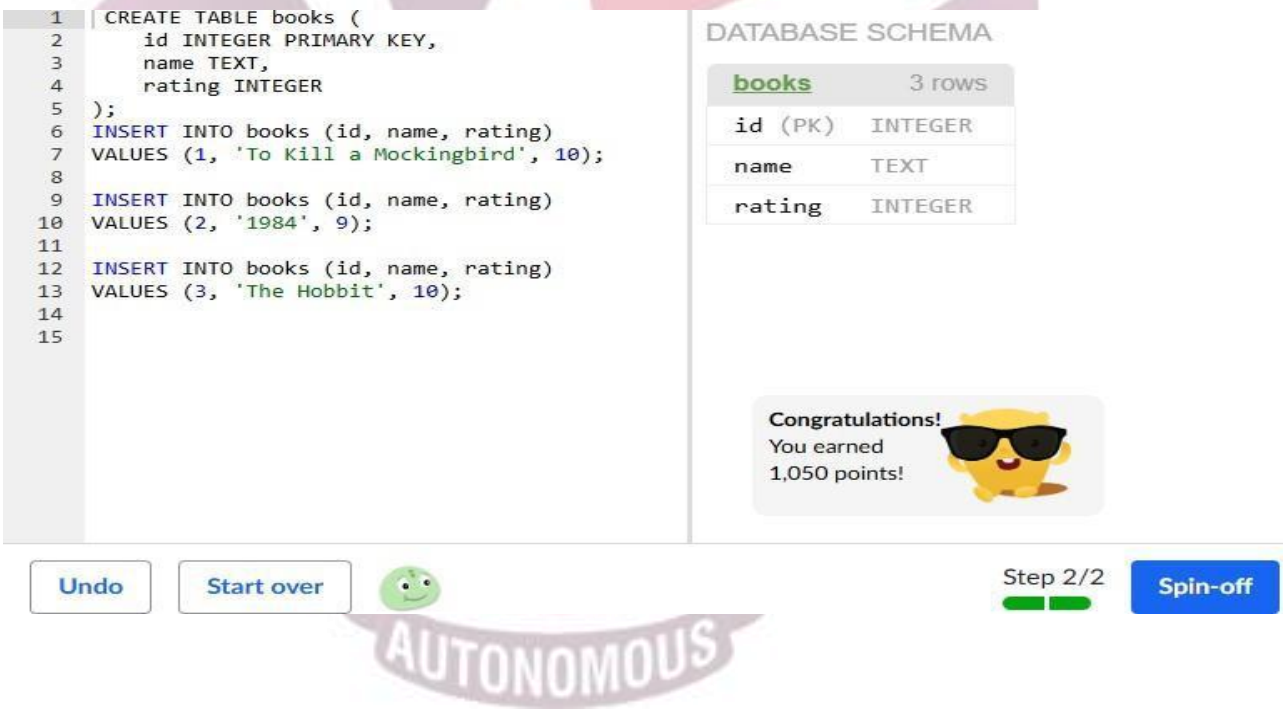


Step 2

Now, add three of your favorite books into the table.

Solution

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY,
  name TEXT,
  rating INTEGER
);
```



AUTONOMOUS

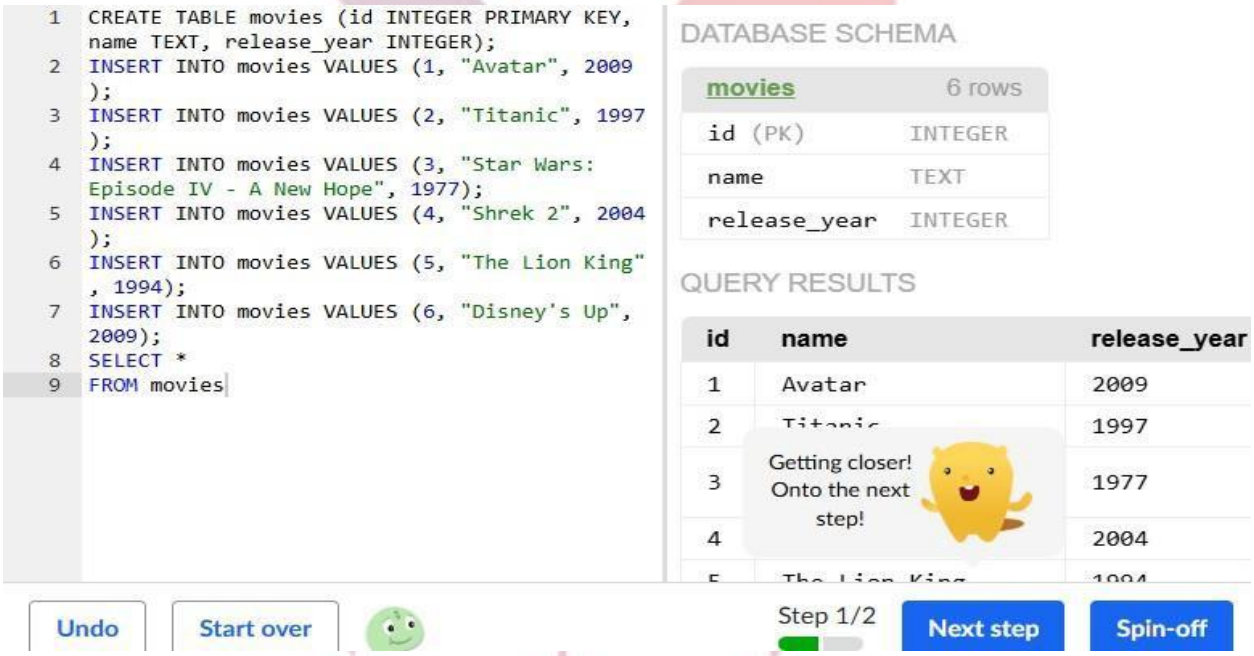
Challenge: Box office hits database

Step :

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies.

SOLUTION

CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);



Step 2

Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 SELECT statements after this step.

SOLUTION

CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

```
1  CREATE TABLE ,movies (id  INTEGIER PRIMARY KEY,
   name  TEXT,.  release_year  IIHEG1ER);
2  INSERT INTO movies VALUES (1,  "Avatar",  2009

   );
3  INSERT INTO movies VALUES (2,  "Titanic",, 1997
   L

4  INSERT INTO movies VALUES {3, "Star vlars:
   Episode IV - A New f--lope", 1977);
5  INSERT INTO movies VALUES (4, "Shrek 2"„ 2004
   );

6  INSERT INTO movies VALUES (5,  "The Lion King"
   ,  1994);
7  INSERT INTO movies VALUES (6,  "Disney's LJp",
   2009);
8  SELECT "
9  FRGJ-1  movies
10
```

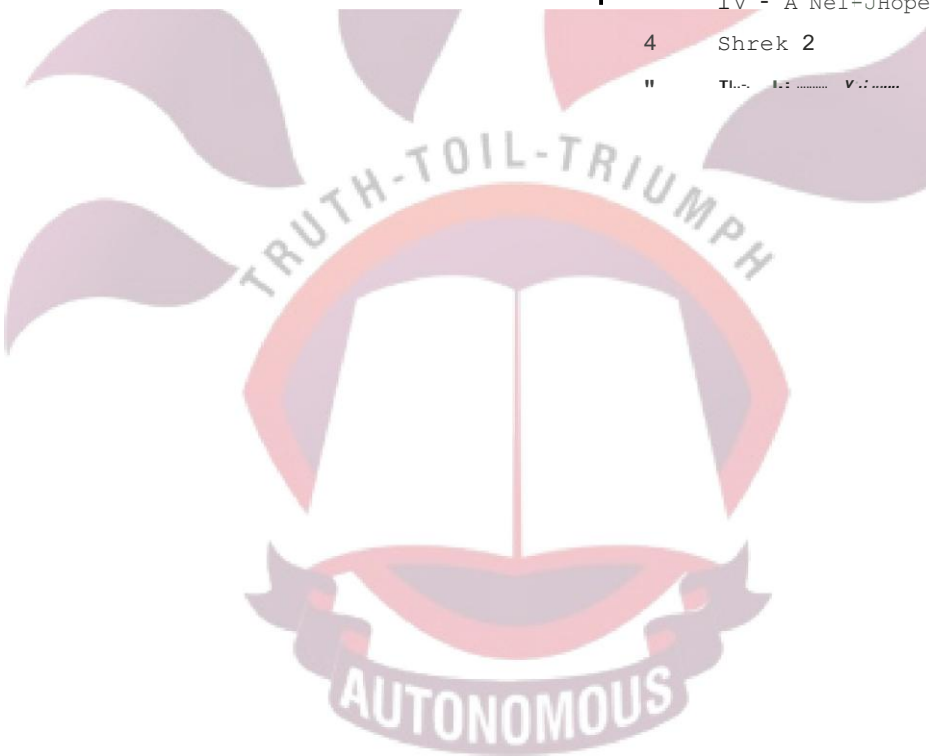
DATABASE SCHEMA

movies 6rows

id (PK)	INTEGER
name	TEXT
release_year	INTEGER

QUERY RESULTS

id	name	re'leasej'ear
1	Avatar	2009
2	Titanic	1997
3	Star Wars: Episode IV - A Nel-JHope	1977
4	Shrek 2	2004
"	TI..-. I.: ..... Y .:	...,{V(V1



Challenge: TODO list database stats

Step 1

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take.

SOLUTION

CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

```
1  CREATE TABLE todo_list (id INTEGER PRIMARY KEY
   , item TEXT, minutes INTEGER);
2  INSERT INTO todo_list VALUES (1, "Wash the
   dishes", 15);
3  INSERT INTO todo_list VALUES (2, "vacuuming",
   20);
4  INSERT INTO todo_list VALUES (3, "Learn some
   stuff on KA", 30);
5  INSERT INTO todo_list VALUES (4, " do
   homework", 45);
6  SELECT SUM(minutes)
7  FROM todo_list;
```

DATABASE SCHEMA

todo_list	4 rows
id (PK)	INTEGER
item	TEXT
minutes	INTEGER

QUERY RESULTS

SUM(minutes)
110

Congratulations!
You earned
2,100 points!



Step 2

Select the SUMof minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

SOLUTION

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

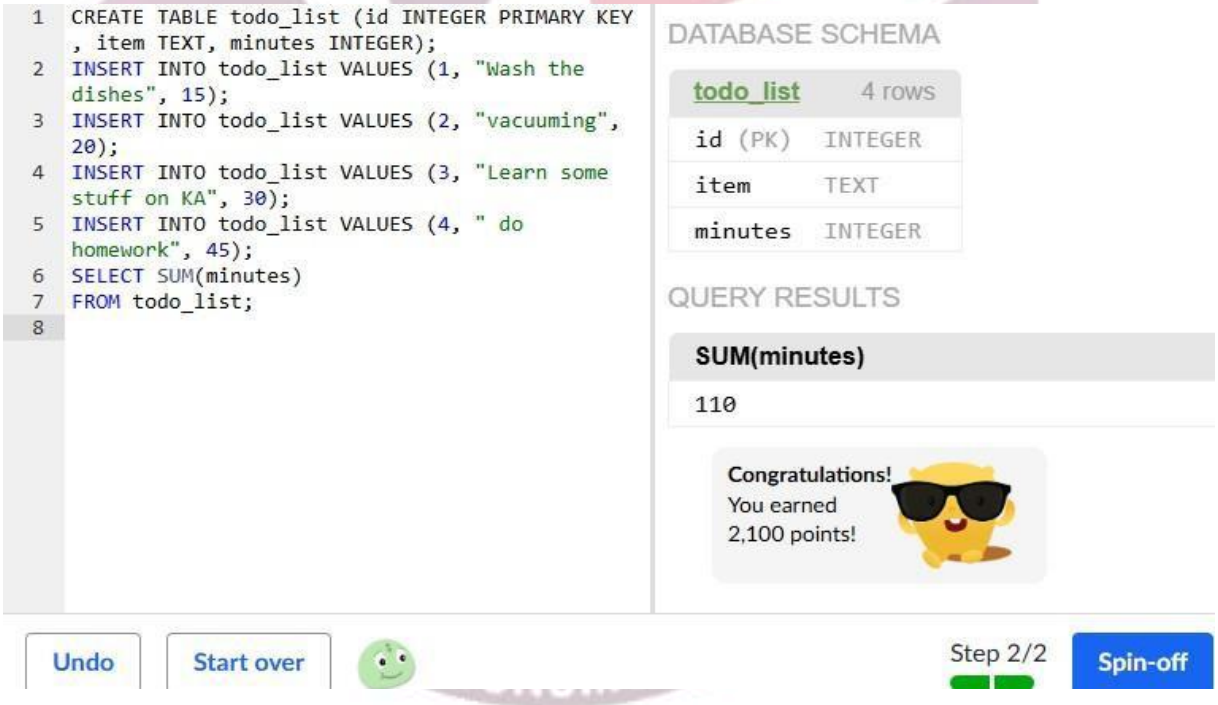
INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, " do homework", 45);

SELECT SUM(minutes)

FROM todo_list;
```



Project: Design a store database

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

SOLUTION

```
CREATE TABLE store
 ( id INTEGER PRIMARY KEY,
 name TEXT,
price INTEGER,
 category TEXT, quantity INTEGER );
 INSERT INTO store VALUES (1, "Pen", 2, "Stationery", 50);
 INSERT INTO store VALUES (2, "Pencil", 1, "Stationery", 100);
 INSERT INTO store VALUES (3, "Notebook", 5, "Stationery", 30);
 INSERT INTO store VALUES (4, "Eraser", 1, "Stationery", 40); INSERT  INTO store VALUES
(5, "Marker", 3, "Stationery", 25);
 INSERT INTO store VALUES (6, "Ruler", 2, "Stationery", 20);
 INSERT INTO store VALUES (7, "Glue", 4, "Stationery", 15);
 INSERT INTO store VALUES (8, "Scissors", 6, "Stationery", 10);
 INSERT INTO store VALUES (9, "Folder", 3, "Stationery", 18);
 INSERT INTO store VALUES (10, "Highlighter", 2, "Stationery", 22);  INSERT INTO store
VALUES (11, "Crayons", 7, "Stationery", 12);
 INSERT INTO store VALUES (12, "Tape", 3, "Stationery", 16);
 INSERT INTO store VALUES (13, "Stapler", 8, "Stationery", 8);
 INSERT INTO store VALUES (14, "Paper Pack", 10, "Stationery", 14); INSERT INTO store
VALUES (15, "Binder", 9, "Stationery", 6);
SELECT * FROM store ORDER BY price;
SELECT AVG(price) AS average_price
FROM store;
```

```
2  CREATE TABLE bikes {
3        id INTEGER PRIMARY KEY,
4        bike_name TEXT,
5        brand TEXT,
6        category TEXT,
7        color TEXT,
8        price INTEGER
g  );
10
11     Insert 15 items
12  INSERT INTO bikes VALUES {1, 'Mountain Pro'
     , 'Hero·', 'Mountain', 'Black', 18000);
13  INSERT INTO bikes VALUES {2, 'Road King',
     'Trek', 'Road', 'Red', 25000);
14  INSERT INTO bikes VALUES {3, 'Speed Rider',
     'Giant', 'Road', 'Blue', 22000);
15  INSERT INTO bikes VALUES {4, 'Trail Master'
     , 'Hero', 'Mountain', 'Green', 20000);
16  INSERT INTO bikes VALUES (5, 'City Ride',
     'Atlas', 'City', 'White', 12000);
17  INSERT INTO bikes VALUES (6, 'Urban X',

     'Firefox', 'City', 'Black', 15000);
```

DATABASE SCHEMA

bikes 15 rows

id (PK)	INTEGER
bike name	TEXT
bra11d	TEXT
category	TEXT
color	TEXT
price	INTEGER

QUERY RESULTS

id	l::Jike_11ame	:brand	category
9	Eco Ride	Atlas	City
;			
	<:it-v Ri ,J.,	At-lai≤a	r:-if-v

Start over] [Request help



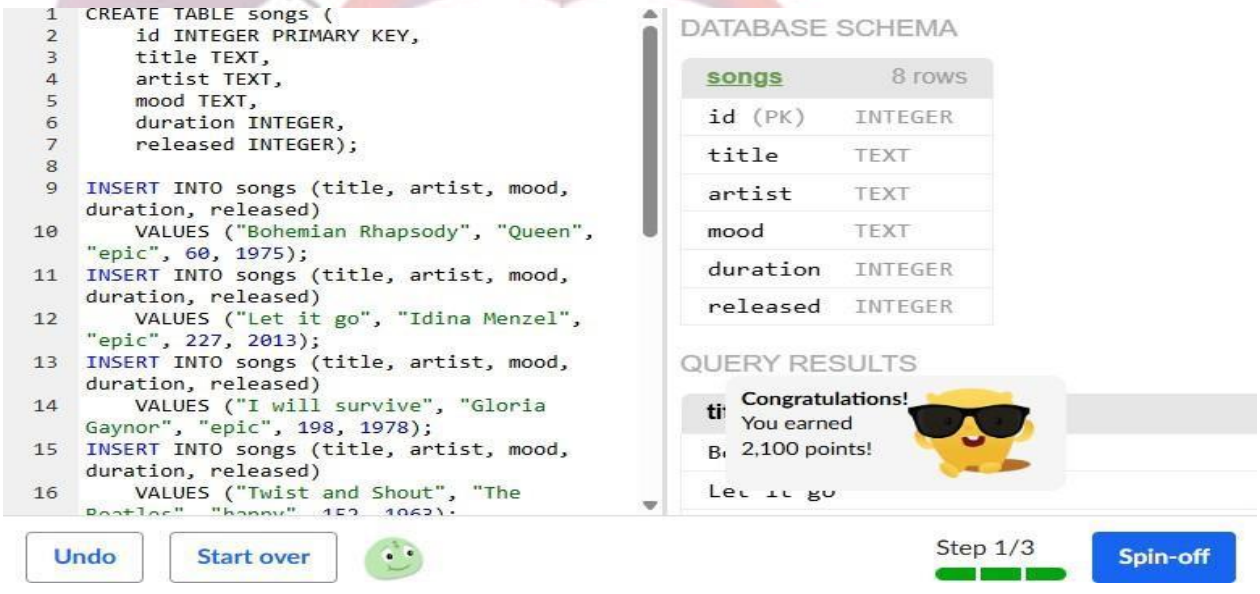
Challenge: Karaoke song selector

Step 1

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles.

SOLUTION

```
CREATE TABLE songs (
  id INTEGER PRIMARY KEY,
  title   TEXT,
  artist TEXT,
  mood TEXT,
  duration INTEGER,
  released INTEGER);
```



Step 2

Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECTthat uses ORto show the titles of the songs that have an 'epic' mood or a release date after 1990.

(Tip: If you're not sure how to select rows where a column equals a value, remember that you can check the documentation below.)

SOLUTION

FROM

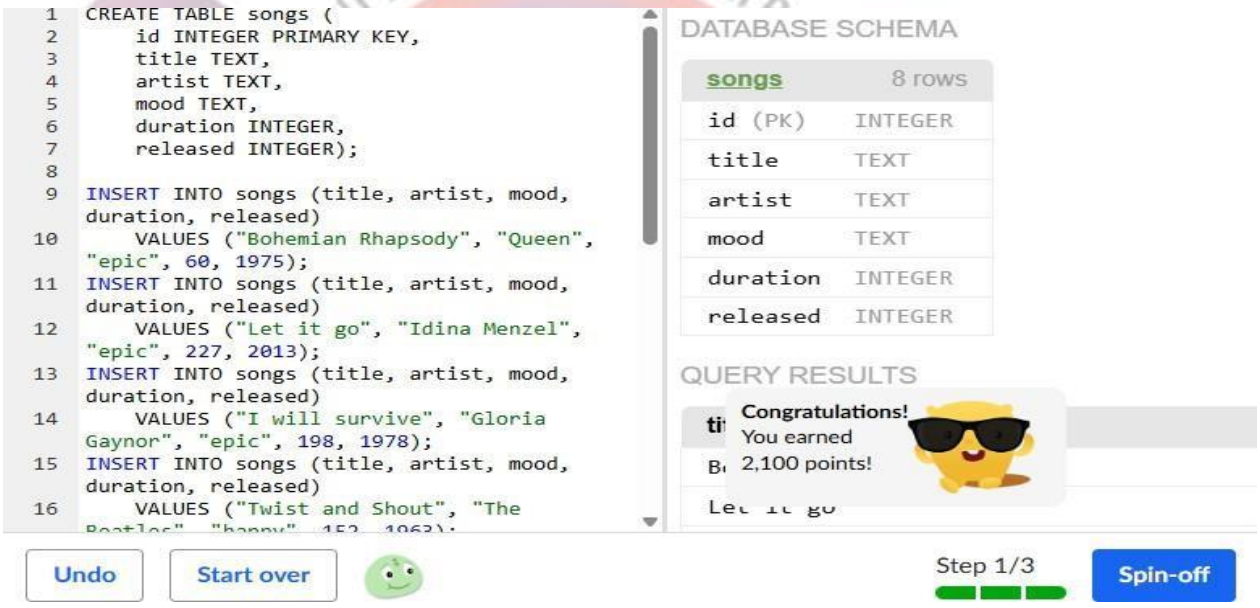
songs;

SELECT title

FROM songs

WHERE mood = 'epic'

OR released > 1990;



Step 3

People get picky at the end of the night. Add another SELECTthat uses ANDto show the titles of songs that are 'epic', and released after 1990, and less than 4 minutes long.
Note that the durationcolumn is measured in seconds.

SOLUTION

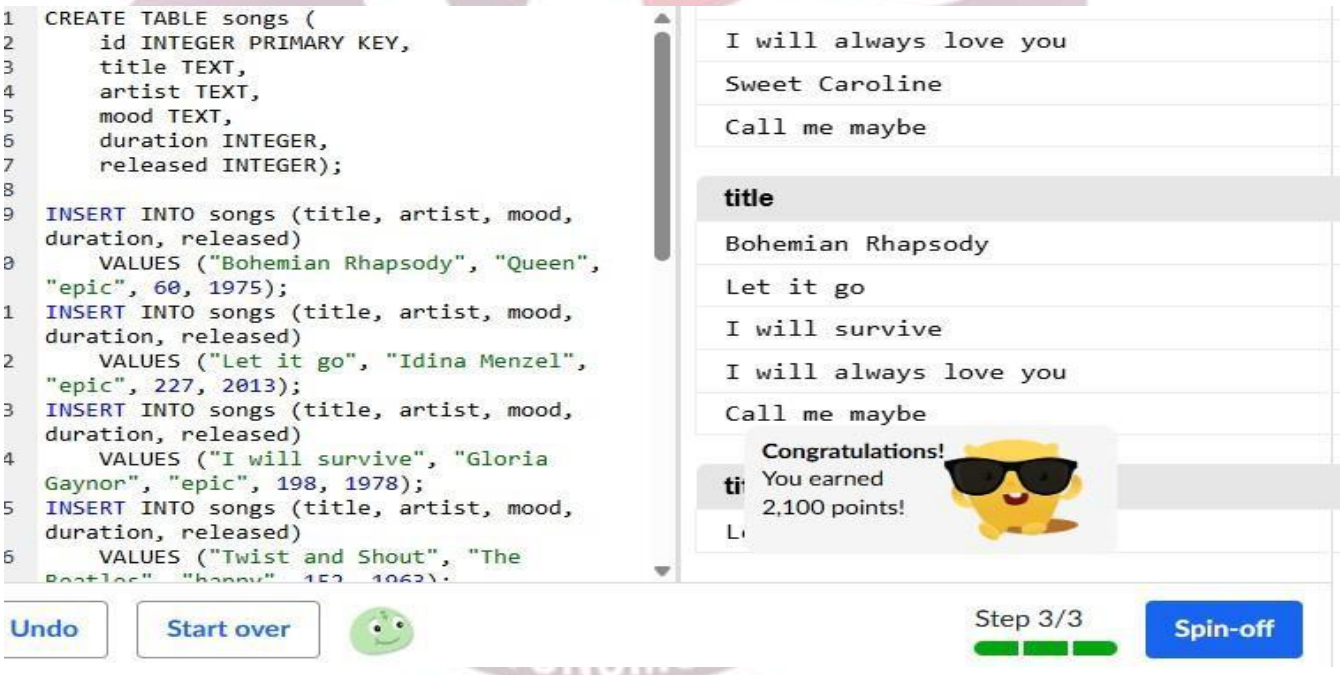
SELECT title

FROM songs

WHERE mood = 'epic'

AND released > 1990

AND duration < 240;



Challenge: Playlist maker

Step 1

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the titleof all the songs by the artistnamed 'Queen'.

SOLUTION

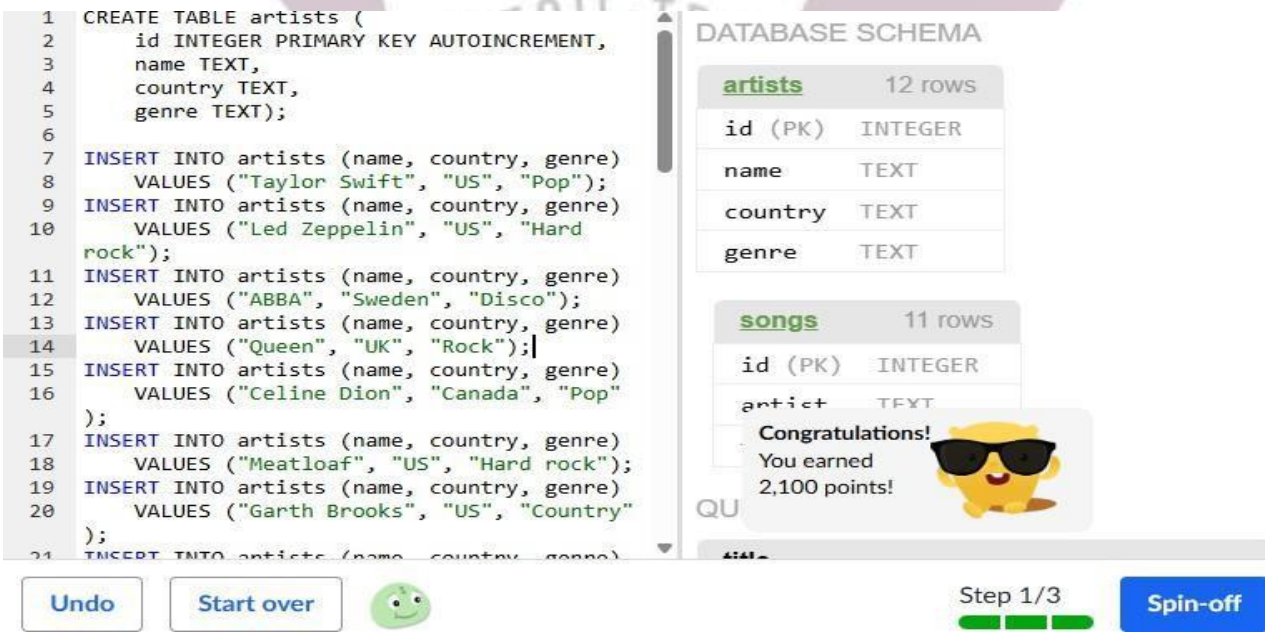
CREATE TABLE artists (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT,

country TEXT,

genre TEXT);



Step 2

Now you'll make a 'Pop' playlist. In preparation, select the nameof all of the artists from the 'Pop' genre.
(Tip: Make sure you type it 'Pop', SQL considers that different from 'pop'.)

SOLUTION

```
INSERT INTO artists (name, country, genre)
  VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
  VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
  VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
  VALUES ("Queen", "UK", "Rock");
  INSERT INTO artists (name, country, genre)
  VALUES ("Celine Dion", "Canada", "Pop");
  INSERT INTO artists (name, country, genre)
  VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
  VALUES ("Garth Brooks", "US",
"Country"); INSERT INTO artists (name,
country, genre)
```

VALUES ("Shania Twain", "Canada", "Country");

INSERT INTO artists (name, country, genre)

VALUES ("Rihanna", "US", "Pop");

INSERT INTO artists (name, country, genre)

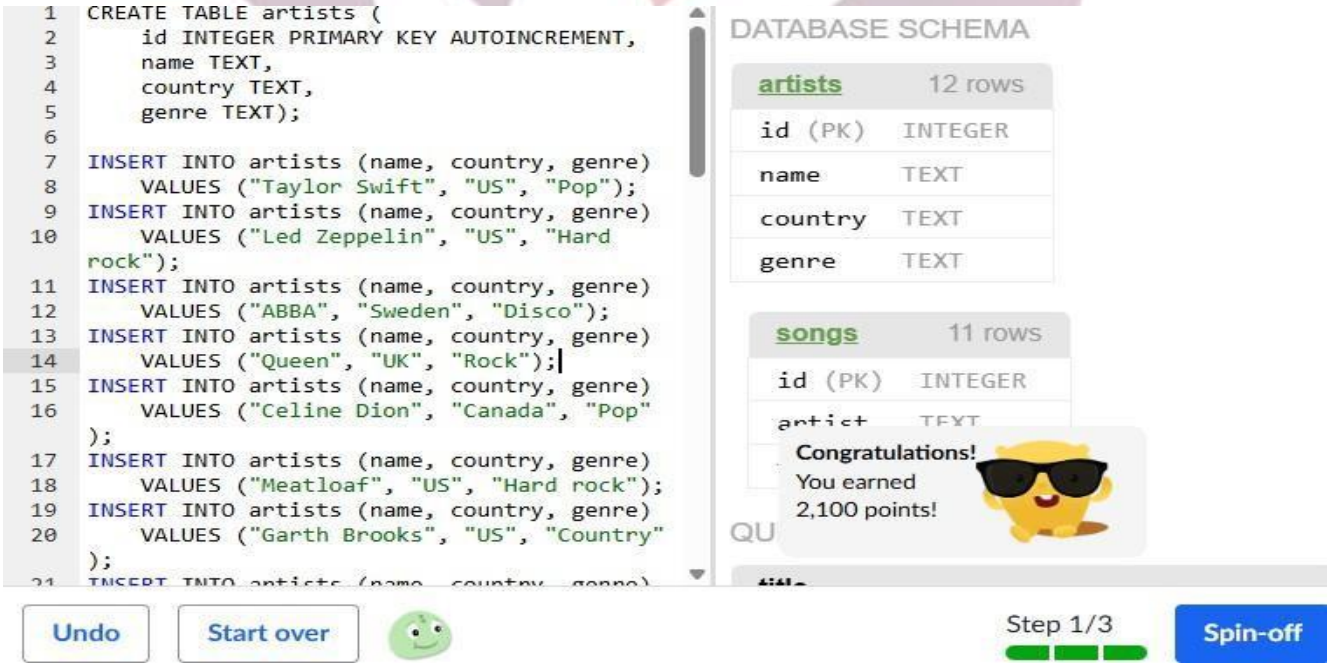
VALUES ("Guns N' Roses", "US", "Hard rock");

INSERT INTO artists (name, country, genre)

VALUES ("Gloria Estefan", "US", "Pop");

INSERT INTO artists (name, country, genre)

VALUES ("Bob Marley", "Jamaica", "Reggae");



Step 3

To finish creating the 'Pop' playlist, add another query that will select the titleof all the songs from the 'Pop' artists. It should use INon a nested subquery that's based on your previous query.

SOLUTION

```
CREATE TABLE songs (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   artist TEXT,
   title TEXT);
INSERT INTO songs (artist, title)
 VALUES ("Taylor Swift", "Shake it off");
 INSERT INTO songs (artist, title)
 VALUES ("Rihanna", "Stay");
 INSERT INTO songs (artist, title)
   VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
   VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
   VALUES ("Shania Twain", "Partyfortwo");
INSERT INTO songs (artist, title)
   VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
   VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
   VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
```

VALUES ("Queen", "Bicycle Race");

INSERT INTO songs (artist, title)

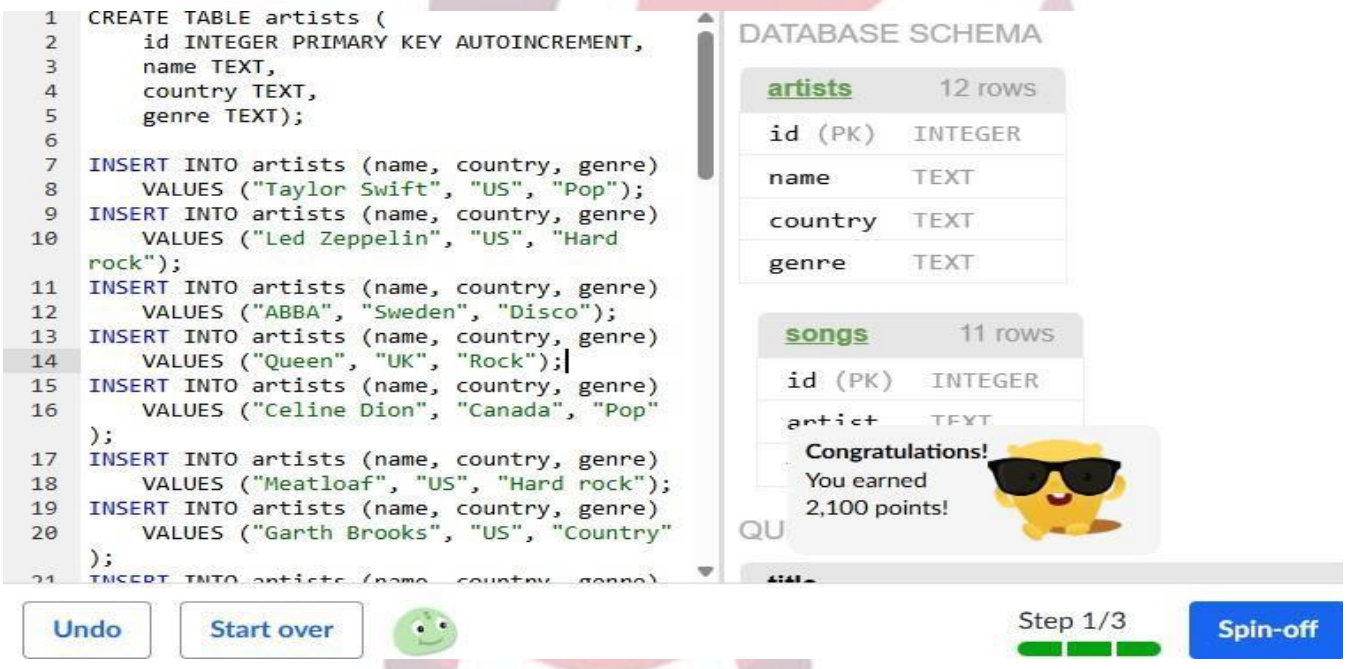
VALUES ("Queen", "Bohemian Rhapsody");

INSERT INTO songs (artist, title)

VALUES ("Guns N' Roses", "Don't cry");

SELECT title

FROM songs



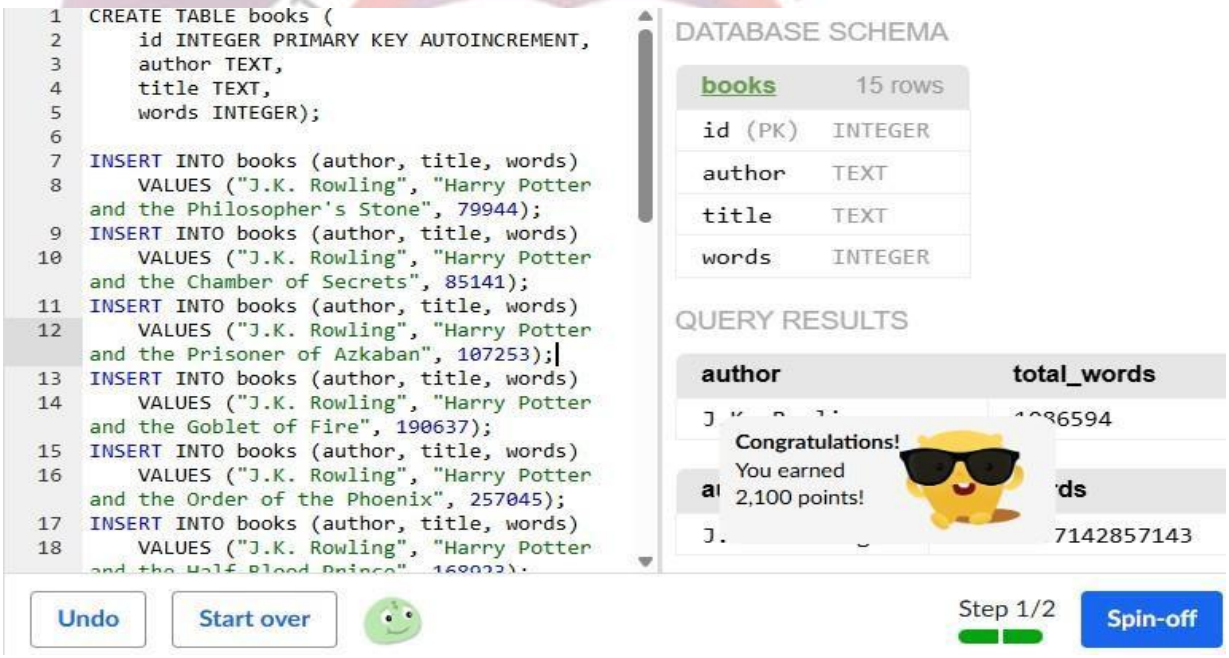
Challenge: The wordiest author

Step 1

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using GROUP BY and HAVING. Your results table should include the 'author' and their total word count as a 'total_words' column.

SOLUTION

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);
```



Step 2

Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

SOLUTION

```
INSERT INTO books (author, title, words)
  VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944); INSERT
INTO books (author, title, words)
  VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141); INSERT
INTO books (author, title, words)
  VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
  VALUES ("J.K. Rowling", "Harry Potterandthe Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
  VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
    INSERT INTO books (author, title, words)
  VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923); INSERT
INTO books (author, title, words)
  VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);
      INSERT INTO books (author, title, words)
  VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
  VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
```

```
   VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);


INSERT INTO books (author, title, words) VALUES
   ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
SELECT author,
     SUM(words) AS total_words
FROM books
GROUP BY author
       HAVING SUM(words) > 1000000;
SELECT author,
     AVG(words) AS avg_words
FROM books
GROUP BY author
HAVING AVG(words) > 150000;
```

```
1  CREATE TABLE books (
2      id INTEGER PRIMARY KEY AUTOINCRE□1ENT,
3      autho.r TEXT,
4      title TEXT,
5      words INTEGER);
6
7  INSERT INTO books (author, title, words)
8      VALUES ("J .K. Rowling"", ""Harry Potter
   and the Philosopher·s Stone"", 79944);,
9  INSERT INTO books (author, title, words)
10     VALUES (""J.K. Rowling"", ""Harry Potter
   and the Chamber o-f Secrets··, 85141};
11 INSERT INTO books (author, title, words)
12     VALUES (""J.K. Rowling"", ""Harry Potter
   and the Prisoner of Az.kaban"", :1'07253);1
13 INSERT INTO books (autihor, title, words)
14     VALLJES (""J.K. Rowling"", ""Harry Potter
   and the Goblet of Fire"", 190637);
15 INSERT INTO books (author, title. words)
16     VALUES (""J.K. Rowling"", ""Harry Potter
   and the ·Order of the ·Phoenix"", 257045);
17 INSERT INTO books (author, title, words)
18     VALUES (""J.K. Rowling"", '"Harry Potter
```

DATABASE SCHEMA

books 15 rows

id (PK)	INTEGER
autnor	TEXT
title	TEXT
words	INTEGER

QUERY RESULTS

author	total_words
J "'	-□"1:6594

Congratulations!,-.-
You earned ._,._□

al 2,100 points! .., ds

J. 7142857143

Undo] [Start over] Step 1/2.



Challenge: Gradebook

Step 1

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completedcolumn.

SOLUTION

CREATE TABLE student_grades (

id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT, number_grade

INTEGER,

fraction_completed REAL);

TRUTH-TOIL-TRIUMPH

```
1
2  CREATE TABLE student_grades (
3      id INTEGER PRIMARY KEY AUTOINCREMENT,
4      name TEXT,
5      number_grade INTEGER,
6      fraction_completed REAL);
7
8  INSERT INTO student_grades (name, number_gr
   ade, fraction_completed)
9      VALUES ("Winston", 90, 0.805);
10 INSERT INTO student_grades (name, number_gr
   ade, fraction_completed)
11     VALUES ("Winnefer", 95, 0.901);
12 INSERT INTO student_grades (name, number_gr
   ade, fraction_completed)
13     VALUES ("Winsteen", 85, 0.906);
14 INSERT INTO student_grades (name, number_gr
   ade, fraction_completed)
15     VALUES ("Wincifer", 66, 0.7054);
16 INSERT INTO student_grades (name, number_gr
   ade, fraction_completed)
17     VALUES ("Winster", 76, 0.5013);
18 INSERT INTO student_grades (name, number_gr
```

DATABASE SCHEMA

student_grades	6 rows
id (PK)	INTEGER
name	TEXT
number_grade	INTEGER
fraction_completed	REAL

QUERY RESULTS

name	number_grade	percent_c
Wi		81
W		90
W		91
Win		71

Congratulations!
You earned
2,100 points!

Undo | Start over | Step 1/2 | Spin-off

Step 2

Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_gradeby using CASEwith the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNTwith GROUP BYto show the number of students with each of those grades.

SOLUTION

```
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
  ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
  ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
      VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
  ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
  ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed) VALUES
  ("Winstonia", 82, 0.9045);
SELECT name,
    number_grade,
    ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
```

```
1
2  CREATE  TABLE  stude,nt_;grades  (
3        id  INTEGER PRIMARY  KEY  AUTOI.NCREMENT,

4        name  TEXT,
5        n,umbet'_grade  INTEGER,
6        ft'action_completed  REAl);

7
8  UISERT IIHO student_grades (name, number_gr
   ade,   fraction_completed)
9        VALUES  ("Winston",  90,  0.805);
10 INSERT IlffO student_grades (name,. number_g,r,
   ade,   fpaction_completed)
11       VAIUES  ( "\•linnefer",  95,.  0. 901.);
1.2 INSERT INTO student_grades (name,. number_gr
   ac:le,  fpaction_completed)

1.3      VALUES  ( "\,tinsteen",  85,.  0. 906);
'.IA INSERT IHTO student_grades (name,. number_gr
   ade,   fraction_completed)

15       VALUES  ("\1/incifer",  66,  0.7054);
1.6 INSERT INTO student_grades  name,. number_gr
   ade,  f,ractio.n_complet,ed)
1.7      VALUES  ("l-linster",  76,  0.5013);
18 INSERT  INTO  student_grades  (name,.  number_gr
```

Undo] [Start over

DATABASE SCHEMA

student grades 6 rows

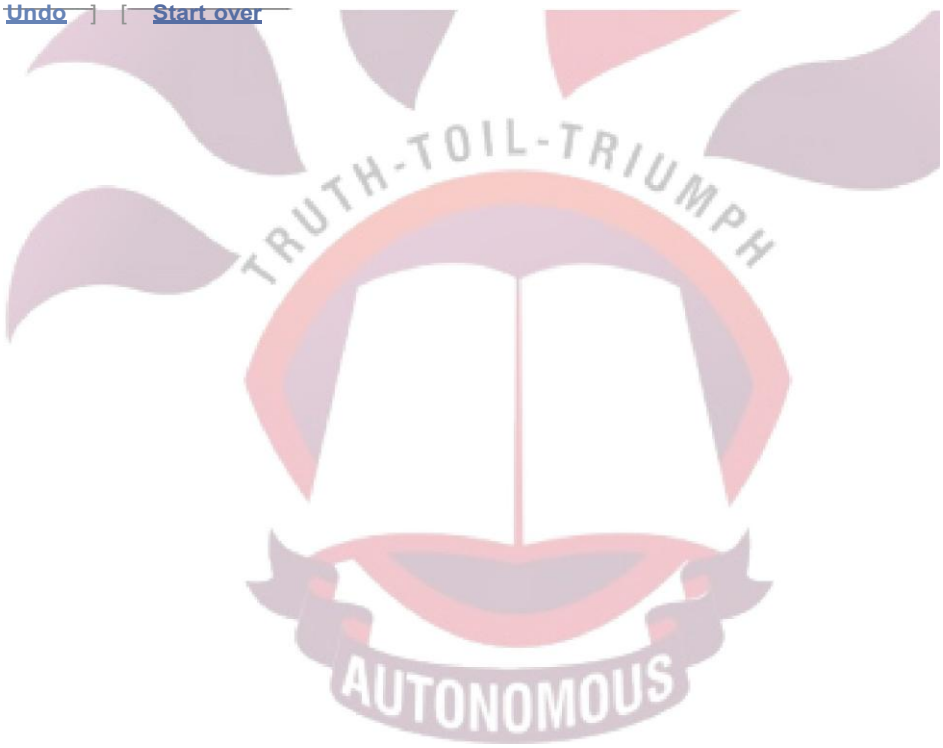
id (PK)	INTEGER
name	TEXT
number_grade	INTEGER
fractio:n_completed	REAL

QUERY RESULTS

name	11umber_girade	percent_c
W		81
Congratulations!		
W: You earned		90
W 2,100 points!		91
w-ifi[		71

2

iii:1·111



Project: Data dig

Pick one of those data sets or create a data set like that, and use advanced SELECTqueries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:

What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?

SOLUTION

```
CREATE TABLE pokemon
 ( id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,
 type TEXT,
hp INTEGER, attack INTEGER );
INSERT INTO pokemon (name, type, hp, attack)
 VALUES ("Pikachu", "Electric", 35, 55),
 ("Charizard", "Fire", 78, 84), ("Blastoise", "Water", 79, 83),
("Bulbasaur", "Grass", 45, 49), ("Arcanine", "Fire", 90, 110),
("Gyarados", "Water", 95, 125), ("Vulpix", "Fire", 38, 41), ("Squirtle",
"Water", 44, 48), ("Flareon", "Fire", 65, 130), ("Lapras", "Water", 130, 85);
SELECT CASE WHEN attack >= 100 THEN 'Strong' WHEN attack >= 70
THEN 'Average' ELSE 'Weak' END AS strength,
COUNT(*) AS total_pokemon,
AVG(attack) AS average_attack,
MAX(attack) AS highest_attack,
 MIN(attack) AS lowest_attack FROM pokemon
 WHERE hp >= 40
AND (type = 'Fire' OR type = 'Water')
GROUP BY strength HAVING COUNT(*) >= 1;
```

```sql
-- Bike Store Advancerl SQL Project

CREATE TABLE bikes (
    id INTEGER PRIMARY KEY,
    bike_name TEXT,
    brand TEXT,
    category TEXT,
    color TEXT,
    price INTEGER
);

INSERT INTO bikes VALUES (1, 'Mountain Pro', 'Hero·, ·Mountain', 'Black',. 18000);
INSERT INTO bikes VALUES (2, 'Road King', 'Trek' ,, "Road,' 'Red,' 25000);
INSERT INTO bikes VALUES (3, 'Speed Rider', 'Giant', 'Road', 'Blue', 2.2000);

INSERT INTO bikes VALUES (4, 'Trail Master', 'Hero·, ·Mountain,· 'Green,' . 2.0000);
INSERT INTO bikes VALUES (5, 'City Ride', 'Atlas', 'City', 'White', 12000);
INSERT INTO bikes VALUES (6, 'Urban X',I 'Firefox', 'City', 'Black', 15000);
```

Start over | Request help

DATABASE SCHEMA

bikes 15 rows

id (PK)	INTEGER
bike_name	TEXT
brand	TEXT
category	TEXT
color	TEXT
price	INTEGER

QUERY RESULTS

average:_Price	:maximum_price
20333.333333333332	30000

4

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Challenge: Bobby's Hobbies

Step 1

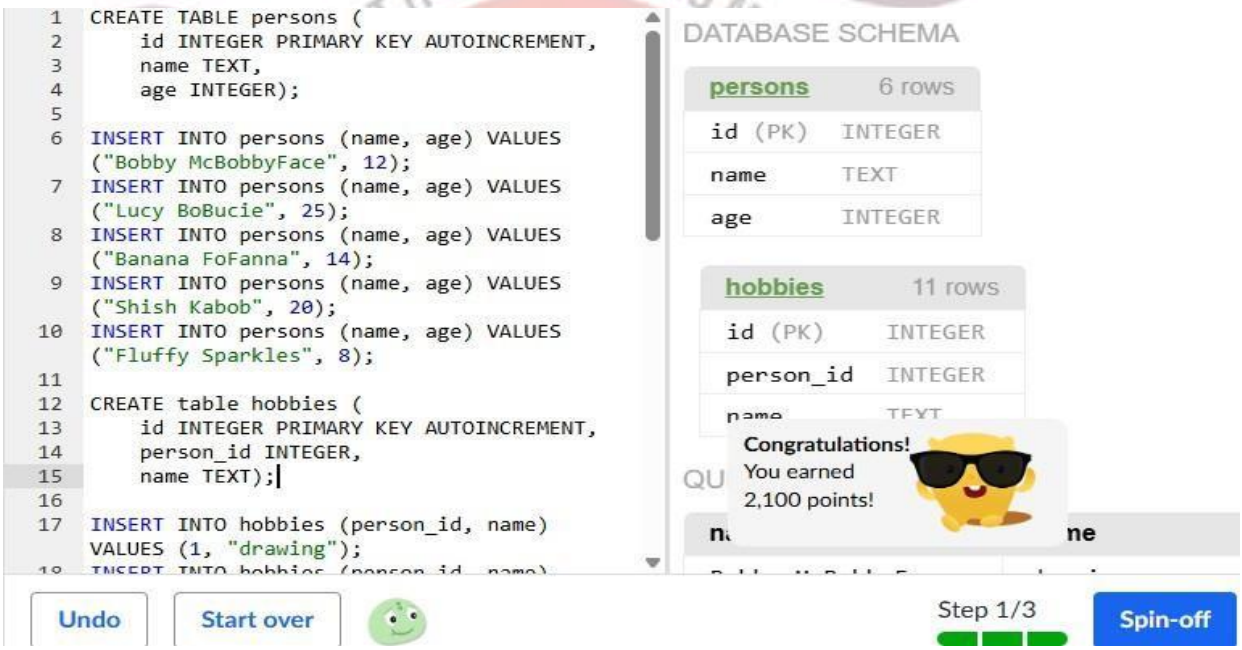
We've created a database of people and hobbies, and each row in hobbiesis related to a row in personsvia the person_idcolumn. In this first step, insert one more row in personsand then one more row in hobbiesthat is related to the newly inserted person.

SOLUTION

CREATE TABLE persons (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT,
age INTEGER);



Step 2

Now, select the 2 tables with a join so that you can see each person's name next to their hobby.

SOLUTION

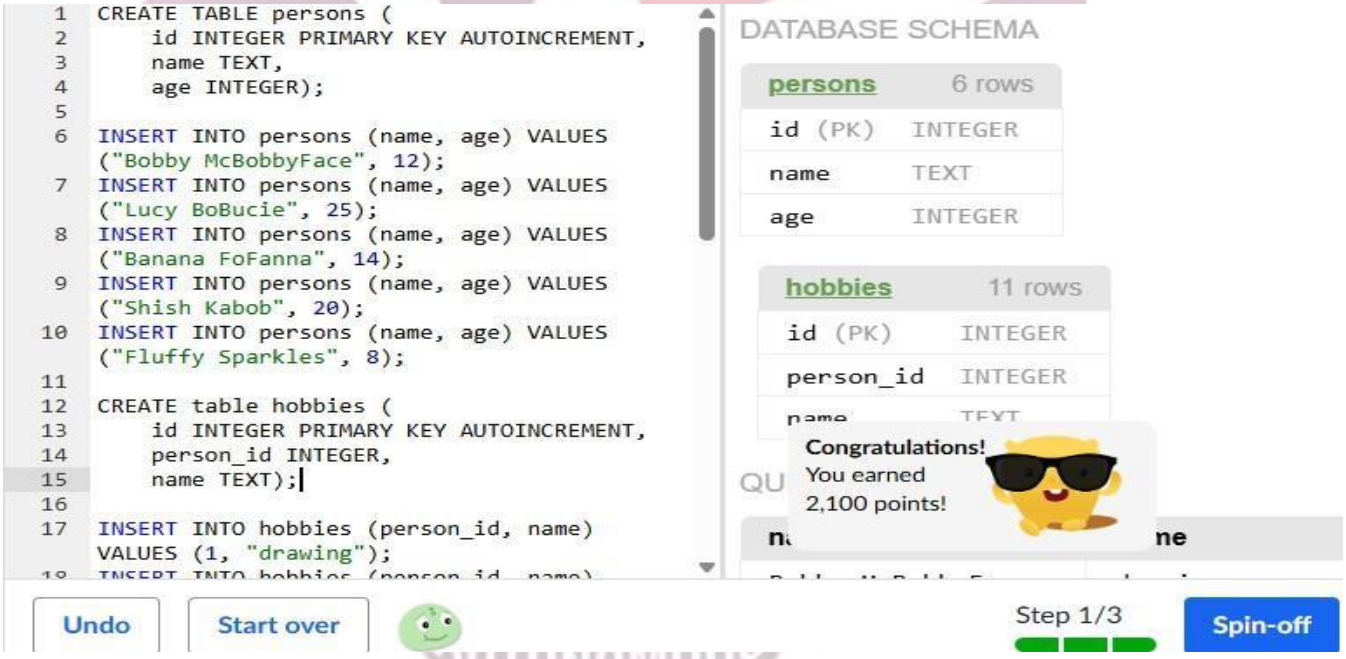
INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);

INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);

INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);

INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);

INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);



Step 3

Now, add an additional query that shows only the name and hobbies of 'Bobby McBobbyFace', using JOINcombined with WHERE.

SOLUTION

```
CREATE table hobbies (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   person_id INTEGER,
   name TEXT);


INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Sam Star", 18);
```

INSERT INTO hobbies (person_id, name)

VALUES (6, "gaming");

SELECT persons.name, hobbies.name

FROM persons

JOIN hobbies

ON persons.id = hobbies.person_id;

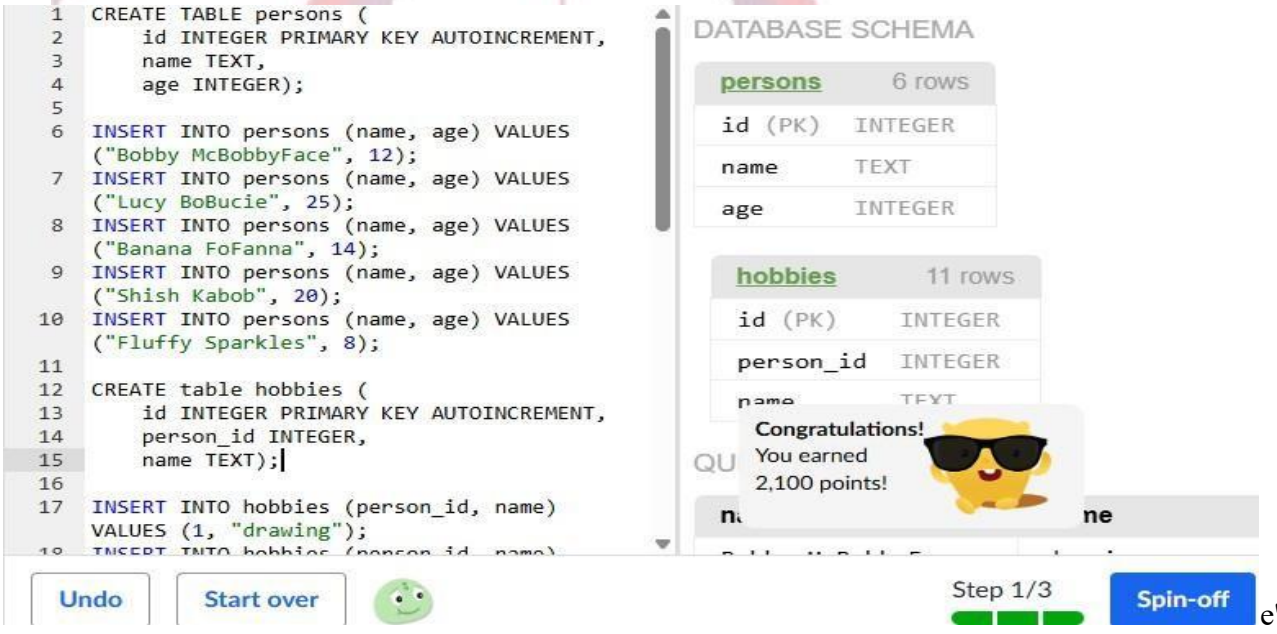
SELECT persons.name, hobbies.name

FROM persons

JOIN hobbies

ON persons.id = hobbies.person_id

WHERE persons.name = "Bobby McBobbyFac



e"

Challenge: Customer's orders

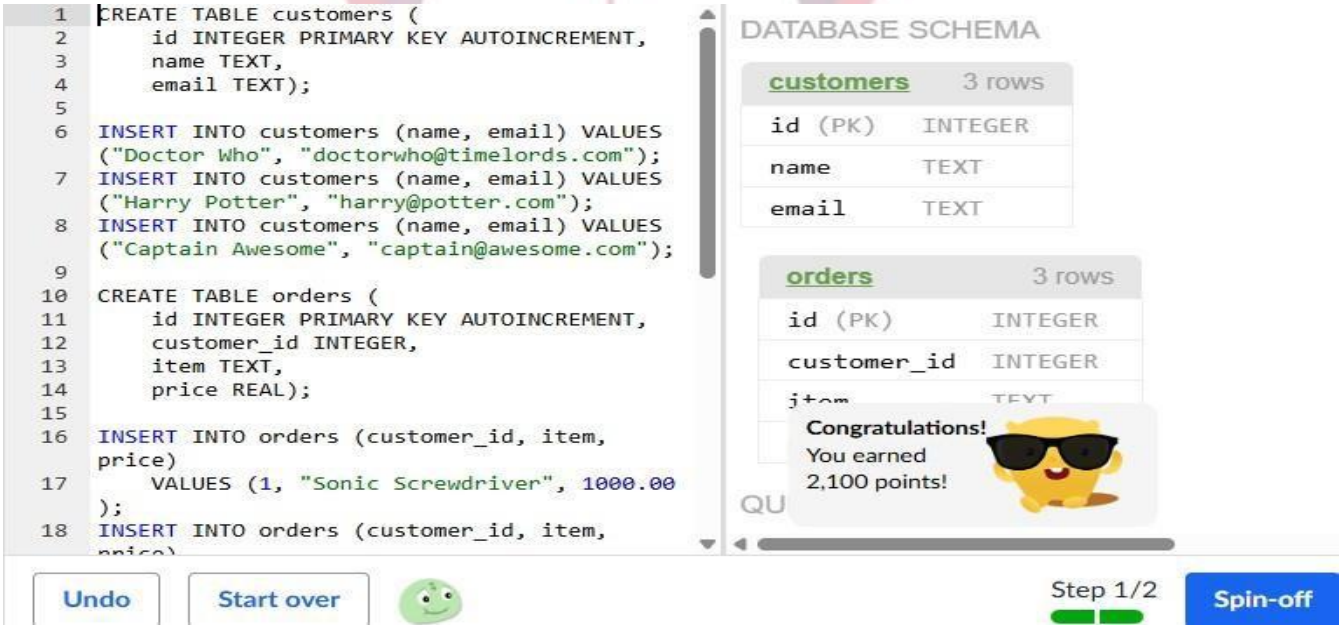
Step 1

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY.

Solution:

```
VALUES (1, "TARDIS", 1000000.00);

SELECT customers.name,
 customers.email,
 orders.item,
orders.price
FROM customers
LEFT OUTER JOIN orders
 ON customers.id = orders.customer_id;
```

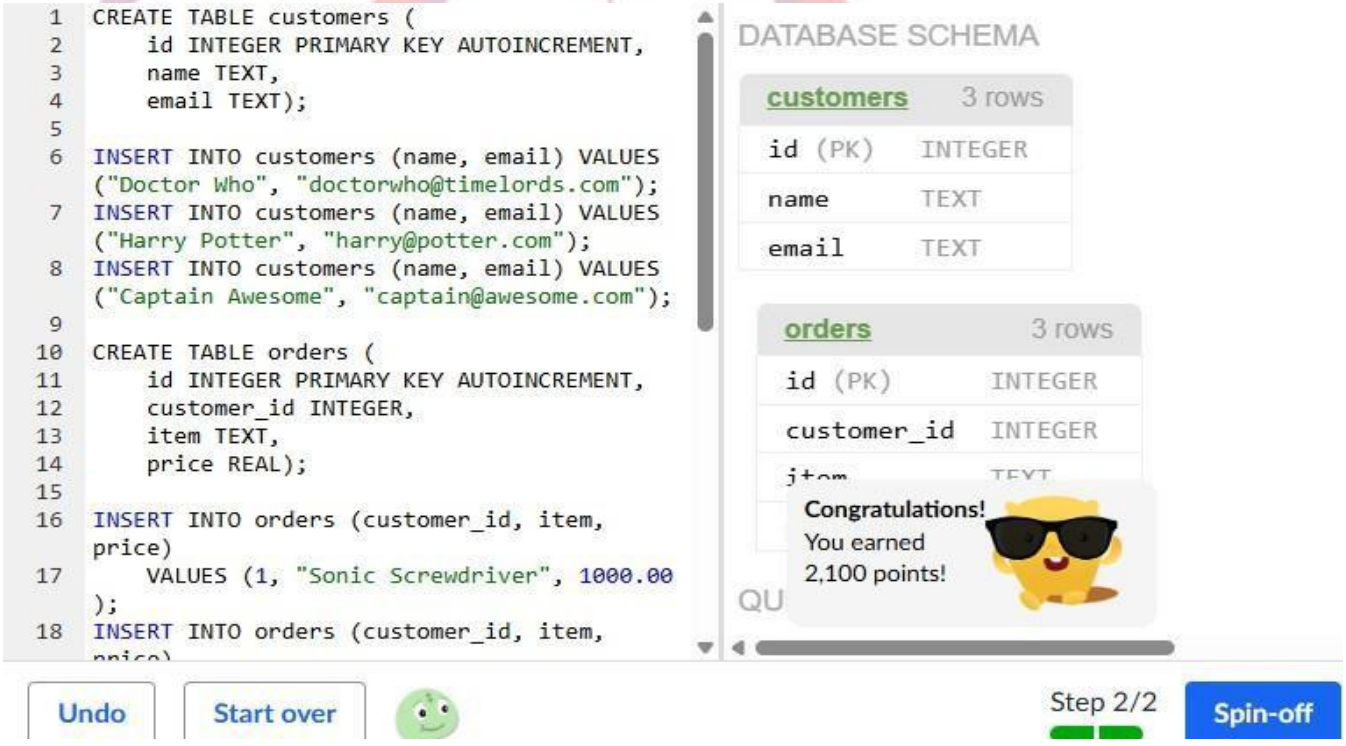


Step 2

Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent. (Tip: You should always GROUP BY on the column that is most likely to be unique in a row.)

Solution:

```
SELECT customers.name,
    customers.email, SUM(orders.price)
    AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id GROUP
BY customers.id
ORDER BY total_spent DESC;
```



Challenge: Sequels in SQL

Step 1

We've created a table with all the 'Harry Potter' movies, with a sequel_idcolumn that matches the idof the sequel for each movie. Issue a SELECTthat will show the title of each movie next to its sequel's title (or NULLif it doesn't have a sequel).

SOLUTION

```
INSERT INTO movies
  VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2); INSERT
INTO movies
  VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3); INSERT
INTO movies
  VALUES (3, "Harry Potterandthe Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
  VALUES (4, "Harry Potterandthe Goblet of Fire", 2005, 5);
INSERT INTO movies
  VALUES (5, "Harry Potterandthe Order of the Phoenix ", 2007, 6); INSERT
INTO movies
  VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7); INSERT
INTO movies
  VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
  VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
```

```
1  CREATE TABLE movies (id IHTEGER PRIMARY KEY &
   AUTOIIKREMENT,
2      title TEXT,
3      released INTEGER,
4      sequel_id INTEGER};
5
6  INSERT INTO movies
7      VALUES (1, "Harry Potter and the
   Philosopher's Stone", 2001, 2);
8  INSERT INTO movies
9      VAlt'JES (2, "Harry Potter and the
   Chamber of Secrets"', 2002, 3J;
1a INSERT INTO movies
11     VALUES (3, "Harry Potter and the
   Prisoner of Azkaban", 2004, 4);
12 INSERT INTO movies
13     VALUES (4, "Harry Potter and the Goblet
   of Fire", 2005, 5);
14 INSERT INTO movies
15     VALUES (5, '"Harry Potter and the Order
   of the Phoenix"', 2007, 6);

16 INSERT INTO movies
17     VALUES (6, "Harry Potter and the Half
```

Undo | Start over |

DATABASE SCHEMA

movies 8 rows

id (PK)	INTEGER
title	TEXT
released	INTEGER
sequel_id	INTEGER

QUERY RESULTS

title	sequel_title
H;,·-·-·· n_ _._.___ ---'	"-□ry Potter
ti Congratulations!	the Chamber
S- 'Ybu earned 2,100 pofnts!	Secrets
the Chamber of	ry Potter
H;	□,,J the

1/1



Challenge: FriendBook

Step 1

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOINto display a table showing people's names with their hobbies.

SOLUTION

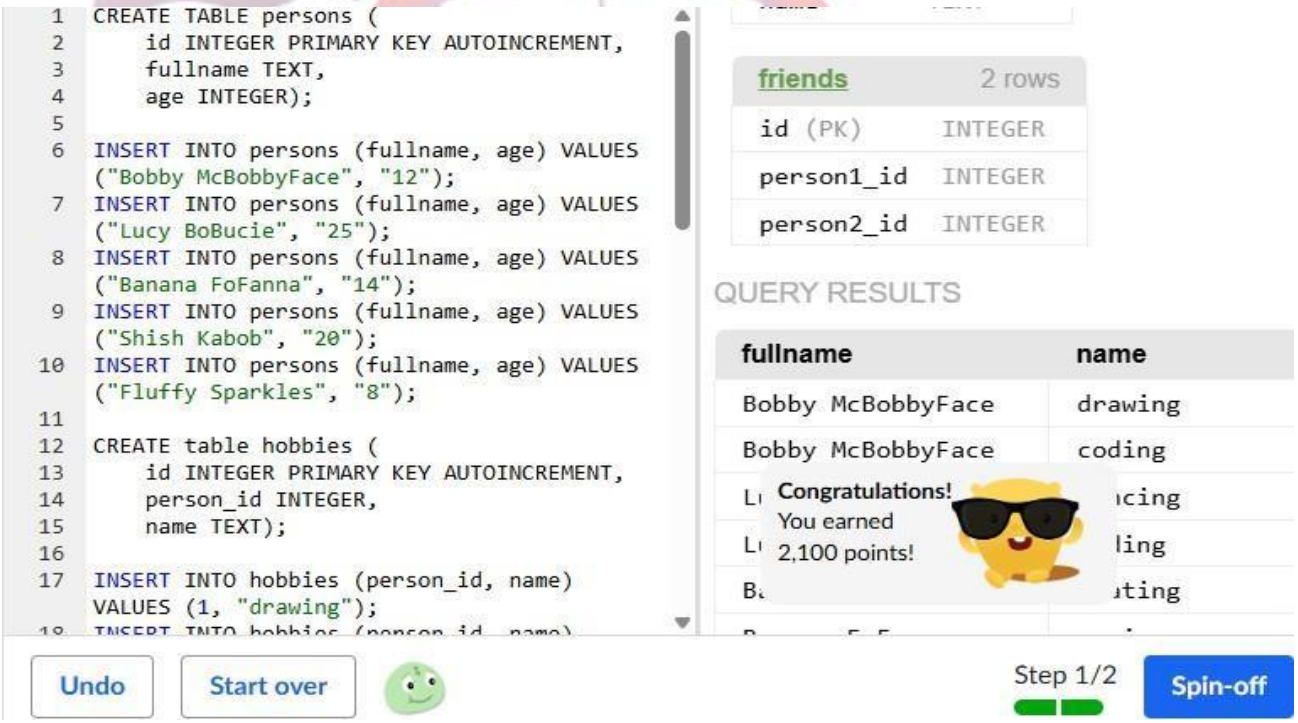
SELECT persons.fullname,

hobbies.name

FROM persons

JOIN hobbies

ON persons.id = hobbies.person_id;

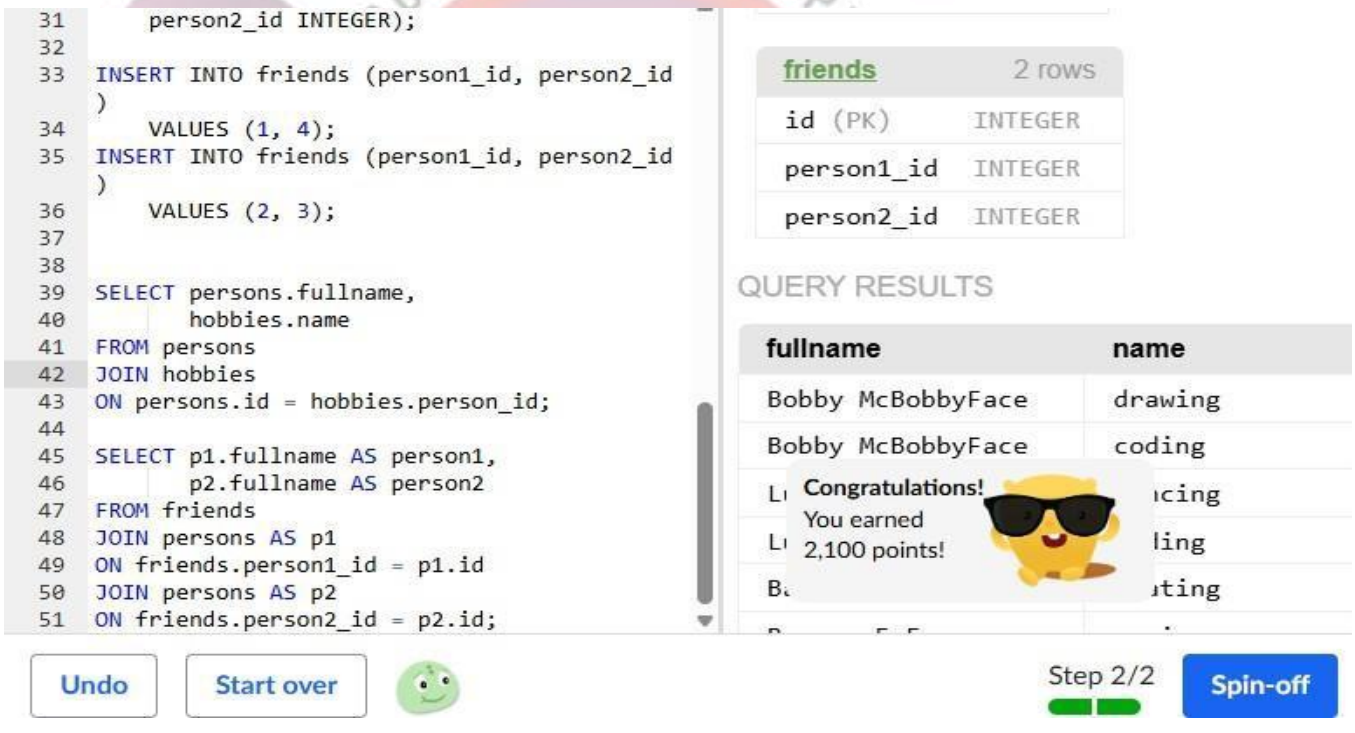


Step 2

Now, use another SELECTwith a JOINto show the names of each pair of friends, based on the data in the friends table.

SOLUTION

```
SELECT p1.fullname AS person1,
    p2.fullname AS person2
FROM friends
JOIN persons AS p1
ON friends.person1_id = p1.id
    JOIN persons AS p2
ON friends.person2_id = p2.id;
```



Project: Famous people

QUESTION

In this project, you're going to make your own table with some small set of "famous people", then make more tables about things they do and join those to create nice human readable lists.

For example, here are types of famous people and the questions your data could answer:

Movie stars: What movies are they in? Are they married to each other?
Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?

SOLUTION

```
CREATE TABLE authors

 ( id INTEGER PRIMARY KEY,

  name TEXT,

country TEXT );

INSERT INTO authors VALUES

(1, "J.K. Rowling", "United Kingdom");

(2, "George Orwell", "United

Kingdom"); (3, "Mark Twain", "United

States");
```

```
(4, "Jane Austen", "United
Kingdom"); CREATE TABLE
books (
 id INTEGER PRIMARY KEY,
 title TEXT,
 author_id INTEGER );
INSERT INTO books VALUES
(1, "Harry Potter and the Sorcerer's Stone", 1);
 (2, "Harry Potter and the Chamber of Secrets", 1);
(3, "1984", 2);
(4, "Animal Farm", 2);
 (5, "The Adventures of Tom Sawyer",
3); (6, "Pride and Prejudice", 4);
CREATE TABLE
 author_friends ( id
 INTEGER PRIMARY KEY,
 author1_id INTEGER,
author2_id INTEGER );
INSERT INTO author_friends VALUES (1, 1, 2);
 INSERT INTO author_friends VALUES (2, 3, 4);
 FROM authors
JOIN books
 ON authors.id = books.author_id;
```

```sql
/* Create table about the people and 111hat they do here */
CREATE TABLE authors (
    id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT
);

INSERT INTO authors VALUES (1, "J.K., Rowling", "United Kingdom");
INSERT INTO authors VALUES (2,, "George Orwell", "United Kingdom");
INSERT INTO authors VALLJES (3,, "Mark T1t1ain"

, "United States");
INSERT INTO authors VALUES (4,, '"Jane Austen", "United Kingdom");


CREATE TABLE books (
    id INTEGER PRIMARY KEY,

    title TEXT,
    author id INTEGER
);
```

DATABASE SCHEMA

authors 4 rows

id (PK) INTEGER

name TEXT

cou1ntry TEXT

books U rows

id (PK) INTEGER

title TEXT

author id INTEGER

author **friends** 2 rows

id (PK) INTEGER

[Start over] J Request help L

ihi:i+H

TRUTH-TOIL-TRIUMPH AUTONOMOUS

Challenge: Dynamic Documents

Step 1

We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATEto change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected.

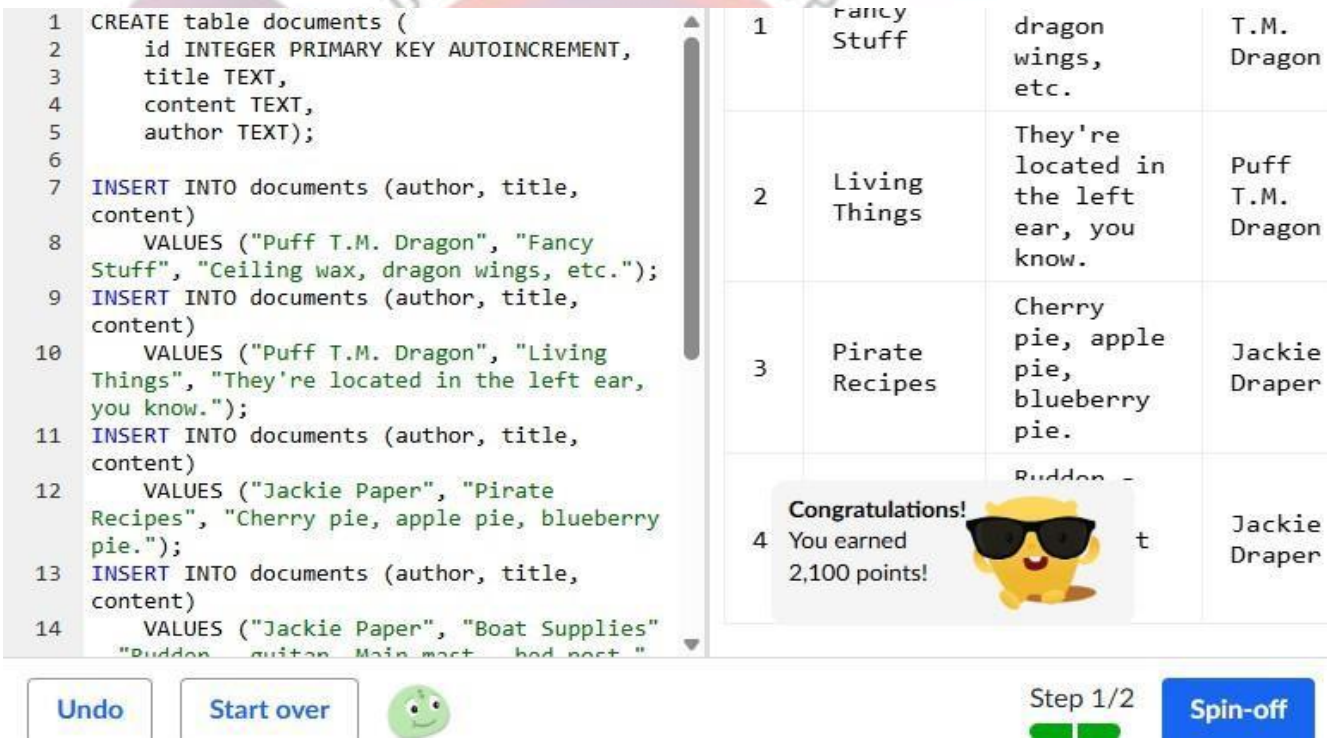
SOLUTION

SELECT * FROM documents;

UPDATE documents

SET author = 'Jackie Draper'

WHERE author = 'Jackie Paper';



Step 2

Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

SOLUTION

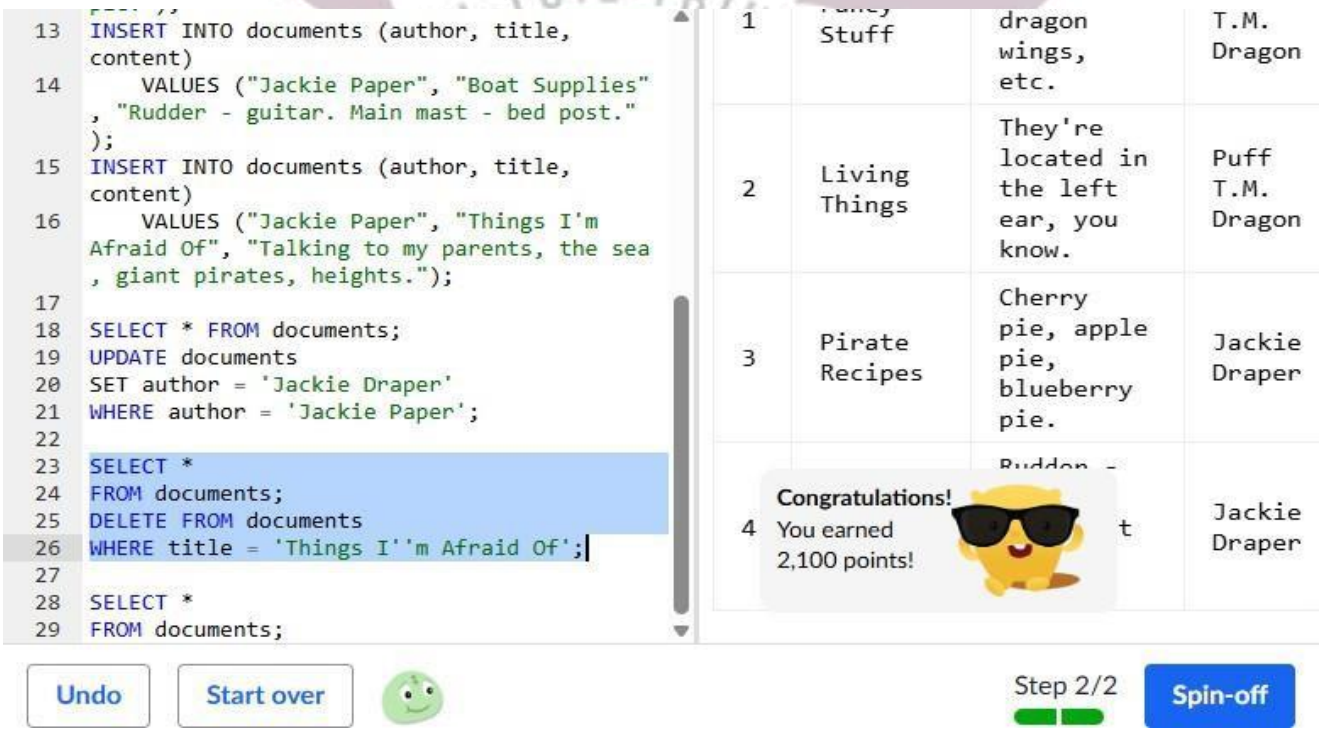
SELECT *

FROM documents;

DELETE FROM

documents

WHERE title = 'Things I''m Afraid Of';



Challenge: Clothing alterations

Step 1

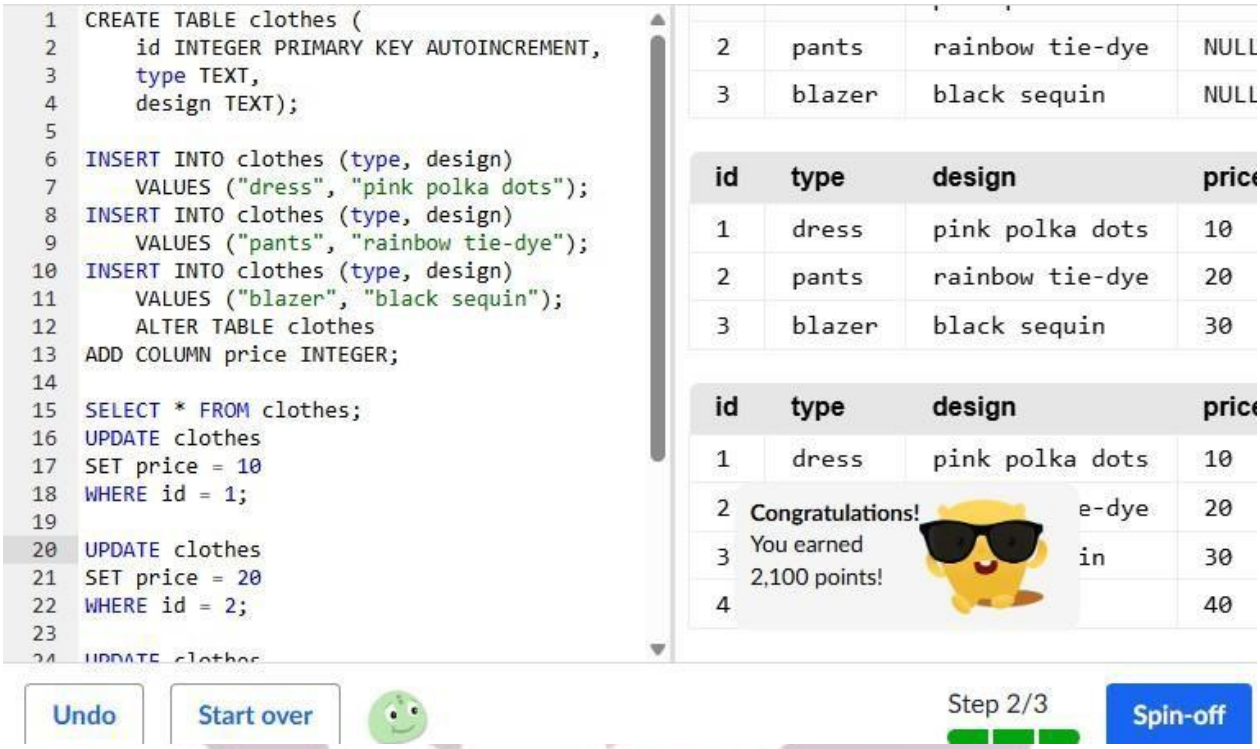
We've created a database of clothes, and decided we need a price column. Use ALTERto add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now.

SOLUTION

```
CREATE TABLE clothes (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT,
   design TEXT);


INSERT INTO clothes (type, design)
    VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
   VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
   VALUES ("blazer", "black sequin");
   ALTER TABLE clothes
ADD COLUMN price INTEGER;
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS



Step 2

Now assign each item a price, using UPDATE- item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECTof all the rows to check that it worked as expected.

SOLUTION

SELECT * FROM clothes;

UPDATE

clothes SET

price = 10

WHERE id = 1;

UPDATE clothes

SET price = 20

WHERE id = 2;

UPDATE clothes

SET price = 30

WHERE id = 3;

```
11      VALUES ("blazer", "black sequin");
12      ALTER TABLE clothes
13  ADD COLUMN price INTEGER;
14
15  SELECT * FROM clothes;
16  UPDATE clothes
17  SET price = 10
18  WHERE id = 1;
19
20  UPDATE clothes
21  SET price = 20
22  WHERE id = 2;
23
24  UPDATE clothes
25  SET price = 30
26  WHERE id = 3;
27
28  SELECT * FROM clothes;
29  INSERT INTO clothes (id, type, price)
30  VALUES (4, 'Jacket', 40);
31
32  SELECT * FROM clothes;
33
```

2	pants	rainbow tie-dye	NULL
3	blazer	black sequin	NULL

id	type	design	price
1	dress	pink polka dots	10
2	pants	rainbow tie-dye	20
3	blazer	black sequin	30

id	type	design	price
1	dress	pink polka dots	10
2		e-dye	20
3		in	30
4			40

Congratulations!
You earned
2,100 points!



Undo | Start over | Step 2/3 | Spin-off

AUTONOMOUS

Step 3

Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECTof all the rows to check it worked.

SOLUTION

SELECT * FROM clothes;

INSERT INTO clothes (id, type, price)

VALUES (4, 'Jacket', 40);

SELECT * FROM clothes;

```
11      VALUES ("blazer", "black sequin");
12      ALTER TABLE clothes
13  ADD COLUMN price INTEGER;
14
15  SELECT * FROM clothes;
16  UPDATE clothes
17  SET price = 10
18  WHERE id = 1;
19
20  UPDATE clothes
21  SET price = 20
22  WHERE id = 2;
23
24  UPDATE clothes
25  SET price = 30
26  WHERE id = 3;
27
28  SELECT * FROM clothes;
29  INSERT INTO clothes (id, type, price)
30  VALUES (4, 'Jacket', 40);
31
32  SELECT * FROM clothes;
33
```

2	pants	rainbow tie-dye	NULL
3	blazer	black sequin	NULL

id	type	design	price
1	dress	pink polka dots	10
2	pants	rainbow tie-dye	20
3	blazer	black sequin	30

id	type	design	price
1	dress	pink polka dots	10
2		e-dye	20
3		in	30
4			40

Congratulations!
You earned
2,100 points!



Undo | Start over | Step 3/3 | Spin-off

AUTONOMOUS

Project: App impersonator

QUESTION

Think about your favorite apps, and pick one that stores your data- like a game that stores scores, an app that lets you post updates, etc. Now in this project, you're going to imagine that the app stores your data in a SQL database (which is pretty likely!), and write SQL statements that might look like their own SQL.

CREATEa table to store the data.
INSERTa few example rows in the table.
Use an UPDATEto emulate what happens when you edit data in the app. Use a DELETEto emulate what happens when you delete data in the app.

SOLUTION

```
CREATE TABLE posts

 ( id INTEGER PRIMARY KEY,

username

 TEXT, caption

 TEXT, likes

 INTEGER

);
```

INSERT INTO posts VALUES

(1, 'Keerthika', 'My first post!', 25),

(2, 'Anu', 'Enjoying my vacation!', 40),

(3, 'Rahul', 'Learning SQL today!', 15);

SELECT * FROM posts;

UPDATE posts

SET caption = 'Learning SQL and databases today!'

WHERE id = 3;

SELECT * FROM posts;

DELETE FROM posts WHERE id = 2;

SELECT * FROM posts;

TRUTH-TOIL-TRIUMPH

```sql
-- Create table
CREATE TABLE posts (
    id INTEGER PRIMARY KEY,
    username TEXT,
    caption TEXT,
    likes INTEGER
);

-- Insert sample posts
INSERT INTO posts VALUES
(1, 'Keerthika', 'My first post!', 25),
(2, 'Anu', 'Enjoying my vacation!', 40),
(3, 'Rahul', 'Learning SQL today!', 15);

-- View all posts
SELECT * FROM posts;

-- Update a post (editing a caption)
UPDATE posts
SET caption = 'Learning SQL and databases today!'
WHERE id = 3;

-- View updated data
```

DATABASE SCHEMA

posts	2 rows
id (PK)	INTEGER
username	TEXT
caption	TEXT
likes	INTEGER

QUERY RESULTS

id	username	caption	likes
1	Keerthika	My first post!	25
2	Anu	Enjoying my vacation!	40
3	Rahul	Learning SQL today!	15

Start over | Request help | Spin-off

Revising the Select Query I

Query all columns for all American cities in the CITY table with populations larger than 100000. The CountryCode for America is USA.

The CITY table is described as follows:

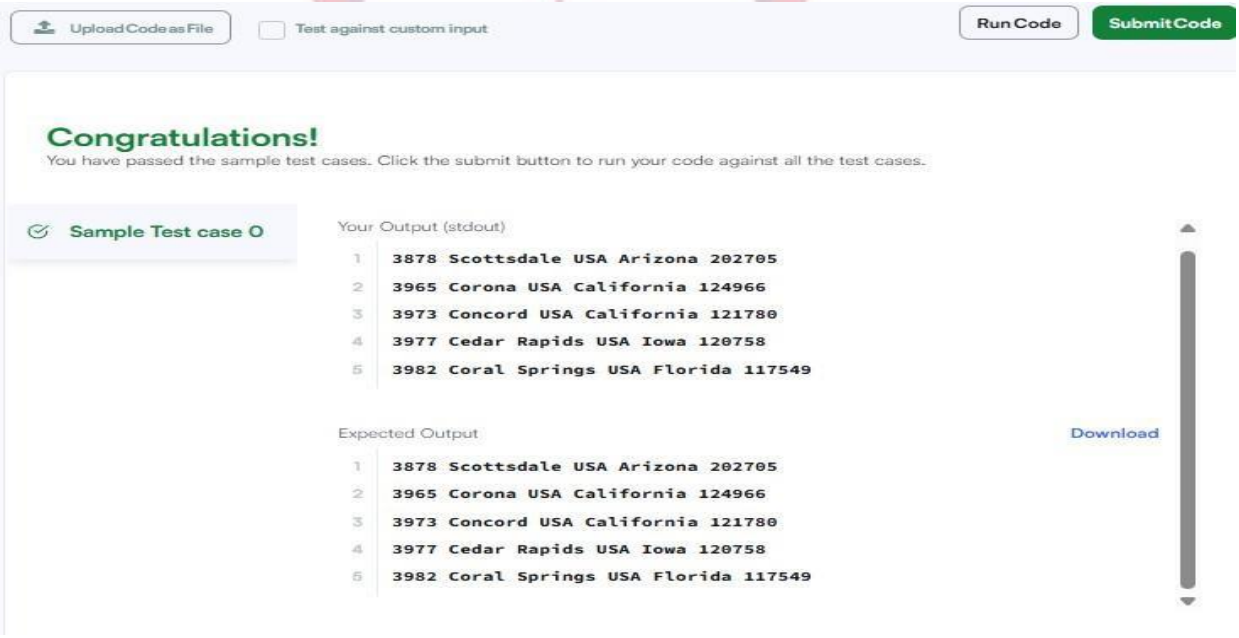
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY
WHERE COUNTRYCODE =
   'USA' AND POPULATION >
   100000;
```

OUTPUT



Revising the Select Query II

Query the NAME field for all American cities in the CITY table with populations larger than 120000. The *CountryCode* for America is USA.

The CITY table is described as follows:

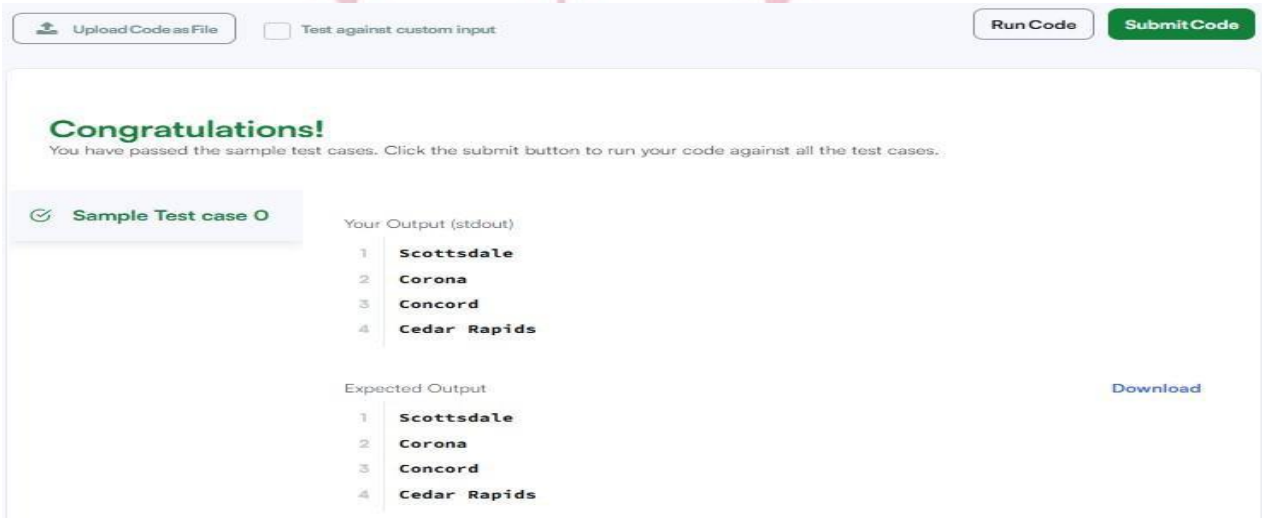
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT
NAME
FROM CITY
WHERE COUNTRYCODE =
   'USA' AND POPULATION >
   120000;
```

OUTPUT



Select All

Query all columns (attributes) for every row in the CITY table. The CITY table is described as follows:

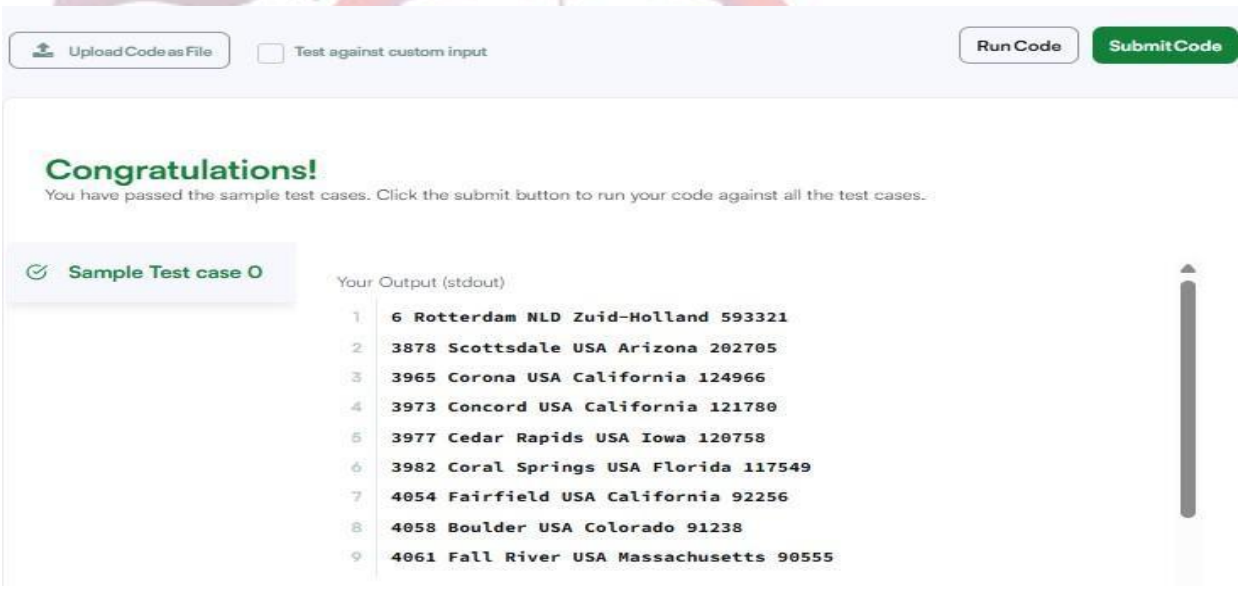
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY;
```

OUTPUT



Select By ID

Query all columns for a city in CITY with the *ID* 1661. The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY
WHERE ID = 1661;
```

OUTPUT



Japanese Cities' Attributes

Query all attributes of every Japanese city in the CITY table. The COUNTRYCODE for Japan is JPN.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT *
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



Japanese Cities' Names

Query the names of all the Japanese cities in the CITY table. The COUNTRYCODE for Japan is JPN.
The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT
NAME
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



Weather Observation Station 1

Query a list of CITY and STATE from the STATION table. The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT CITY, STATE
FROM STATION;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Kissee Mills MO
Loma Mar CA
Sandy Hook CT
Tipton IN
Arlington CO
Turner AR
Slidell LA
Negreet LA
Glencoe KY
Chelsea IA
Chignik Lagoon AK
Pelahatchie MS
Hanna City IL
```

TRUTH-TOIL-TRIUMPH

Weather Observation Station 3

Query a list of CITY names from STATION for cities that have an even ID number. Print the results in any order, but exclude duplicates from the answer.
The STATION table is described as follows:

STATION

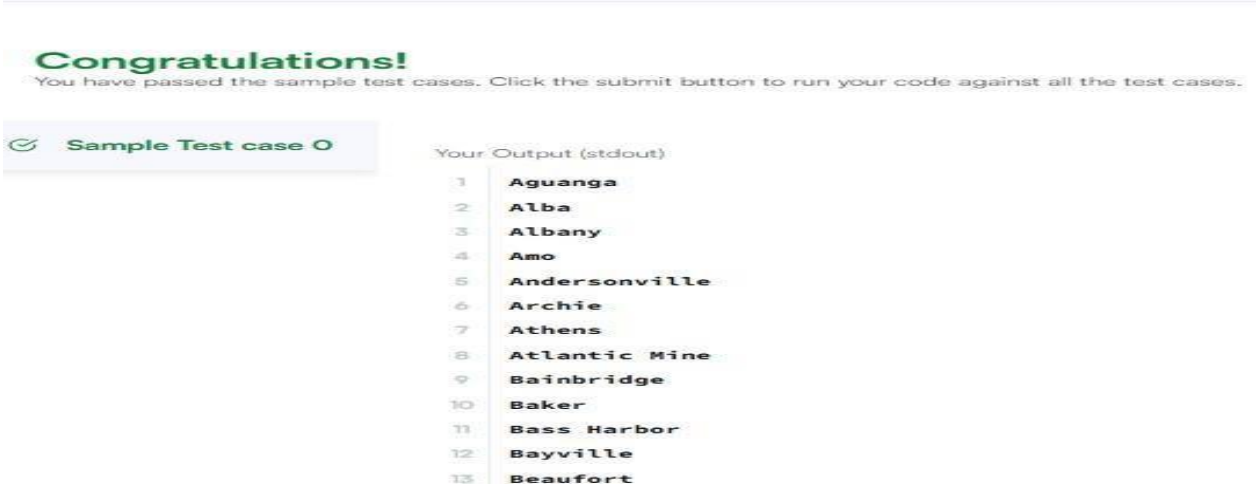
Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION

WHERE MOD(ID, 2) = 0;
```

OUTPUT



Weather Observation Station 4

Find the difference between the total number of CITY entries in the table and the number of distinct CITY entries in the table.
The STATION table is described as follows:

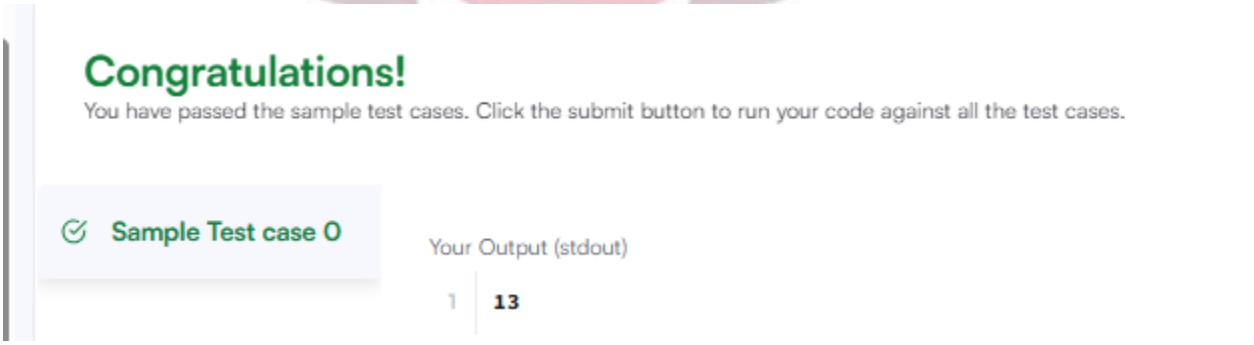
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT COUNT(CITY) - COUNT(DISTINCT
CITY) FROM STATION;
```

OUTPUT



Weather Observation Station 5

Query the two cities in STATION with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
-- Query for the shortest city name SELECT
CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) ASC, CITY
ASC LIMIT 1;
-- Query for the longest city name SELECT
CITY, LENGTH(CITY)
FROM STATION
ORDER BY LENGTH(CITY) DESC, CITY ASC
LIMIT 1;
```

OUTPUT



Weather Observation Station 6

Query the list of *CITY* names starting with vowels (i.e., a, e, i, o, or u) from STATION. Your result *cannot* contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT CITY
FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  Acme
2  Addison
3  Agency
4  Aguanga
5  Alanson
6  Alba
7  Albany
8  Albion
9  Algonac
10 Aliso Viejo
11 Allerton
12 Alpine
13 Alton
```

Weather Observation Station 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from STATION. Your result *cannot* contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT
CITY FROM STATION
WHERE LOWER(SUBSTR(CITY, LENGTH(CITY), 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Acme
Aguanga
Alba
Aliso Viejo
Alpine
Amazonia
Amo
Andersonville
Archie
Arispe
Arkadelphia
Atlantic Mine
Baileyville
```

Weather Observation Station 8

Query the list of *CITY* names from STATION which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

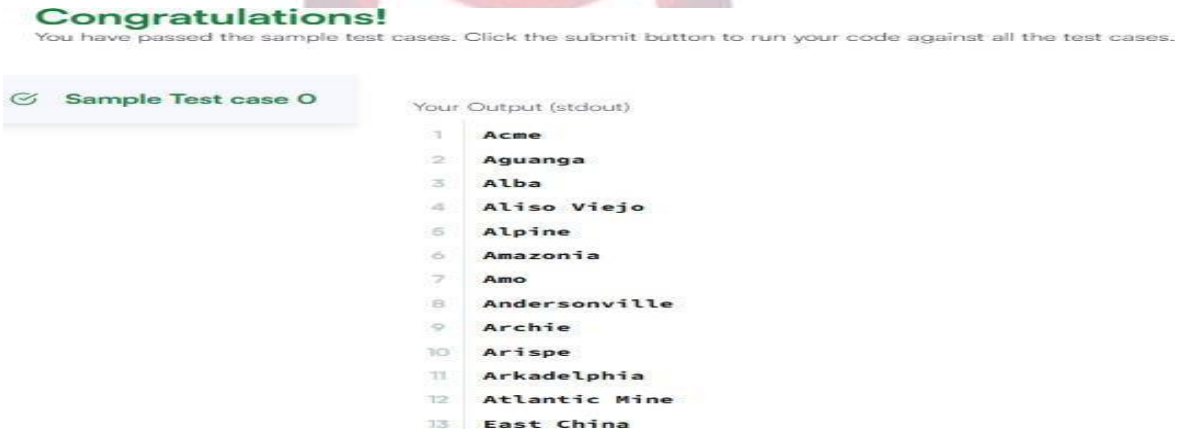
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT
CITY FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) IN ('a', 'e', 'i', 'o', 'u')
 AND LOWER(SUBSTR(CITY, LENGTH(CITY), 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



Weather Observation Station 9

Query the list of *CITY* names from STATION that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT
CITY FROM STATION
WHERE LOWER(SUBSTR(CITY, 1, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Baileyville
Bainbridge
Baker
Baldwin
Barrigada
Bass Harbor
Baton Rouge
Bayville
Beaufort
Beaver Island
Bellevue
Benedict
Bennington
```

Weather Observation Station 10

Query the list of *CITY* names from STATION that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

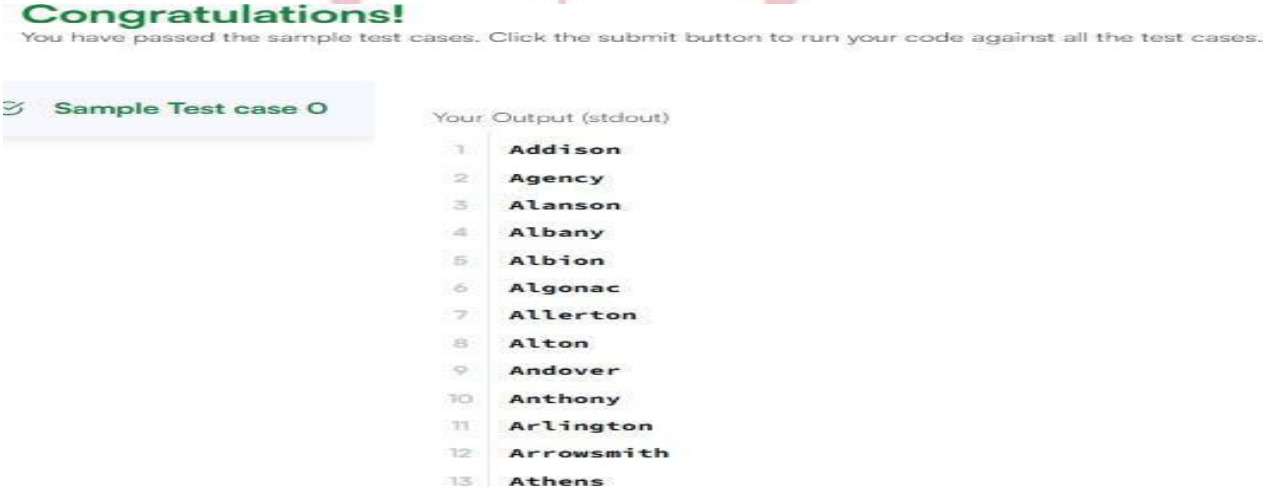
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT
CITY FROM STATION
WHERE LOWER(RIGHT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



Weather Observation Station 11

Query the list of *CITY* names from STATION that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT
CITY FROM STATION
WHERE LOWER(LEFT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u')
  OR LOWER(RIGHT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Addison
Agency
Alanson
Albany
Albion
Algonac
Allerton
Alton
Andover
Anthony
Arlington
Arrowsmith
Athens
```

Weather Observation Station 12

Query the list of *CITY* names from STATION that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

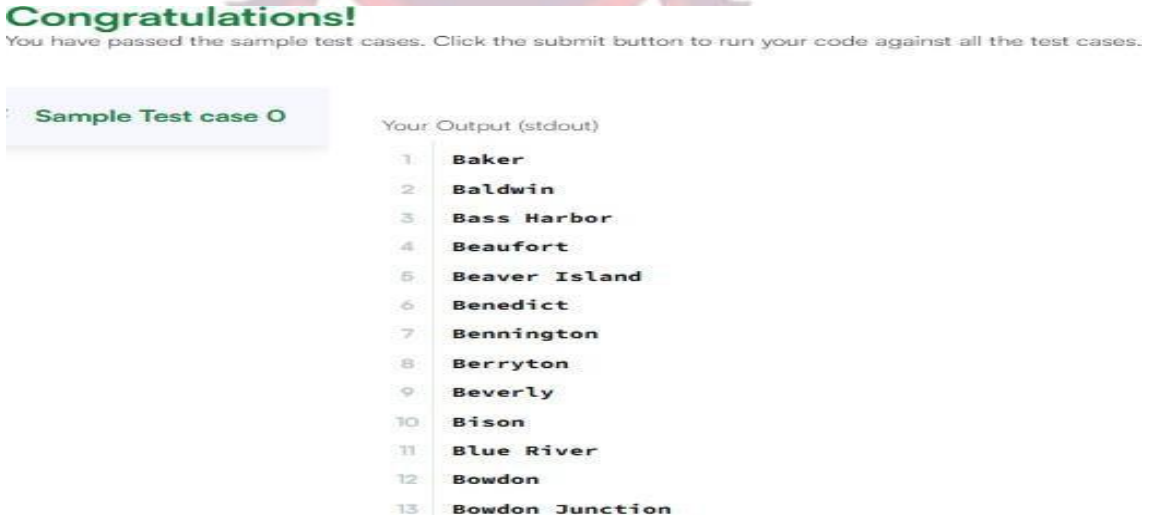
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT DISTINCT
CITY FROM STATION
WHERE LOWER(LEFT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u')
  AND LOWER(RIGHT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



Higher Than 75 Marks

Query the *Name* of any student in **STUDENTS** who scored higher than *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

The **STUDENTS** table is described as

Column	Type
ID	Integer
Name	String
Marks	Integer

follows:

INPUT

SELECT

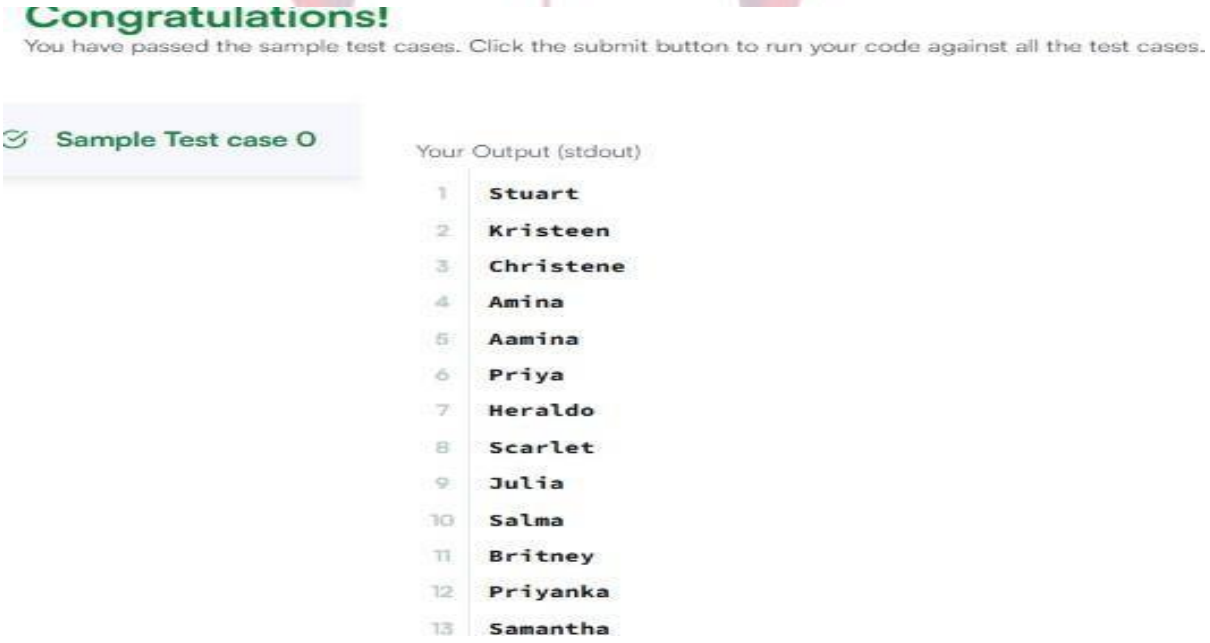
Name

FROM STUDENTS

WHERE Marks > 75

ORDER BY RIGHT(Name, 3) ASC, ID ASC;

OUTPUT



Print Prime Numbers

Write a query to print all *primenumbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).For example, theoutputfor all prime numbers wouldbe: 2&3&5&7

INPUT

```
WITH RECURSIVE Numbers AS (
  SELECT 2 AS num
  UNION ALL
  SELECT num + 1
        FROM
  Numbers WHERE
  num < 1000
)
SELECT GROUP_CONCAT(n1.num SEPARATOR '&')
FROM Numbers n1
WHERE NOT EXISTS (
  SELECT 1
  FROM Numbers n2
  WHERE n2.num < n1.num AND n1.num % n2.num = 0
);
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
2&3&5&7&11&13&17&19&23&29&31&37&41&43&47&53&59&61&67&71&73&79&83&89&97
&101&103&107&109&113&127&131&137&139&149&151&157&163&167&173&179&181&1
91&193&197&199&211&223&227&229&233&239&241&251&257&263&269&271&277&281
&283&293&307&311&313&317&331&337&347&349&353&359&367&373&379&383&389&3
97&401&409&419&421&431&433&439&443&449&457&461&463&467&479&487&491&499
&503&509&521&523&541&547&557&563&569&571&577&587&593&599&601&607&613&6
17&619&631&641&643&647&653&659&661&673&677&683&691&701&709&719&727&733
&739&743&751&757&761&769&773&787&797&809&811&821&823&827&829&839&853&8
57&859&863&877&881&883&887&907&911&919&929&937&941&947&953&967&971&977
&983&991&997
```

Weather Observation Station 17

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in **STATION** is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N >
38.7780 ORDER BY
LAT_N ASC LIMIT 1;
```

OUTPUT



Weather Observation Station 18

Consider and to be two points on a *2D* plane.

- happens to equal the minimumvalue in *Northern Latitude* (*LAT_N* in **STATION**).
- happens to equal the minimum value in *Western Longitude* (*LONG_W* in **STATION**).
- happens to equal the maximumvalue in *Northern Latitude* (*LAT_N* in **STATION**).
- happens to equal the maximum value in *Western Longitude* (*LONG_W* in **STATION**).

Query the Manhattan Distance between points and and round it to a scale of decimal places.

Input Format

The **STATION** table is described as follows:

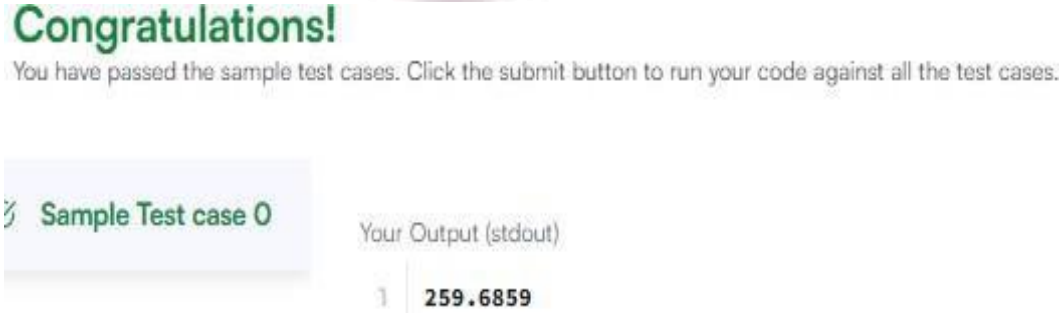
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT ROUND(
  (MAX(LAT_N) - MIN(LAT_N)) + (MAX(LONG_W) -
  MIN(LONG_W)), 4
)
FROM
STATION;
```

OUTPUT



Weather Observation Station 19

Consider and to be two points on a 2D plane where are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in **STATION**.

Query the Euclidean Distance between points and and *formatyour answer* to display decimal digits.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT
  ROUND(
  SQRT(
      POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
    POWER(MAX(LONG_W) - MIN(LONG_W), 2)
  ),
  4
)
FROM
STATION;
```

OUTPUT



Weather Observation Station 20

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from **STATION** and round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
WITH Ranked AS (
  SELECT LAT_N,
      ROW_NUMBER() OVER (ORDER BY LAT_N) AS row_num,
      COUNT(*) OVER () AS total_count
  FROM STATION
)
SELECT ROUND(AVG(LAT_N), 4)
FROM Ranked
WHERE row_num IN (FLOOR((total_count + 1) / 2.0), CEIL((total_count + 1) / 2.0));
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
83.8913
```

Placements

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

INPUT

```
SELECT S.Name
FROM Students S
JOIN Friends F ON S.ID = F.ID
JOIN Packages P1 ON S.ID = P1.ID
JOIN Packages P2 ON F.Friend_ID = P2.ID
WHERE P2.Salary > P1.Salary
ORDER BY P2.Salary ASC;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Stuart
Priyanka
Paige
Jane
Julia
Belvet
Amina
Kristeen
Scarlet
Priya
Meera
```

Symmetric Pairs

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	Type
X	Integer
Y	Integer

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetricpairs* in ascendingorder by thevalue of *X*. List the rows such that $X_1 \leq Y_1$.

Sample Input

X	Y
20	20
20	20
20	21
23	22
22	23
21	20

INPUT

```
SELECT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2 ON f1.X = f2.Y AND f1.Y = f2.X
GROUP BY f1.X, f1.Y
HAVING f1.X < f1.Y OR (f1.X = f1.Y AND COUNT(*) > 1)
ORDER BY f1.X ASC;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
2 24
4 22
5 21
6 20
8 18
9 17
11 15
13 13
```

Interviews

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

INPUT

```
WITH Agg_Submissions AS (
  SELECT
    challenge_id,
    SUM(total_submissions) AS total_submissions,
    SUM(total_accepted_submissions) AS total_accepted_submissions
  FROM Submission_Stats
  GROUP BY challenge_id
),
Agg_Views AS (
  SELECT
    challenge_id,
    SUM(total_views) AS total_views, SUM(total_unique_views)
    AS total_unique_views
```

```
    FROM View_Stats
    GROUP BY challenge_id
)
SELEC
T
    con.contest_id,
    con.hacker_id,
    con.name,
    SUM(COALESCE(s.total_submissions, 0)) AS sum_total_submissions,
    SUM(COALESCE(s.total_accepted_submissions, 0)) AS
sum_total_accepted_submissions,
    SUM(COALESCE(v.total_views, 0)) AS sum_total_views,
    SUM(COALESCE(v.total_unique_views, 0)) AS sum_total_unique_views
FROM Contests con
JOIN Colleges col ON con.contest_id = col.contest_id JOIN
Challenges cha ON col.college_id = cha.college_id
LEFT JOIN Agg_Submissions s ON cha.challenge_id = s.challenge_id
LEFT JOIN Agg_Views v ON cha.challenge_id = v.challenge_id GROUP
BY con.contest_id, con.hacker_id, con.name
HAVING (
    SUM(COALESCE(s.total_submissions, 0)) +
    SUM(COALESCE(s.total_accepted_submissions, 0)) +
    SUM(COALESCE(v.total_views, 0)) +
    SUM(COALESCE(v.total_unique_views, 0))
) > 0
ORDER BY con.contest_id ASC;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
845 579 Rose 1987 580 1635 566
858 1053 Angela 703 160 1002 384
883 1055 Frank 1121 319 1217 338
1793 2655 Patrick 1337 360 1216 412
2374 2765 Lisa 2733 815 3368 904
2963 2845 Kimberly 4306 1221 3603 1184
3584 2873 Bonnie 2492 652 3019 954
4044 3067 Michael 1323 449 1722 528
4249 3116 Todd 1452 376 1767 463
4269 3256 Joe 1018 372 1766 530
4483 3386 Earl 1911 572 1644 477
4541 3608 Robert 1886 516 1694 504
4601 3868 Amy 1900 639 1738 548
```



15 Days of Learning SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of thehacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

INPUT

```
SELECT
  s1.submission_date,

  -- 1. Count hackers who made at least one submission EVERY day up to s1.submission_date
  (
    SELECT COUNT(DISTINCT hacker_id)
    FROM Submissions s2
    WHERE s2.submission_date = s1.submission_date
     AND (
       SELECT COUNT(DISTINCT s3.submission_date)
       FROM Submissions s3
```

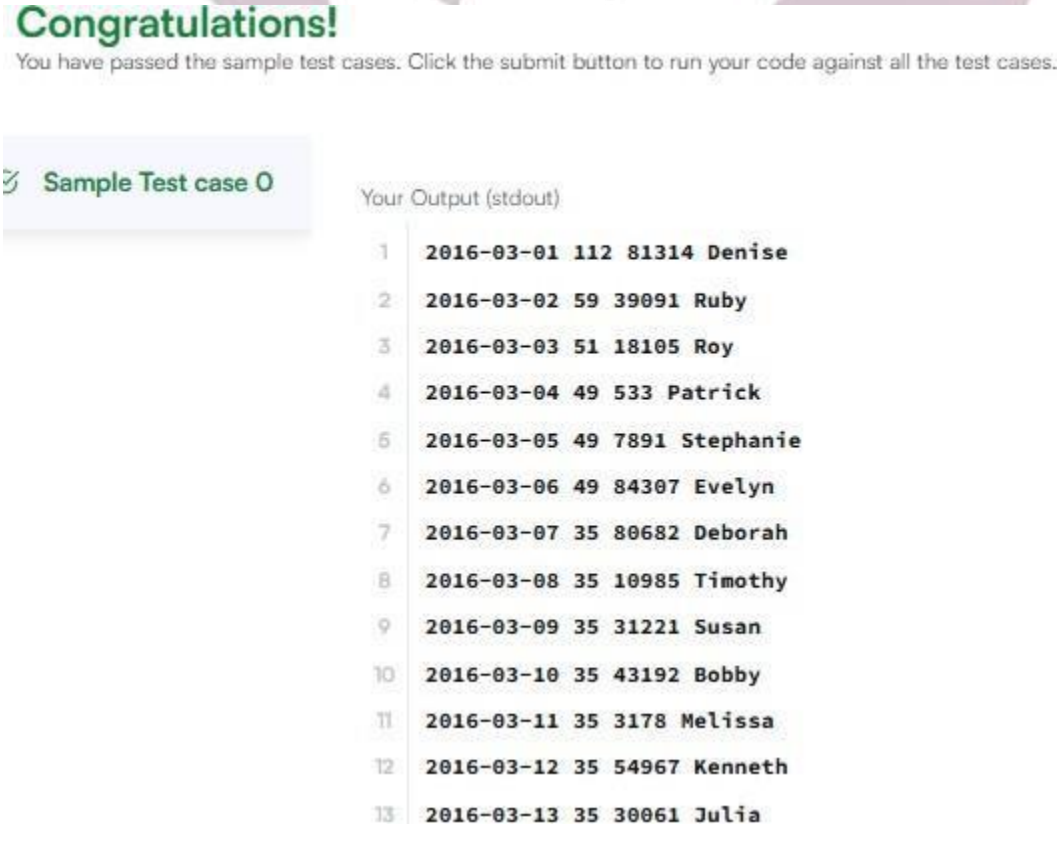
```
        WHERE s3.hacker_id = s2.hacker_id
          AND s3.submission_date <= s1.submission_date
      ) = DATEDIFF(s1.submission_date, '2016-03-01') + 1
  ) AS consistent_hackers,

  -- 2. Find top hacker ID with max submissions on s1.submission_date (tie-breaker: lowest hacker_id)
  (
    SELECT hacker_id
    FROM Submissions s3
    WHERE s3.submission_date = s1.submission_date GROUP
    BY hacker_id
    ORDER BY COUNT(submission_id) DESC, hacker_id ASC
    LIMIT 1
  ) AS top_hacker_id,
  -- 3. Get name of top hacker (
    SELECT name
    FROM Hackers
    WHERE hacker_id = (
      SELECT hacker_id
      FROM Submissions s4
      WHERE s4.submission_date = s1.submission_date
      GROUP BY hacker_id
```

```
        ORDER BY COUNT(submission_id) DESC, hacker_id ASC
        LIMIT 1
      )
  ) AS top_hacker_name

FROM (
  SELECT DISTINCT submission_date
  FROM Submissions
) s1
ORDER BY s1.submission_date ASC;
```

OUTPUT



African Cities

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

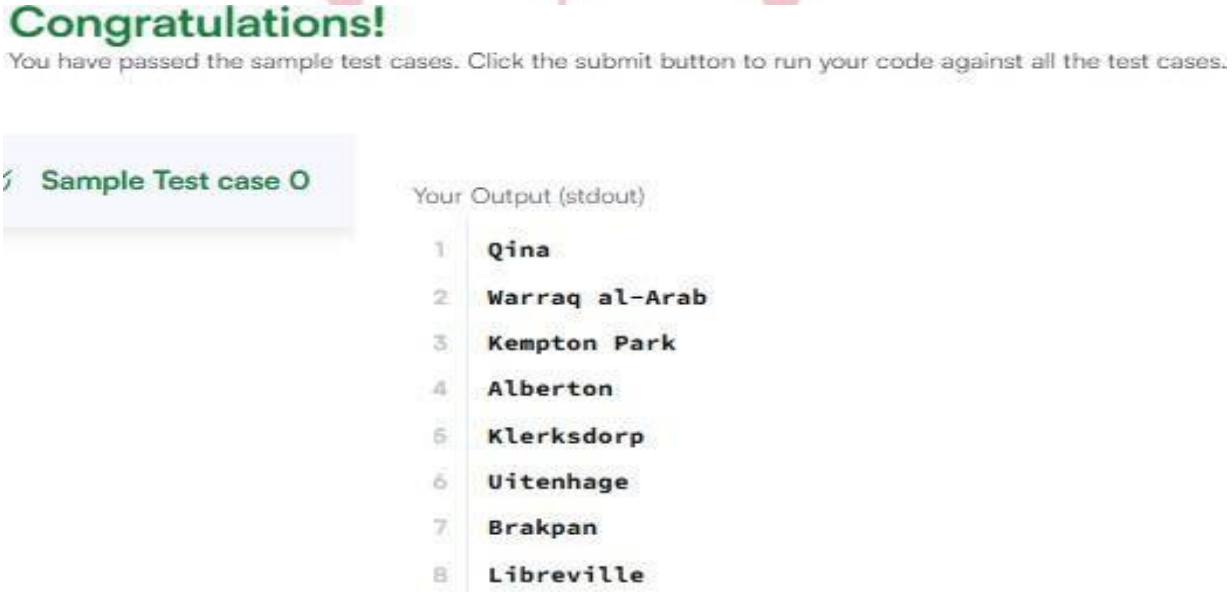
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT CITY.NAME
FROM CITY
JOIN COUNTRY ON CITY.COUNTRYCODE =
COUNTRY.CODE WHERE COUNTRY.CONTINENT =
'Africa';
```

OUTPUT



Average Population of Each Continent

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (*COUNTRY.Continent*) andtheir respectiveaverage city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

:

INPUT

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))
FROM CITY
JOIN COUNTRY ON CITY.CountryCode = COUNTRY.Code
GROUP BY
COUNTRY.Continent;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1 Asia 693038
2 Europe 175138
3 Oceania 109189
4 South America 147435
5 Africa 274439
```

The Report

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	Type
ID	Integer
Name	String
Marks	Integer

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containingthree columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a gradelower than *8*. The report must be in descendingorder by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their namealphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

Write a query to help Eve.

Sample Input

ID	Name	Marks
1	Julia	88
2	Samantha	68
3	Maria	99
4	Scarlet	78
5	Ashley	63
6	Jane	81

INPUT

```
SELECT
   CASE
      WHEN g.Grade >= 8 THEN s.Name
      ELSE NULL
   END AS Name,
   g.Grade,
   s.Marks
FROM Students s
JOIN Grades g
 ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
   g.Grade DESC,
   s.Name ASC,
   s.Marks ASC;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
Britney 10 95
Heraldo 10 94
Julia 10 96
Kristeen 10 100
Stuart 10 99
Amina 9 89
Christene 9 88
Salma 9 81
Samantha 9 87
Scarlet 9 80
Vivek 9 84
Aamina 8 77
Belvet 8 78
```



Top Competitors

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by
ascending *hacker_id*.

Input Format

The following tables contain contest data:

Hackers: The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

Difficulty: The *difficult_level* is the level of difficulty of the challenge, and *score* is the maximum score that can be achieved for a challenge at that difficulty level.

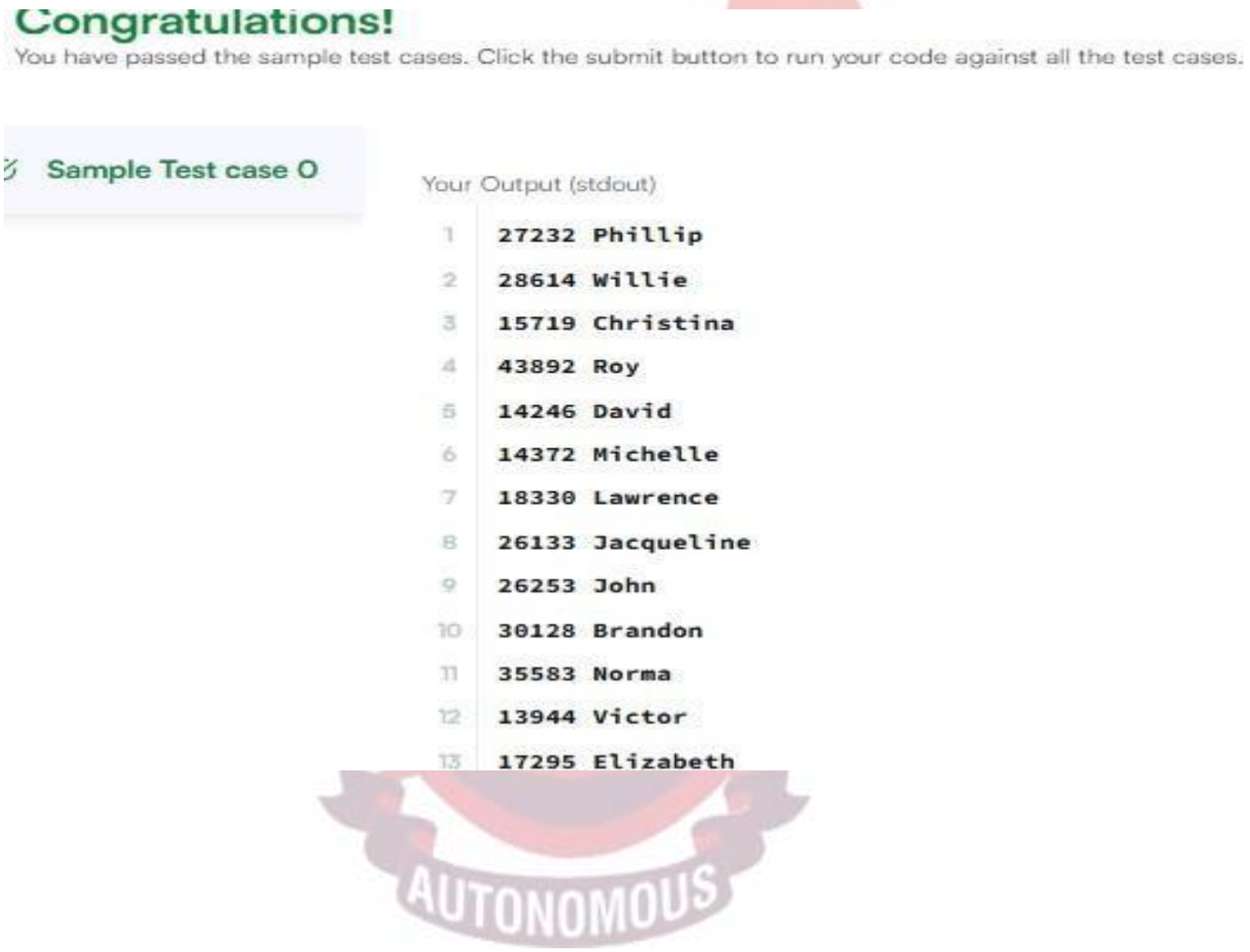
Column	Type
difficulty_level	Integer
score	Integer

INPUT

```
SELECT
  h.hacker_id,
  h.name
FROM Submissions s
JOIN Hackers h ON s.hacker_id = h.hacker_id
JOIN Challenges c ON s.challenge_id = c.challenge_id JOIN
Difficulty d ON c.difficulty_level = d.difficulty_level WHERE
s.score = d.score
```

```
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```

OUTPUT



AUTONOMOUS

Ollivander's Inventory

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of thewands that Ron'sinterested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

INPUT

```
SELECT
   w.id, wp.age,
   w.coins_needed,
   w.power
FROM Wands w
JOIN Wands_Property wp ON w.code = wp.code
WHERE wp.is_evil = 0
  AND w.coins_needed = (
     SELECT MIN(w1.coins_needed)
```

```
    FROM Wands w1
    JOIN Wands_Property wp1 ON w1.code = wp1.code WHERE
    wp1.is_evil = 0
     AND w1.power = w.power
     AND wp1.age = wp.age
)
ORDER BY w.power DESC, wp.age DESC;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1038 496 4789 10
1130 494 9439 10
1315 492 4126 10
9 491 7345 10
858 483 4352 10
1164 481 9831 10
1288 464 4952 10
861 462 8302 10
412 455 5625 10
996 451 8884 10
1608 446 8351 10
1376 443 1735 10
1330 430 5182 10
```

Type of Triangle

Write a queryidentifyingthe *type* of each record in the **TRIANGLES** table usingits three side lengths. Output one of the following statements for each record in the table:

- **Equilateral**: It's a triangle with sides of equal length.
- **Isosceles**: It's a triangle with sides of equal length.
- **Scalene**: It's a trianglewith sides of differing lengths.
- **Not A Triangle**: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

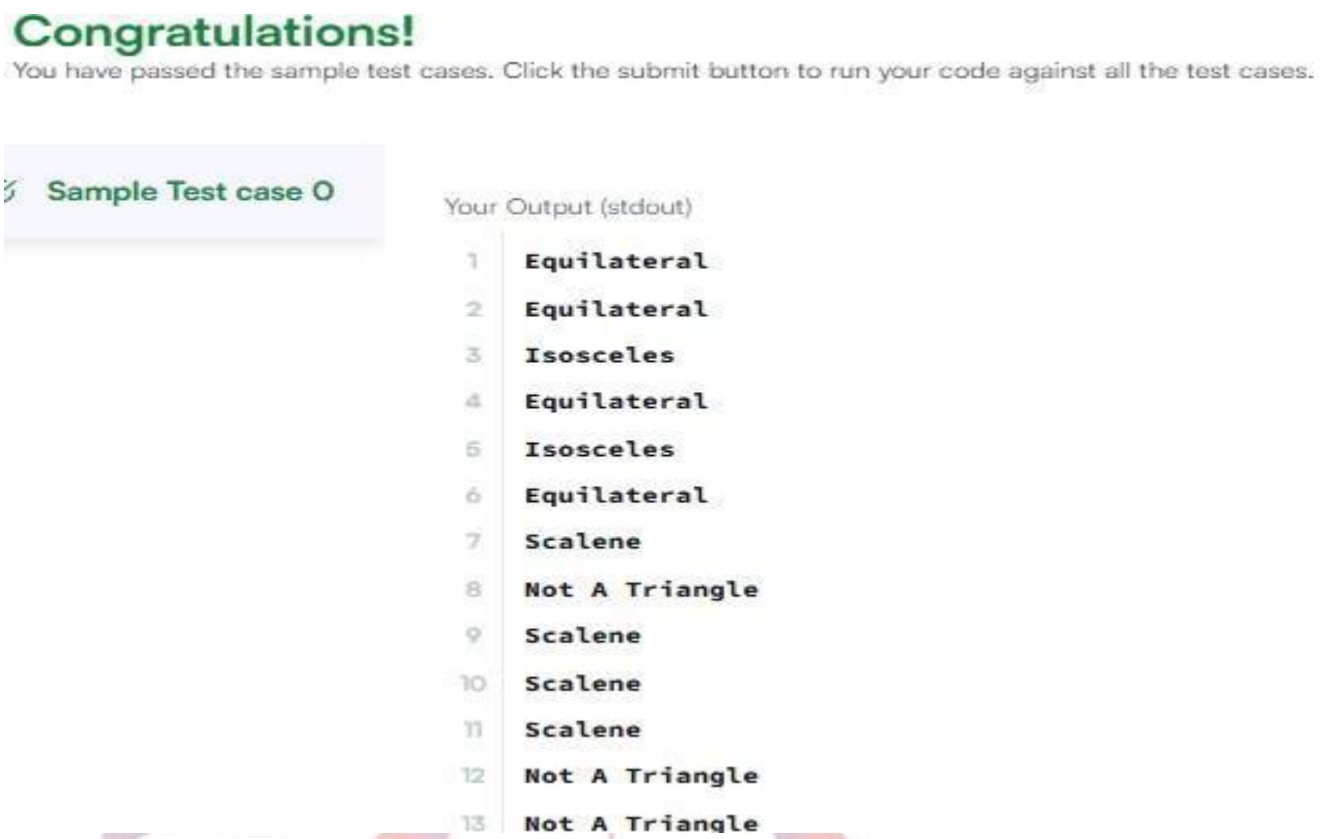
The **TRIANGLES** table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

INPUT

```
SELECT
   CASE
     WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle'
     WHEN A = B AND B = C THEN 'Equilateral'
     WHEN A = B OR B = C OR A = C THEN
     'Isosceles'
     ELSE'Scalene'
  END
FROM TRIANGLES;
```

OUTPUT



AUTONOMOUS

The PADS

Generatethefollowing two result sets:

1. Query an *alphabetically ordered* list of all names in **OCCUPATIONS**, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For
   example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).
2. Query the number of ocurrences of each occupation in **OCCUPATIONS**. Sort the occurrences in *ascending order*, and output them in the following format:
3. There are a total of [occupation_count] [occupation]s.

where [occupation_count] is the number of occurrences of an occupation in **OCCUPATIONS** and [occupation] is the *lowercase* occupation name. If more than one *Occupation* has the same [occupation_count], they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

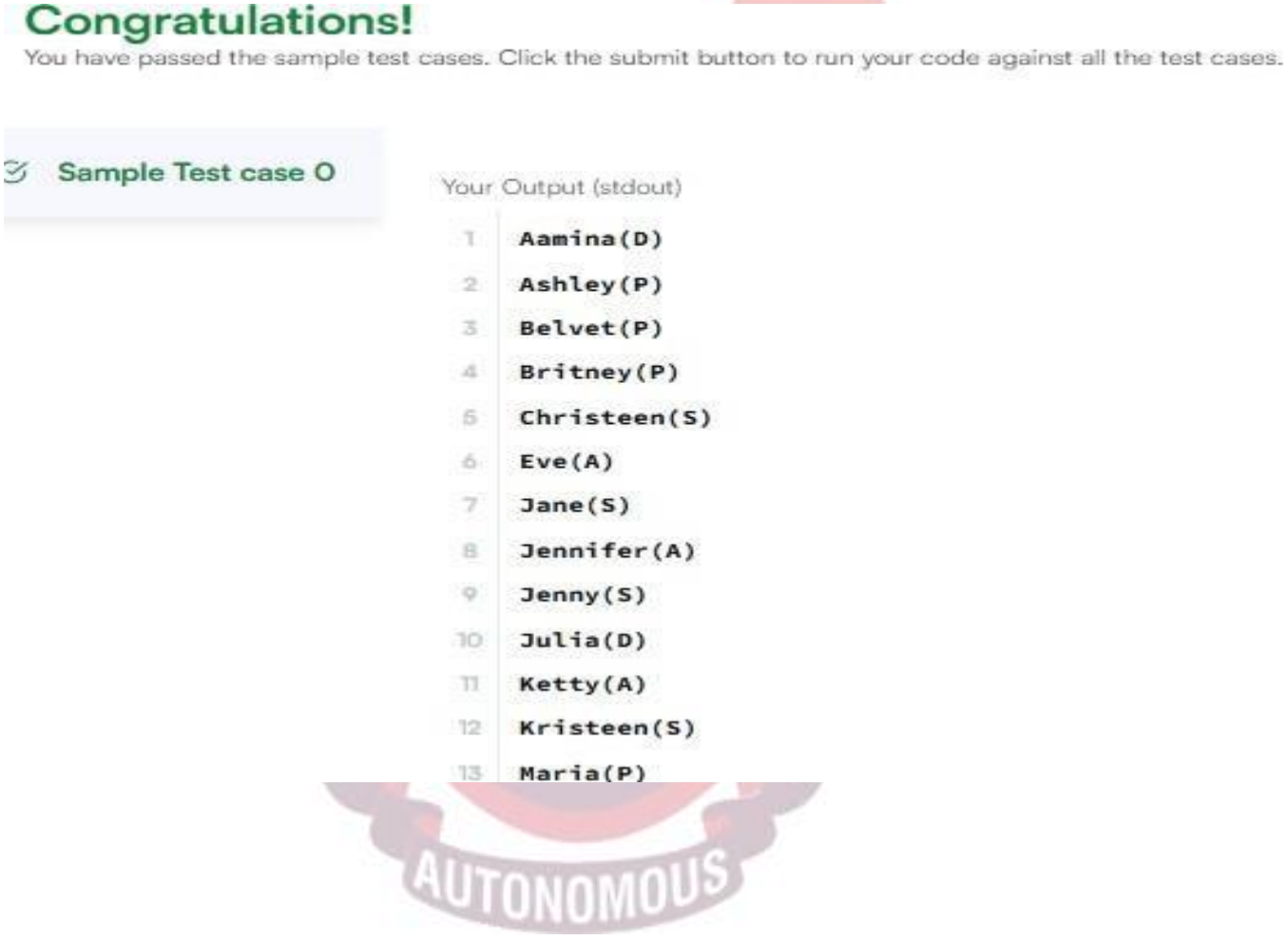
Column	Type
Name	String
Occupation	String

INPUT

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name ASC;
```

```
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ', LOWER(Occupation), 's.')
FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(Occupation) ASC, LOWER(Occupation) ASC;
```

OUTPUT



AUTONOMOUS

Occupations

Pivot the *Occupation* column in **OCCUPATIONS** so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. Theoutput should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print **NULL** when there are no more names corresponding to an occupation.

Input Format

The **OCCUPATIONS** table is described as follows:

Column	Type
Name	String
Occupation	String

INPUT

```
SELECT
  MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
  MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
  MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
  MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
  SELECT
    Name,
    Occupation,
    ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS
  row_num FROM OCCUPATIONS
) AS ranked
GROUP BY row_num
ORDER BY row_num;
```

OUTPUT

Cong;ratullati o ns !

You ha Ye µassad the samp e tesl. i.:..a:ses_ Click the subtnil buHo ,. to run your code against all the test cases.

3' Sample Test case0

Y□u Oulµut (sl(.k!Ul□

```
Aamina Ashley Christeen Eve
Julia Belvet Jane Jennifer
Pr-'li y.t;i Br"i tney :J enny Ketty
IMU I..IL Mar"i a1 l!C.ll"i st□@1111 $ amal'\ll'tlh;ai
NULL Meera NULL NULL
.MUI..IL Na•Cldl"l IMUILL NllfLL
NULIL PriyaBka NULL NULL
```



Binary Tree Nodes

You are given a table, *BST*, containing two columns: *N* and *P,* where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

Column	Type
N	Integer
P	Integer

Write a query to findthe node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

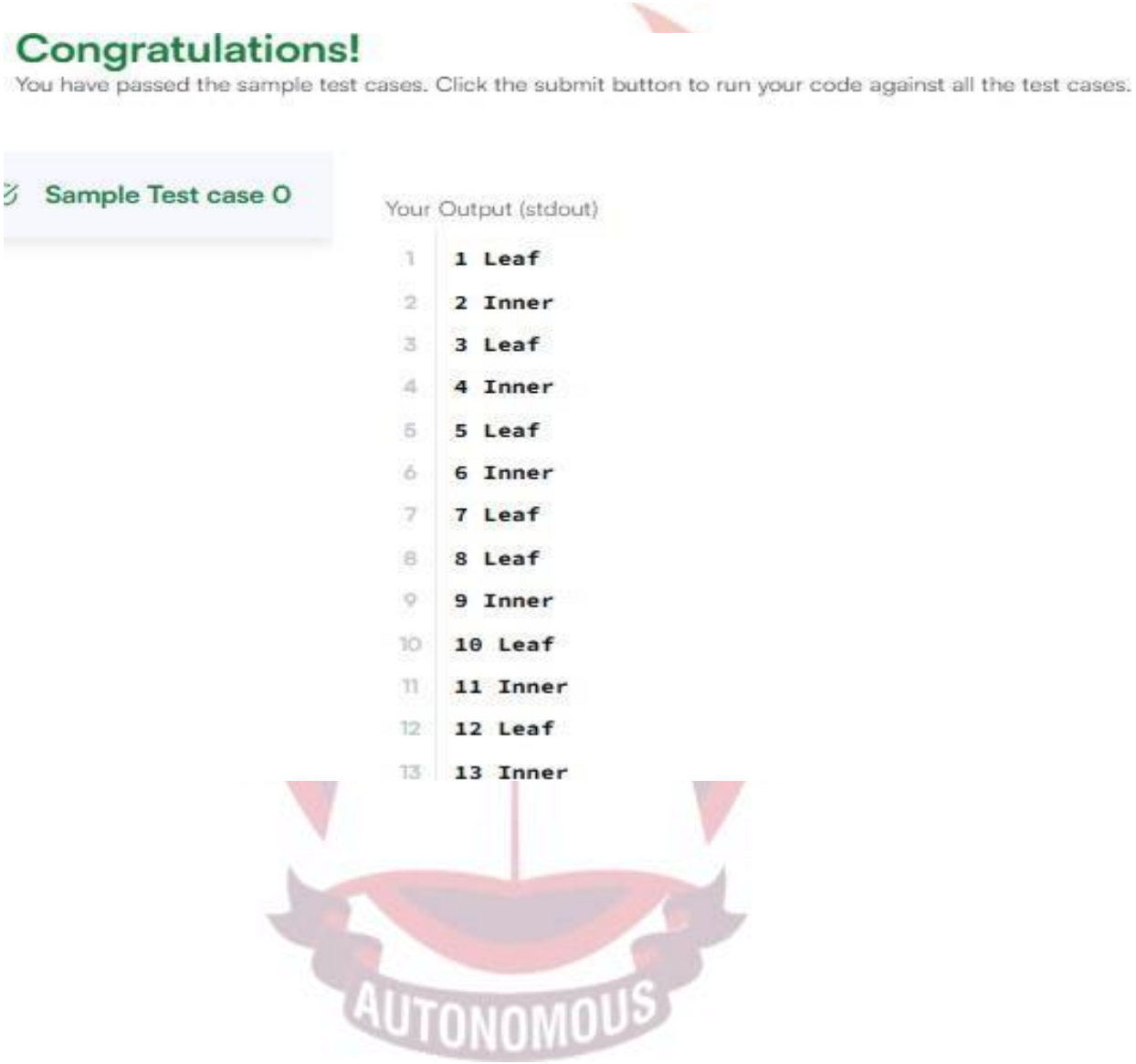
- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

INPUT

```
SELECT N,
    CASE
      WHEN P IS NULL THEN 'Root'
      WHEN N IN (SELECT DISTINCT P FROM BST WHERE P IS NOT NULL) THEN
      'Inner' ELSE 'Leaf'
    END
FROM
BST
    ORDE
R BY N
ASC;
```

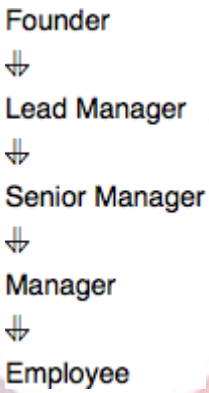
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT


Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.
Sample Test case 0
Your Output (stdout)
1 Leaf
2 Inner
3 Leaf
4 Inner
5 Leaf
6 Inner
7 Leaf
8 Leaf
9 Inner
10 Leaf
11 Inner
12 Leaf
13 Inner

New Companies

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



INPUT

```
SELECT
   c.company_code,
   c.founder,
   COUNT(DISTINCT lm.lead_manager_code),
   COUNT(DISTINCT sm.senior_manager_code),
   COUNT(DISTINCT m.manager_code),
   COUNT(DISTINCT e.employee_code)
FROM Company c
JOIN Lead_Manager lm ON c.company_code = lm.company_code JOIN
Senior_Manager sm ON c.company_code = sm.company_code JOIN
Manager m ON c.company_code = m.company_code
JOIN Employee e ON c.company_code = e.company_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code ASC;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
C1 Angela 1 2 5 13
C10 Earl 1 1 2 3
C100 Aaron 1 2 4 10
C11 Robert 1 1 1 1
C12 Amy 1 2 6 14
C13 Pamela 1 2 5 14
C14 Maria 1 1 3 5
C15 Joe 1 1 2 3
C16 Linda 1 1 3 5
C17 Melissa 1 2 3 7
C18 Carol 1 2 5 6
C19 Paula 1 2 4 7
C2 Frank 1 1 1 3
```



Weather Observation Station 16

Query the smallest *Northern Latitude* (*LAT_N*) from **STATION** that is greater than . Round your answer to decimal places.

Input Format

The **STATION** table is described as follows:

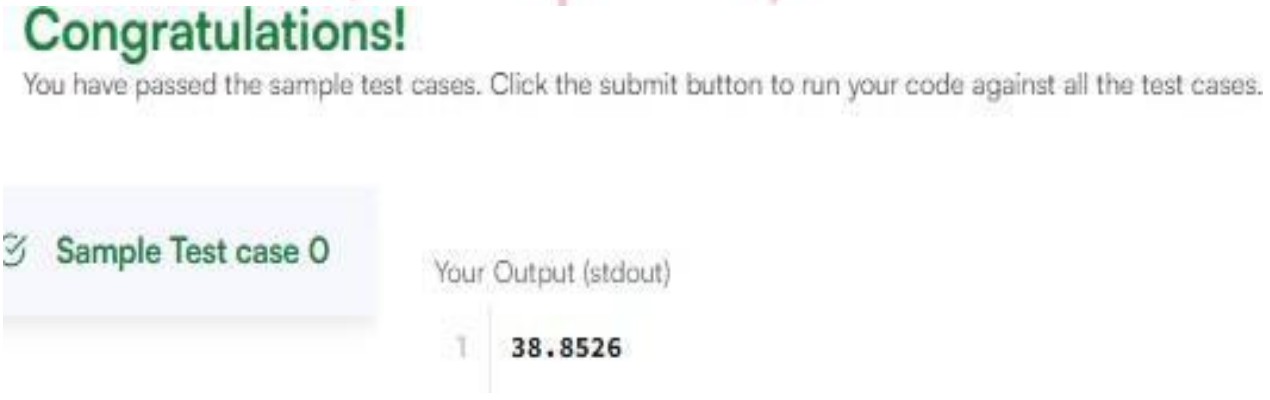
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT
ROUND(MIN(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7880;
```

OUTPUT



Weather Observation Station 5

Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT CITY,
LENGTH(CITY) FROM
STATION
ORDER BY LENGTH(CITY) ASC, CITY
ASC LIMIT 1;
SELECT CITY,
LENGTH(CITY) FROM
STATION
ORDER BY LENGTH(CITY) DESC, CITY
ASC LIMIT 1;
```

OUTPUT



Draw The Triangle 1

P(R) represents a pattern drawn by Julia in *R* rows. Thefollowing pattern represents *P(5)*:

* * * * *

* * * *

* * *

* *

*

Write a query to print the pattern *P(20)*.

INPUT

SET @number = 21;

SELECT REPEAT('* ', @number := @number - 1)

FROM information_schema.tables

LIMIT

20;

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  * * * * * * * * * * * * * * * * * * * * *
2  * * * * * * * * * * * * * * * * * * * *
3  * * * * * * * * * * * * * * * * * * *
4  * * * * * * * * * * * * * * * * * *
5  * * * * * * * * * * * * * * * * *
6  * * * * * * * * * * * * * * * *
7  * * * * * * * * * * * * * * *
8  * * * * * * * * * * * * * *
9  * * * * * * * * * * * * *
10 * * * * * * * * * * * *
11 * * * * * * * * * * *
12 * * * * * * * * * *
13 * * * * * * * * *
```

Draw The Triangle 2

P(R) represents a pattern drawn by Julia in *R* rows. Thefollowing pattern represents *P(5)*:

*

* *

* * *

* * * *

* * * * *

Write a query to print the pattern *P(20)*.

INPUT

```
SET @number = 0;

SELECT REPEAT('* ', @number := @number + 1)

FROM information_schema.tables

LIMIT

20;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
1  *
2  * *
3  * * *
4  * * * *
5  * * * * *
6  * * * * * *
7  * * * * * * *
8  * * * * * * * *
9  * * * * * * * * *
10 * * * * * * * * * *
11 * * * * * * * * * * *
12 * * * * * * * * * * * *
13 * * * * * * * * * * * * *
```

Weather Observation Station 2

Query the following two values from the **STATION** table:

1. The sum of all values in *LAT_N* rounded to a scale of decimal places.
2. The sum of all values in *LONG_W* rounded to a scale of decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT
  ROUND(SUM(LAT_N), 2),
  ROUND(SUM(LONG_W), 2)
FROM STATION;
```

OUTPUT



Weather Observation Station 14

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from **STATION** that is less than . Truncate your answer to decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

INPUT

```
SELECT TRUNCATE(MAX(LAT_N), 4)
FROM STATION
WHERE LAT_N < 137.2345;
```

OUTPUT



Weather Observation Station 15

Input Format

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
  SELECT
  MAX(LAT_N)
  FROM STATION
  WHERE LAT_N < 137.2345
);
```

INPUT

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
  SELECT
  MAX(LAT_N)
  FROM STATION
  WHERE LAT_N < 137.2345
);
```

OUTPUT



Revising Aggregations - The Count Function

Query a *count* of the number of cities in **CITY** having a *Population* larger than .

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

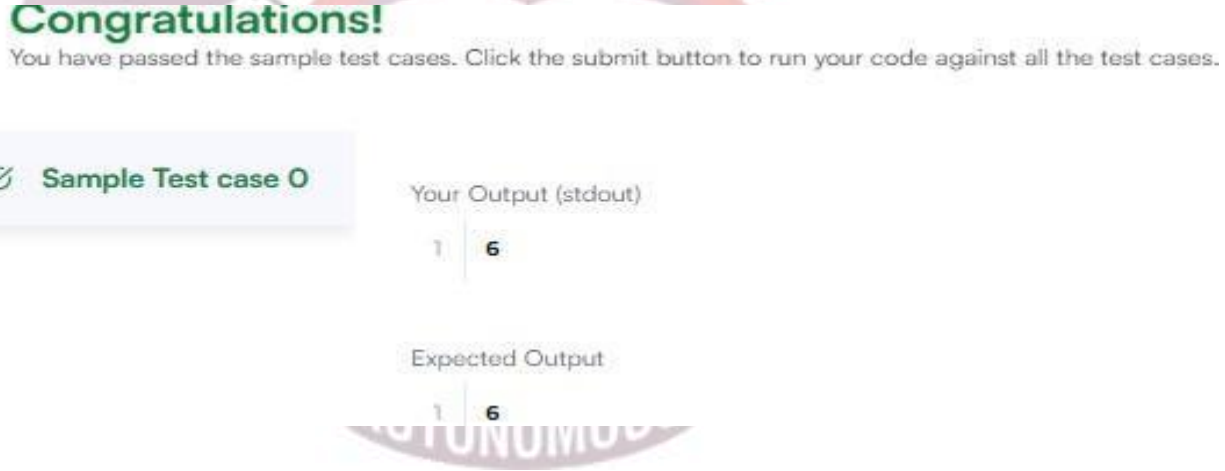
INPUT

SELECT COUNT(*)

FROM CITY

WHERE POPULATION > 100000;

OUTPUT



Revising Aggregations - The Sum Function

Query the total population of all cities in **CITY** where *District* is **California**.

Input Format

The **CITY** table is described as follows:

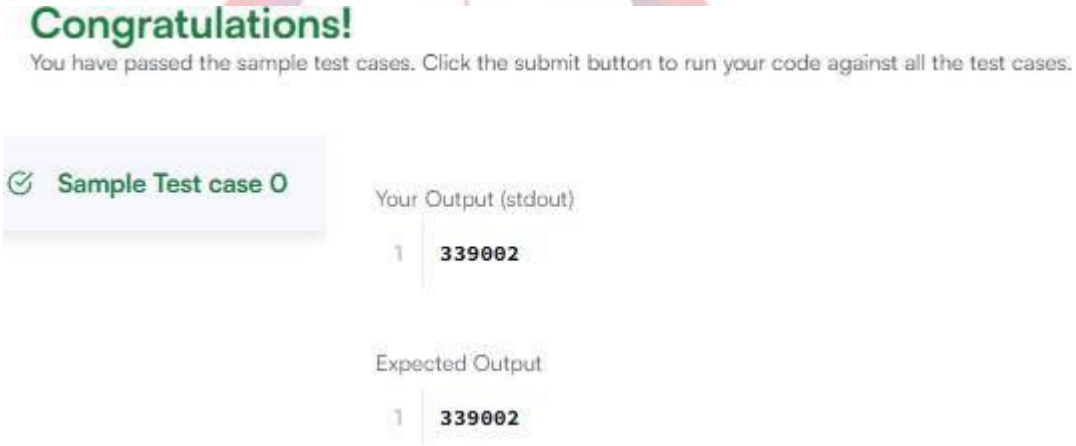
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT SUM(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT



Revising Aggregations – Averages

Query theaverage populationn of all cities in **CITY** where *District* is **California**.

Input Format

The **CITY** table is described as follows:

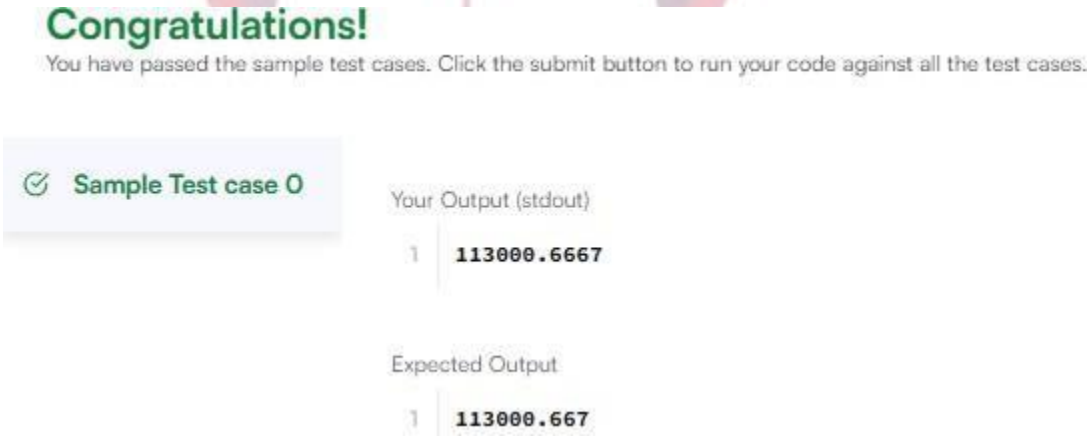
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT AVG(POPULATION)
FROM CITY
WHERE DISTRICT = 'California';
```

OUTPUT



Average Population

Query theaverage population forall cities in **CITY**, rounded *down* to the nearest integer.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT FLOOR(AVG(POPULATION))
FROM CITY;
```

OUTPUT



Japan Population

Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT SUM(POPULATION)
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Population Density Difference

Query the difference between the maximum and minimum populations in **CITY**.

Input Format

The **CITY** table is described as follows:

CITY

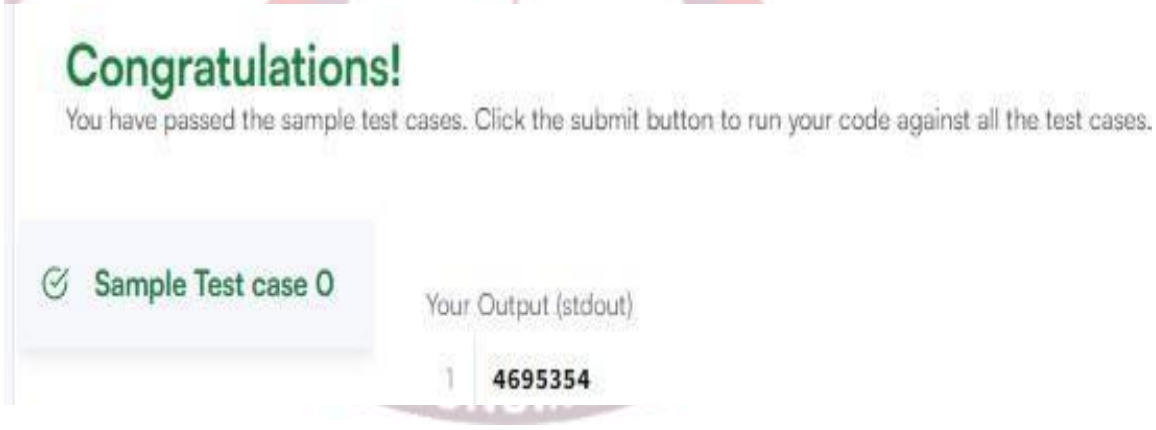
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

SELECT MAX(POPULATION) -

MIN(POPULATION) FROM CITY;

OUTPUT



The Blunder

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the

difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

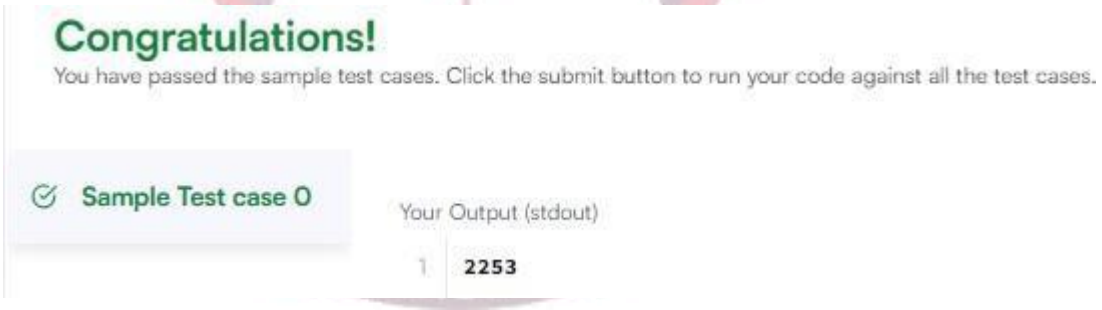
The **EMPLOYEES** table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

INPUT

```
SELECT CEIL(AVG(Salary) - AVG(REPLACE(Salary, '0', '')))
FROM EMPLOYEES;
```

OUTPUT



Top Earners

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

INPUT

```
SELECT (months * salary) AS total_earnings, COUNT(*)
FROM Employee
GROUP BY total_earnings
ORDER BY total_earnings DESC
LIMIT 1;
```

OUTPUT



Contest Leaderboard

You did such a great job helping Julia with her last coding contest challengethat she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

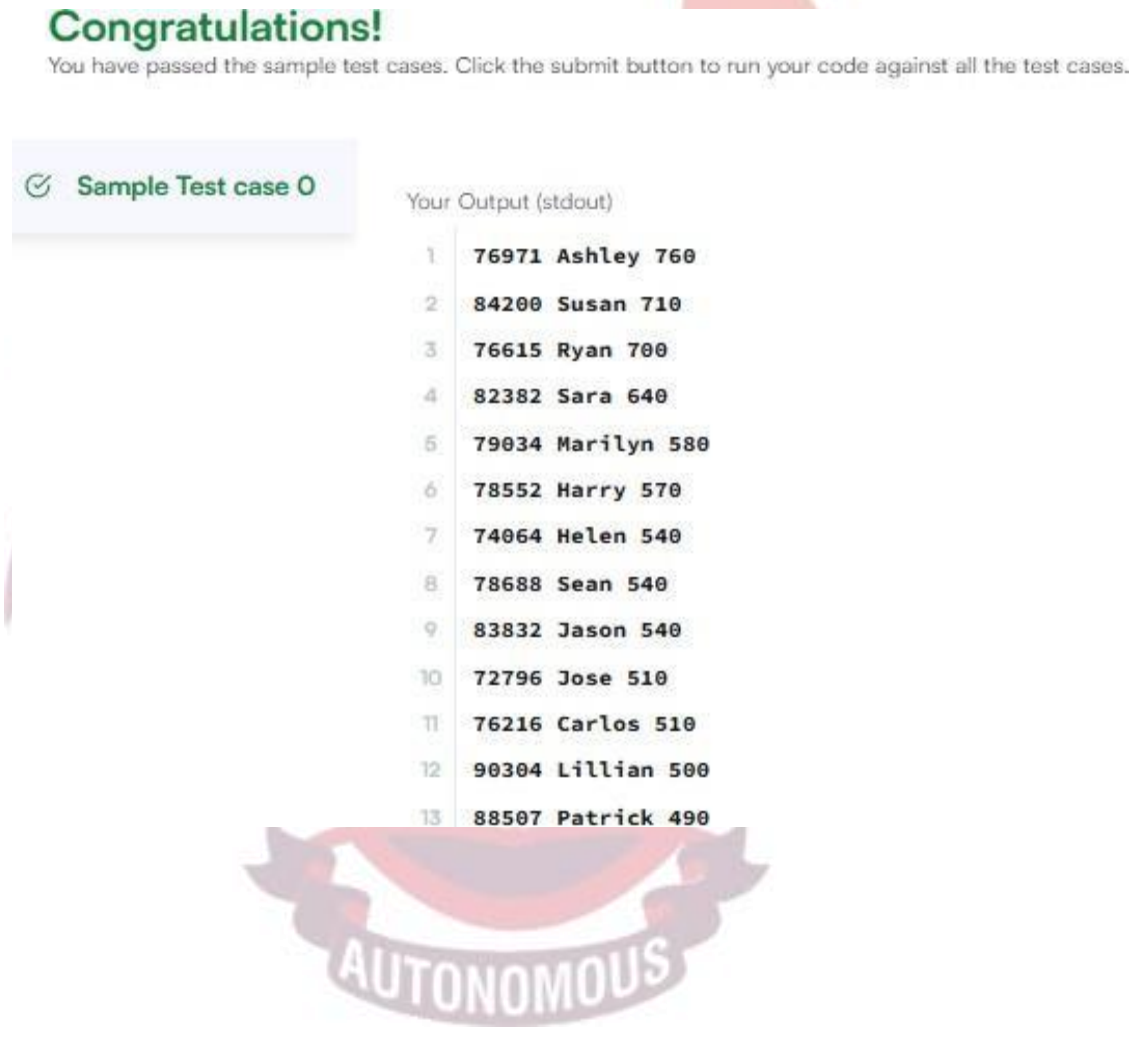
Column	Type
hacker_id	Integer
name	String

INPUT

```
SELECT
   h.hacker_id,
   h.name,
   SUM(m.max_score) AS total_score
FROM Hackers h
JOIN (
   SELECT
      hacker_id,
      challenge_id,
      MAX(score) AS max_score
   FROM Submissions
```

```
    GROUP BY hacker_id, challenge_id
) m ON h.hacker_id = m.hacker_id GROUP
BY h.hacker_id, h.name HAVING
total_score > 0
ORDER BY total_score DESC, h.hacker_id ASC;
```

OUTPUT



AUTONOMOUS

Population Census

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

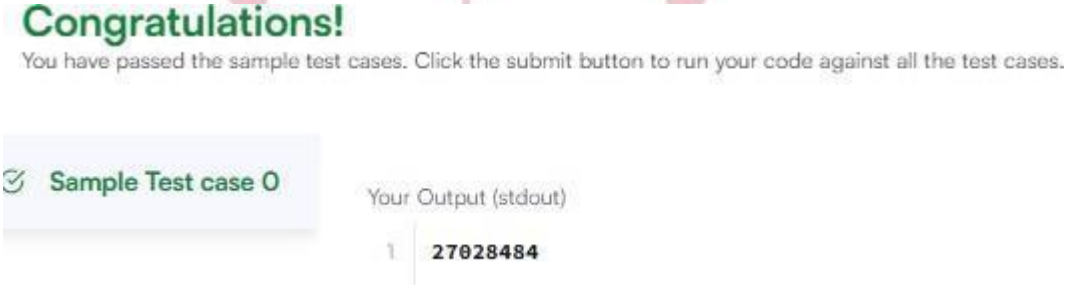
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

INPUT

```
SELECT SUM(c.POPULATION)
FROM CITY c
JOIN COUNTRY co ON c.COUNTRYCODE =
co.CODE WHERE co.CONTINENT = 'Asia';
```

OUTPUT



Challenges

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, andthe total number of challenges created by each student.

Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

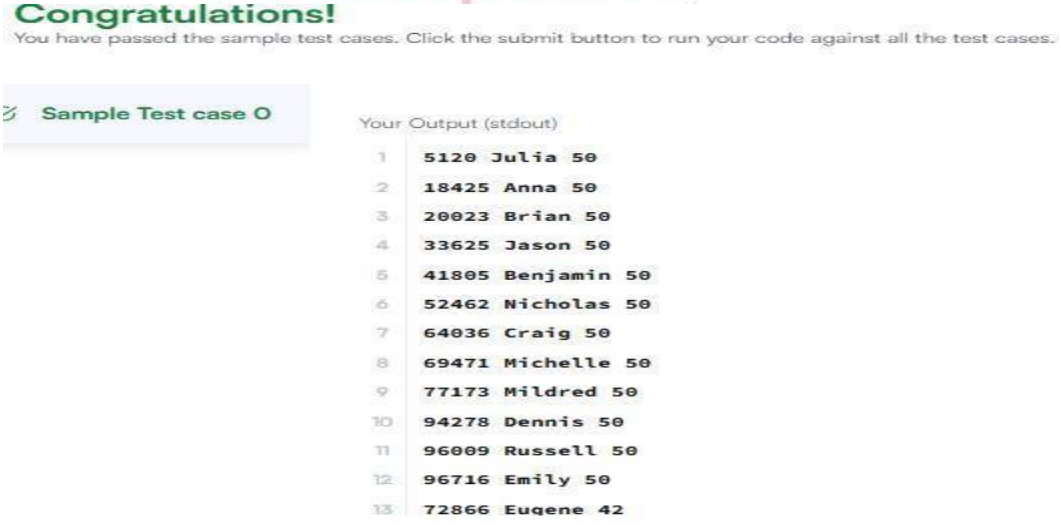
Column	Type
hacker_id	Integer
name	String

INPUT

```
WITH ChallengeCounts AS (
  SELECT
    h.hacker_id,
    h.name,
    COUNT(c.challenge_id) AS total_challenges
  FROM Hackers h
  JOIN Challenges c ON h.hacker_id = c.hacker_id
  GROUP BY h.hacker_id, h.name
)
```

```
SELECT hacker_id, name, total_challenges
FROM ChallengeCounts
WHERE total_challenges = (
    -- Includehackers who created the maximum number of challenges SELECT
    MAX(total_challenges)
    FROM ChallengeCounts
)
OR total_challenges IN (
    -- Includehackers whose challenge count is unique (no ties) SELECT
    total_challenges
    FROM ChallengeCounts
    GROUP BY total_challenges
    HAVING COUNT(*) = 1
)
ORDER BY total_challenges DESC, hacker_id ASC;
```

OUTPUT



SQL Project Planning

You are given a table, *Projects*, containingthree columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	Type
Task_ID	Integer
Start_Date	Date
End_Date	Date

If the *End_Date* of the tasks are consecutive, then theyare part of the same project. Samantha is interested in finding the total number of different projects completed.

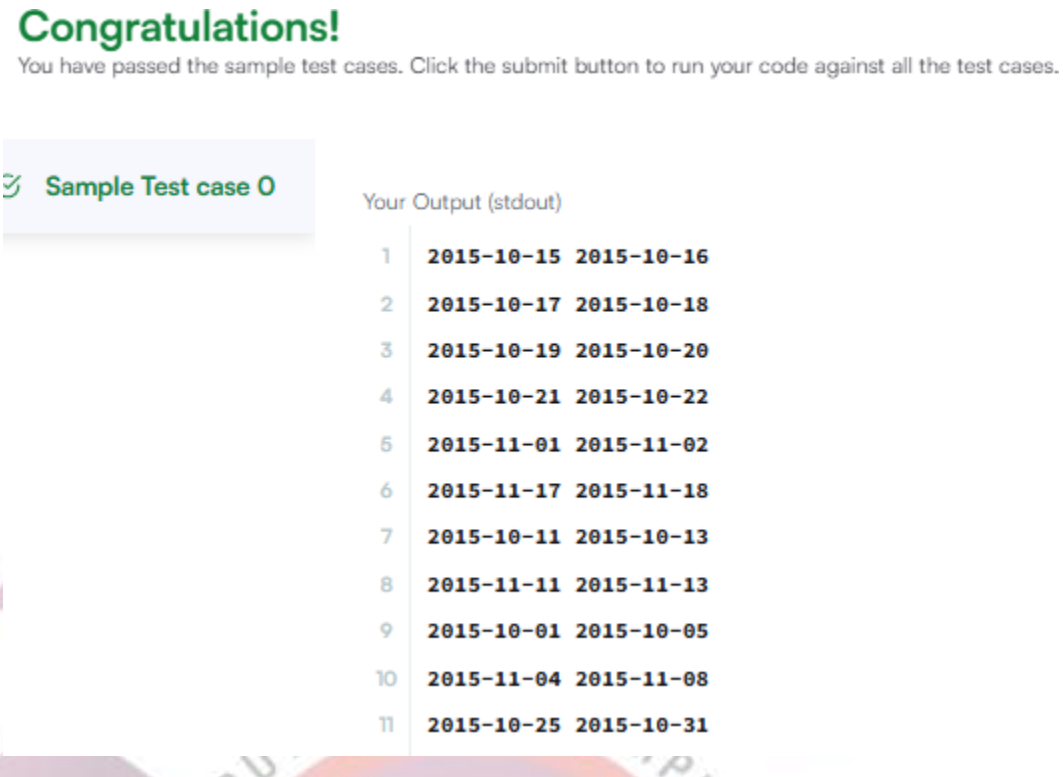
Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

INPUT

```
SELECT Start_Date, MIN(End_Date)
FROM (
  SELECTStart_Date
  FROM Projects
  WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)
) s
JOIN (
  SELECT End_Date
  FROM Projects
  WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)
) e
ON s.Start_Date < e.End_Date
GROUP BY s.Start_Date
```

ORDER BY DATEDIFF(MIN(e.End_Date), s.Start_Date) ASC, s.Start_Date ASC;

OUTPUT



AUTONOMOUS

Placements

You are given three tables: Students, Friends and Packages. Students contains two columns: ID and Name. Friends contains two columns: ID and Friend_ID (ID of the ONLY best friend). Packages containstwo columns: ID and Salary (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

Sample Input

ID	Friend_ID
1	2
2	3
3	4
4	1

Friends

ID	Name
1	Ashley
2	Samantha
3	Julia
4	Scarlet

Students

ID	Salary
1	15.20
2	10.06
3	11.55
4	12.12

Packages

Sample Output

Samantha Julia

Scarlet

Explanation

See the following table:

ID	1	2	3	4
Name	Ashley	Samantha	Julia	Scarlet
Salary	15.20	10.06	11.55	12.12
Friend ID	2	3	4	1
Friend Salary	10.06	11.55	12.12	15.20

Now,

- *Samantha's* best friend got offered a higher salary than her at 11.55
- *Julia's* best friend got offered a higher salary than her at 12.12
- *Scarlet's* best friend got offered a higher salary than her at 15.2
- *Ashley's* best friend did NOT get offered a higher salary than her

The name output, when ordered by the salary offered to their friends, will be:

- *Samantha*
- *Julia*

INPUT

```
SELECT S.NAME
FROM STUDENTS
S
JOIN FRIENDS F ON S.ID = F.ID JOIN
PACKAGES P1 ON S.ID = P1.ID
JOIN PACKAGES P2 ON F.FRIEND_ID = P2.ID
WHERE P2.SALARY > P1.SALARY
ORDER BY P2.SALARY;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Symmetric Pairs

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	Type
X	Integer
Y	Integer

Two pairs *(X1, Y1)* and *(X2, Y2)* are said to be *symmetric pairs* if $X1 = Y2$ and $X2 = Y1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X1 \leq Y1$.

Sample Input

X	Y
20	20
20	20
20	21
23	22
22	23
21	20

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

Sample Output

20 20
20 21
22 23

INPUT

```
SELECT f1.X, f1.Y FROM Functions f1
INNER JOIN Functions f2 ON f1.X = f2.Y AND f1.Y = f2.X GROUP
BY f1.X, f1.Y
HAVING f1.X < f1.Y OR COUNT(f1.X) > 1
ORDER BY f1.X;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

15 Days of Learning SQL

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

Sample Output

```
2016-03-01  4  20703  Angela
2016-03-02  2  79722  Michael
2016-03-03  2  20703  Angela
2016-03-04  2  20703  Angela
```



Sample Output

2016-03-01 4 20703 Angela
2016-03-02 4 79722 Michael
2016-03-03 2 20703 Angela
2016-03-04 2 20703 Angela
2016-03-05 1 36396 Frank
2016-03-06 1 20703 Angela

INPUT

```
SELECT SUBMISSION_DATE,
(SELECT COUNT(DISTINCT
HACKER_ID) FROM SUBMISSIONS
S2
WHERE S2.SUBMISSION_DATE = S1.SUBMISSION_DATE
AND (SELECT COUNT(DISTINCT S3.SUBMISSION_DATE)
FROM SUBMISSIONS S3 WHERE S3.HACKER_ID = S2.HACKER_ID AND
S3.SUBMISSION_DATE
< S1.SUBMISSION_DATE) = DATEDIFF(S1.SUBMISSION_DATE , '2016-03-01')),
(SELECT HACKER_ID FROM SUBMISSIONS S2 WHERE S2.SUBMISSION_DATE =
S1.SUBMISSION_DATE
GROUP BY HACKER_ID ORDER BY COUNT(SUBMISSION_ID) DESC, HACKER_ID
LIMIT 1) AS TMP,
(SELECT NAME FROM HACKERS WHERE HACKER_ID
= TMP) FROM
(SELECT DISTINCT SUBMISSION_DATE FROM

SUBMISSIONS) S1 GROUP BY SUBMISSION_DATE;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code ag

Sample Test case 0

Your Output (stdout)

```
2016-03-01 112 81314 Denise
2016-03-02 59 39091 Ruby
2016-03-03 51 18105 Roy
2016-03-04 49 533 Patrick
2016-03-05 49 7891 Stephanie
2016-03-06 49 84307 Evelyn
2016-03-07 35 80682 Deborah
2016-03-08 35 10985 Timothy
2016-03-09 35 31221 Susan
2016-03-10 35 43192 Bobby
2016-03-11 35 3178 Melissa
2016-03-12 35 54967 Kenneth
2016-03-13 35 30061 Julia
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Draw The Triangle 1

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern *P(20)*.

INPUT

```
SET @TEMP := 21;
SELECT REPEAT('* ', @TEMP := @TEMP - 1)
FROM INFORMATION_SCHEMA.TABLES;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the te

Sample Test case 0

Your Output (stdout)

```
1  * * * * * * * * * * * * * * * * * * * *
2  * * * * * * * * * * * * * * * * * * *
3  * * * * * * * * * * * * * * * * * *
4  * * * * * * * * * * * * * * * * *
5  * * * * * * * * * * * * * * * *
6  * * * * * * * * * * * * * * *
7  * * * * * * * * * * * * * *
8  * * * * * * * * * * * * *
9  * * * * * * * * * * * *
10 * * * * * * * * * * *
11 * * * * * * * * * *
12 * * * * * * * * *
13 * * * * * * * *
```

Draw The Triangle 2

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern *P(20)*.

INPUT

```
SET @number = 0;
SELECT REPEAT('* ', @number := @number + 1)
FROM information_schema.tables
WHERE @number < 20;
```

OUTPUT

Congratulations!

You have passed the sample test cases. Click the submit button to run your cod

Sample Test case 0

Your Output (stdout)

```
1  *
2  * *
3  * * *
4  * * * *
5  * * * * *
6  * * * * * *
7  * * * * * * *
8  * * * * * * * *
9  * * * * * * * * *
10 * * * * * * * * * *
11 * * * * * * * * * * *
12 * * * * * * * * * * * *
13 * * * * * * * * * * * * *
```

Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

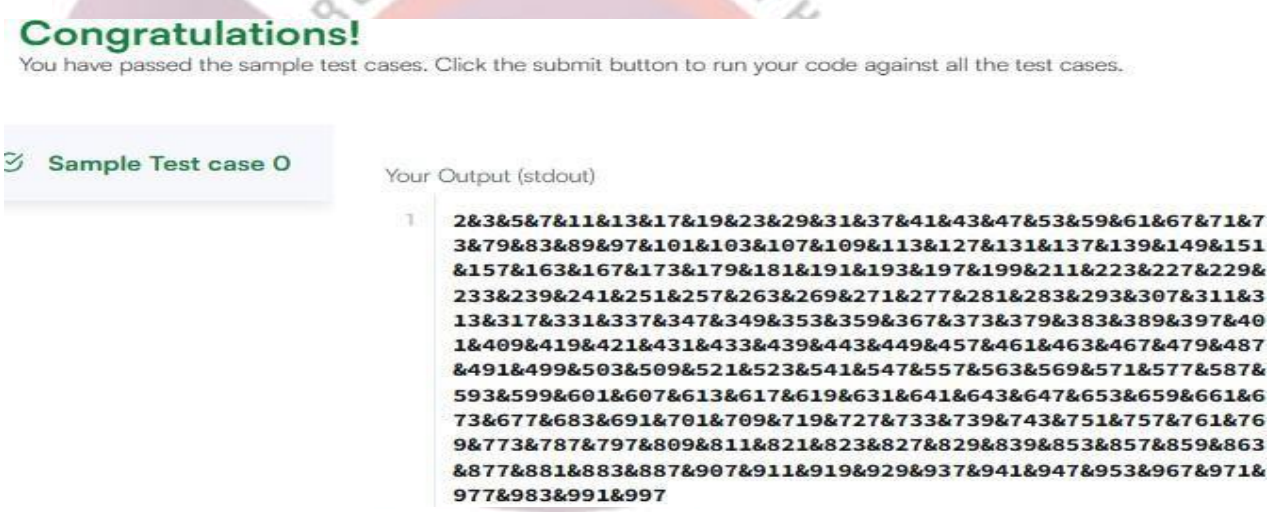
Print Prime Numbers

For example, the output for all prime numbers would be:
2&3&5&7

INPUT

```
WITH RECURSIVE nums AS (
SELECT 2 AS n
UNION ALL
SELECT n + 1 FROM nums WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM nums p
WHERE NOT EXISTS (
SELECT 1 FROM nums d WHERE d.n <= SQRT(p.n) AND d.n > 1 AND p.n % d.n = 0
);
```

OUTPUT



Interviews

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.
- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

Column	Type
college_id	Integer
contest_id	Integer

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Sample Input

Contests Table: *Colleges* Table:

college_id	contest_id
11219	66406
32473	66556
56685	94828

challenge_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

challenge_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

Sample Output

66406 17973 Rose 111 39 156 56
66556 79153 Amngela 0 0 11 10
94828 80275 Frank 150 38 41 15

Explanation

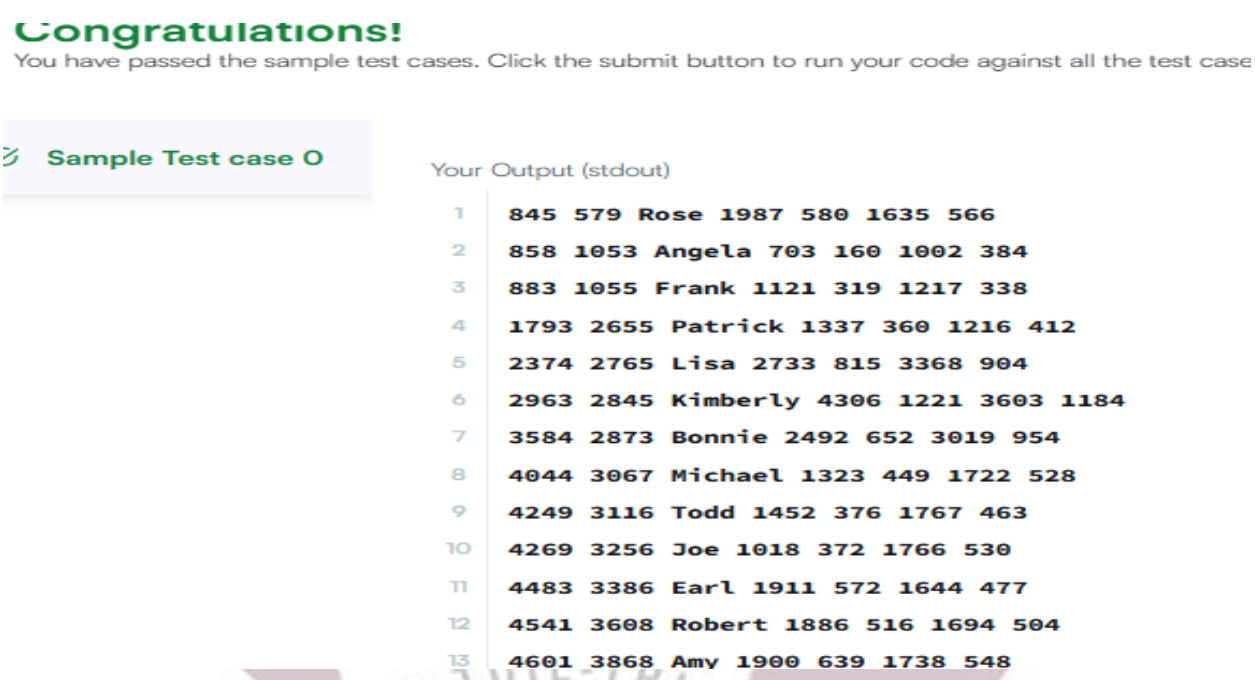
The contest is used in the college . In this college , challenges and are asked, so from the *view* and *submission* stats:

- Sum of total submissions
- Sum of total accepted submissions
- Sum of total views
- Sum of total unique views

INPUT

```
SELECT t1.contest_id, t1.hacker_id, t1.name,
SUM(ts) as ts, SUM(tas) as tas, SUM(tv) as tv, SUM(tuv) as tuv FROM
CONTESTS AS t1
LEFT JOIN COLLEGES AS t2
ON t1.contest_id = t2.contest_id LEFT
JOIN CHALLENGES AS t3
ON t2.college_id = t3.college_id LEFT
JOIN (SELECT challenge_id,
SUM(total_views) AS tv, SUM(total_unique_views) as tuv FROM
View_Stats
GROUP BY challenge_id) AS t4
ON t3.challenge_id = t4.challenge_id LEFT
JOIN (SELECT challenge_id,
SUM(total_submissions) AS ts, SUM(total_accepted_submissions) as tas
FROM Submission_Stats
GROUP BY challenge_id) as t5
ON t3.challenge_id = t5.challenge_id
GROUP BY t1.contest_id, t1.hacker_id, t1.name HAVING
(ts + tas + tv + tuv > 0)
ORDER BY t1.contest_id;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

CODE CHEF

All Products

Task

Write a query to output Productstable.

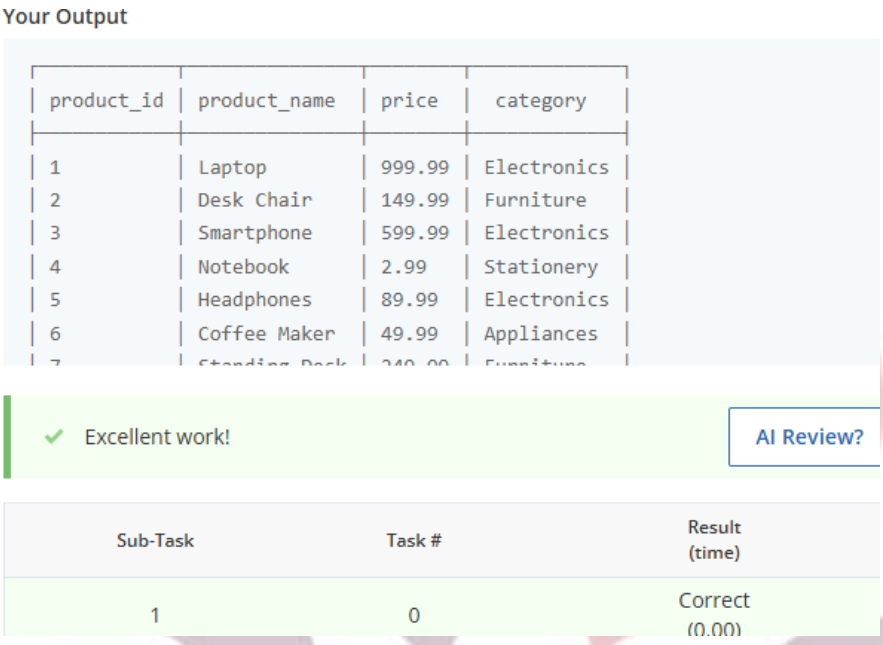
Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT:

SELECT * FROM Products;

OUTPUT:



High Price of Products Task

Write a query to find all product_nameand categorythat have a price greater than 100.00 from the Products table.

Table name: **Products**

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances

7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

Expected columns

- product_name
- category

INPUT:

SELECT product_name, category FROM Products WHERE price > 100.00;

OUTPUT:



Average Salary

Task

- Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

employee_id	company_name	salary
1	TeachCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

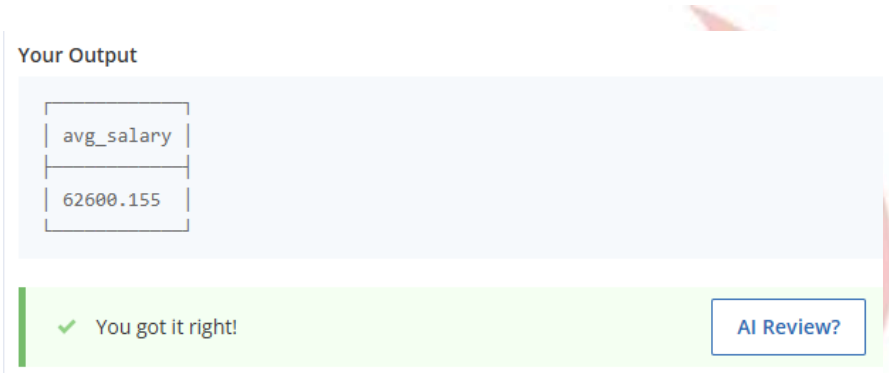
Expected column

- avg_salary

INPUT :

```
SELECT AVG(salary) AS avg_salary FROM Works;
```

OUTPUT:



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Locate People

Task

- Write a query to retrieve the department_nameand locationof departments located in a city that starts with 'S'.

Table name: **departments**

department_id	department_name	Location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

Expected column

- department_name

- location

INPUT:

SELECT department_name, location FROM departments WHERE location LIKE 'S%';

OUTPUT:

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Distinct Companies

Task

Write a query to select all the distinct companies (company_name) in the Workstable.
Table name: **Works**

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

Expected Column

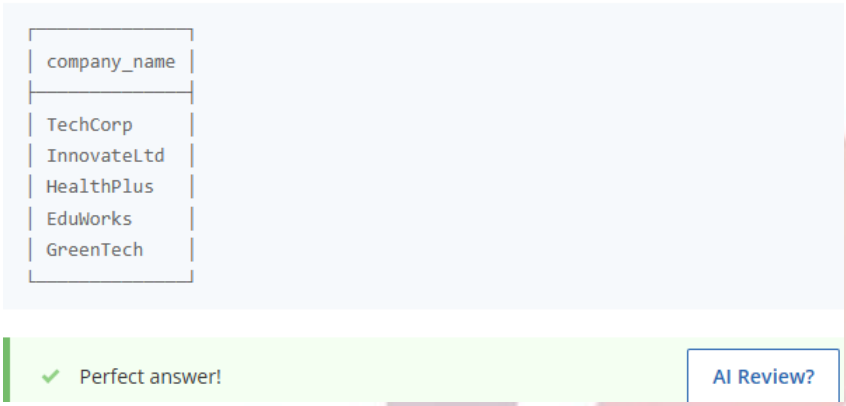
- company_name

INPUT:

SELECT DISTINCT company_name FROM Works;

OUTPUT:

Your Output



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Fiction Collection Size

Task

Write a query to find the total count of books whose genre is Fiction. Note: Output column name should be fiction_count.

Table name: **Books**

id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869

9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

Expected column

- fiction_count

INPUT:

SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';

OUTPUT:

Your Output

```
┌─────────────┐
│ fiction_count │
├─────────────┤
│ 7           │
└─────────────┘
```

✔ Excellent work! AI Review?



List of Movies with Ratings

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: **Cinema**

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

Expected column

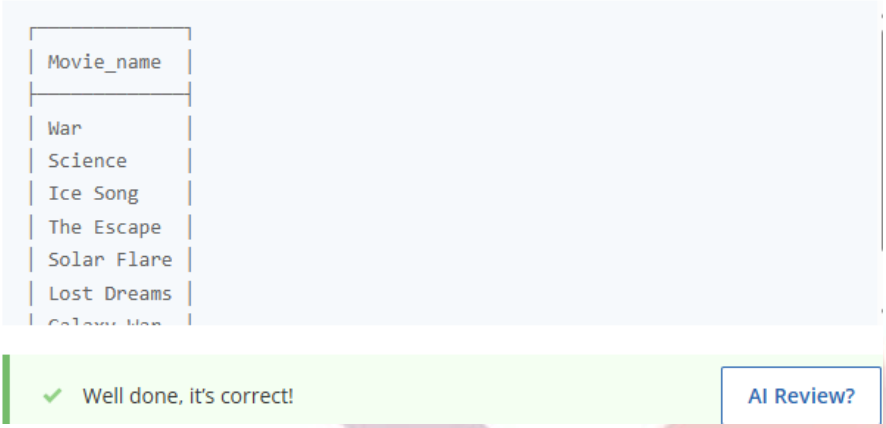
- Movie_name

INPUT:

```
SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;
```

OUTPUT:

Your Output



Well done, it's correct! AI Review?

Handling NULL Values

Task

Write a query to retrieve book_id, title, author and published_year of the books which have NULL rating for their books.

Table name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

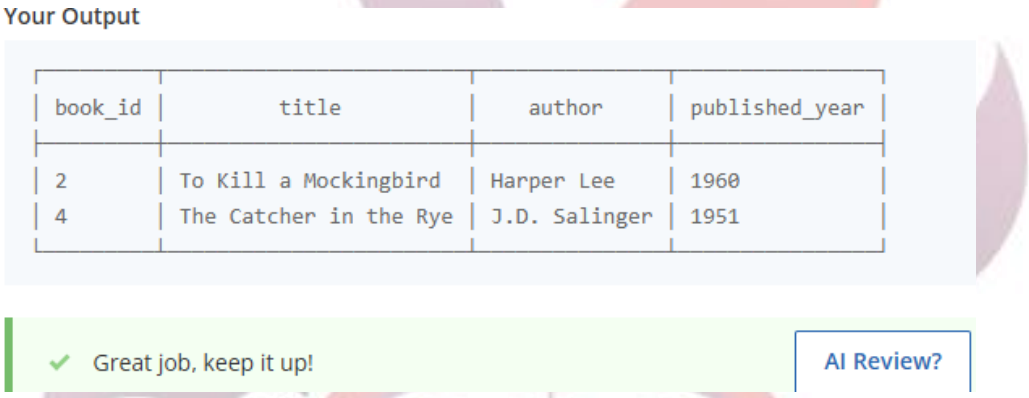
Expected column

- book_id
- title
- author
- published_year

INPUT:

SELECT book_id, title, author, published_year FROM Library WHERE rating IS NULL;

OUTPUT:



Salary of Employees

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in descending order

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000

2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected column

- employee_name
- company
- salary

INPUT:

SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;

OUTPUT:



Department of Each Employee

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected Column

- department

- total_employees

INPUT:

SELECT department, COUNT(*) AS total_employees FROM Employees GROUP BY department;

OUTPUT:



Article views

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in ascending order.

Table name: Views

article_id	author_id	author_name	viewer_id	view_count	publication_name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

Expected column

- author_id
- author_name
- publication_name

INPUT:

SELECT DISTINCT author_id, author_name, publication_name FROM Views WHERE view_count = 0 ORDER BY author_id ASC;

OUTPU

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

Well done, it's correct!

AI Review?

Player Performance Insights

Task

Write a query to find the names of the top 3 distinct players by highest score who have won matches, including their scores. Table

1: Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: Matches

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18

105	Charlie	Eve	Charlie	2024-01-19

INPUT:

```
SELECT DISTINCT player_name, score
FROM Players JOIN Matches WHERE winner=player_name
ORDER BY score DESC
LIMIT 3;
```

OUTPUT:



Player Details Task

Write a query to retrieve the details of the last five matches played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **Players**and **Matches**.

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

Expected Output Columns:

- match_id
- player_1
- player_2
- winner
- match_date
- score

AUTONOMOUS

INPUT:

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score FROM Matches
m JOIN Players p ON m.winner = p.player_name ORDER BY m.match_date DESC LIMIT 5;
```

OUTPUT:

Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

✓ Excellent work!

AI Review?

